

Learn
Explore
Practice
Automate !

LINUX / UNIX
FOR ECE, VLSI & EDA ENGINEERS

Command Today Better Tomorrow !

- ✔ Build Skills
- ✔ Work Efficiently
- ✔ Automate Tasks
- ✔ Use in EDA Flow
- ✔ Grow as an Engineer !

{ Learn
{ Practice
{ Apply
{ Succeed }

Terminal Commands → File System & Directories → Users, Permissions → Shell Scripting → Text Processing (Grep, Sed, Awk) → Processes & Job Management → Networking & Utilities → Productive Engineer !

SAME OS • MANY POSSIBILITIES • FROM BASICS TO EDA AUTOMATION



LINUX

POWERING ECE / VLSI / EDA FLOW

More than Commands A Powerful Engineering Skill !



Linux is the backbone of EDA tools, servers and high-performance computing.
Master Linux to work efficiently, automate tasks and excel in VLSI design flow.

WHY LINUX ?

- ✔ Open source & powerful
- ✔ Widely used in EDA industry
- ✔ Stable, secure and flexible
- ✔ Essential for servers & HPC
- ✔ Scripting & automation
- ✔ Helps in design, verification and silicon bring-up

Linux Skills = Better Productivity = Stronger Career in VLSI/EDA

COMMON LINUX DISTRIBUTIONS

- Ubuntu (Beginner Friendly)
- CentOS (Stable)
- Red Hat (Enterprise)
- Debian (Lightweight)
- Fedora (Modern)
- ... And More...

LINUX IN EDA FLOW

- ✔ Running EDA tools (Synopsys, Cadence, etc.)
- ✔ Scripting and automation (Tcl, Shell)
- ✔ Handling large design databases
- ✔ Managing compute servers and jobs
- ✔ Parsing logs and reports
- ✔ Setting up design environments
- ✔ Improving productivity and reproducibility

KEY AREAS COVERED

1	Linux Installation & Development Environment
2	OS & Linux Fundamentals
3	Linux Architecture & Linux vs Windows
4	Terminal, Shell & Shell Scripting
5	Filesystem & File/Directory Management
6	Permissions, Users & Groups
7	File Operations, Navigation & Editors
8	Shell Configuration & Environment
9	Metacharacters & Grep
10	SED
11	AWK & Text Processing
12	Pipes, Filters & Sorting
13	VI / GVIM
14	Servers, Processes & Job Management
15	Communication, Networking & Basic Utilities
16	CTAGS & Common User Commands
17	Archives, Compression & Links
18	Shell Programming & Linux Directory Structure
19	User Administration & System Performance
20	File Compression
21	Linux Information Commands
22	Network & User Communication
23	Programming Utilities
24	Miscellaneous Linux Commands

Step by Step → Practice with Examples → Use in Real EDA Flow

ESSENTIAL COMMAND CATEGORIES

- File & Directory Commands (ls, cd, cp, mv, rm)
- Permission Commands (chmod, chown, chgrp)
- User Management (useradd, passwd, su)
- Text Processing (grep, sed, awk, sort, uniq)
- Process & Job Control (ps, top, kill, jobs)
- Networking & Communication (ssh, scp, ping, mail)
- Compression & Archives (tar, gzip, zip)
- System Information (df, du, free, uname)
- Shell Scripting (variables, loops, functions)

LEARNING APPROACH

1. Understand concepts with simple examples
2. Practice commands in terminal
3. Try hands-on exercises and mini projects
4. Build your own scripts and utilities
5. Apply skills in real EDA/design environment

OUTCOME

- ✔ Confident in using Linux for daily work
- ✔ Write and debug shell scripts
- ✔ Handle files, processes and servers efficiently
- ✔ Automate repetitive EDA tasks
- ✔ Be job-ready for VLSI/EDA industry

Same Commands
Multiple Tools
Bigger Opportunities !

Author : Konda Tharun Vivekananda

"Linux Today
Leads to a Brighter VLSI Tomorrow !"

Practice = Skill = Opportunity = Growth



Think Learn Practice Automate !

INDEX

- Commands
- Scripting
- EDA Environment
- Productivity
- From Basics to Practice !

Page No. i

Small Commands Big Possibilities !

LINUX / UNIX FOR ECE, VLSI & EDA
– SYLLABUS INDEX –

Module No.	Module Title	Topics Covered	Pages
1	Linux Installation & Development Environment	• Installing Linux, Virtual Machines (VMware), Cygwin, Perl, Python, Tcl setup for EDA	1
2	OS & Linux Fundamentals	• What is OS, types, interfaces, files & directories, permissions, sending mails, submitting jobs	2-4
3	Linux Architecture & Linux vs Windows	• Linux working, aspects, kernel, shell, login & password, SU, users, changing password, case sensitivity	5-8
4	Terminal, Shell & Shell Scripting	• Terminal, shell types, shell scripting introduction, command structure, comments, basic scripts	9-11
5	Filesystem & File/Directory Management	• Linux directory structure, files, directories, navigation, navigation, absolute paths	12-18
6	Permissions, Users & Groups	• File permissions, chmod, chown, users, groups, user management, umask	19-22
7	File Operations, Navigation & Editors	• Common file operations, cp, mv, rm, find, locate, viewing files, text editors overview	23-26
8	Shell Configuration & Environment	• Environment variables, .bashrc, .profile, PATH, aliases, shell customization	27-31
9	Metacharacters & Grep	• Shell metacharacters, regular expressions, grep, egrep, fgrep, pattern matching	32-33
10	SED	• Stream editor, sed commands, search/replace, editing text in files	34-36
11	AWK & Text Processing	• AWK fundamentals, patterns, actions, fields, text processing examples	37-39
12	Pipes, Filters & Sorting	• Pipe concept, filters, sort, uniq, cut, paste, tr, wc and related commands	40
13	VI / GVIM	• VI editor basics, modes, commands, configuration, GVIM usage	41-43
14	Servers, Processes & Job Management	• Processes, ps, top, kill, jobs, bg, fg, nohup, cron, at, scheduling jobs	44-45
15	Communication, Networking & Basic Utilities	• Networking basics, ssh, scp, ping, netstat, ftp, wget, curl, mail, aliases, useful utilities	46-47
16	CTAGS & Common User Commands	• CTAGS, source code navigation, man, which, whatis, finger, history, clear, date, cal	48-49
17	Archives, Compression & Links	• tar, gzip, gunzip, zip, unzip, bzip2, links, symbolic vs hard links	50-51
18	Shell Programming & Linux Directory Structure	• Writing shell scripts, functions, command line arguments, Linux directory hierarchy in depth	52
19	User Administration & System Performance	• User administration, groups, sudo, system monitoring, CPU, memory, disk usage, performance tools	53
20	File Compression	• Compression tools, tar options, gzip, bzip2, xz, compression best practices	54
21	Linux Information Commands	• System information, uname, hostname, df, du, free, lscpu, lshw, lsb_release	55
22	Network & User Communication	• Mail, wall, write, mesg, talk, social commands, remote access utilities	56
23	Programming Utilities	• Compilers, gcc basics, make, gdb, diff, patch, other development utilities	57
24	Miscellaneous Linux Commands	• Miscellaneous but important commands, shortcuts, tips & tricks for EDA engineers	58



Author : Konda Tharun Vivekananda

" Command Today Better Tomorrow "

Note :
- Pages indicate where the topics start in this material.
- Use this index to quickly navigate to any topic.
- Practice along with examples for better understanding.

Linux Powers Ideas !

Linux
Learn
Practice
Explore ...

LINUX FOR ECE / VLSI & EDA

Module 1

Linux Installation & Development Environment

```
$ ls    ~/project
$PATH   .sh
```

Build • Run • Learn

1. INSTALLING LINUX

1.1 Why Linux ?

- Linux is open-source, stable and widely used in EDA / Semiconductor industry.
- Most EDA tools (Synopsys, Cadence, Mentor) are developed and certified on Linux.
- Provides powerful command line, scripting support and multi-user environment.
- Used for design, simulation, synthesis, place & route, signoff, data analysis etc.
- Can be installed on physical machine or as a virtual machine (preferred for beginners).

1.2 Installation Options

Linux can be used in different ways :

1. Dual Boot (Linux + Windows)
2. Install on a separate system
3. Use a Virtual Machine (VMware)
4. Use a Windows based Linux environment (Cygwin)

1.3 a) Installing Linux using VMware

- VMware is a virtualization software which allows us to run Linux as a virtual machine inside Windows without affecting the existing OS.

Steps for Installation :

1. Download VMware Workstation / Player (from VMware website).
2. Download Linux ISO file (e.g. Ubuntu, CentOS, RHEL, etc).
3. Create a new Virtual Machine.
4. Select the Linux ISO file.
5. Allocate system resources (CPU, RAM, Disk).
6. Install Linux (follow on-screen instructions).
7. Login and configure basic settings.

VMware Architecture



Advantages :

- Safe and easy to install
- No effect on existing Windows OS
- Multiple Linux versions can be created
- Useful for learning and EDA tool evaluation

Limitations :

- Requires more RAM and disk space
- Performance is lower than direct installation

1.4 Basic System Requirements (for VM)

Component	Recommended
Host OS	Windows 10 / 11 (64-bit)
RAM	8 GB or higher (16 GB preferred)
CPU	Intel i5 / i7 or above (with virtualization)
Disk Space	40 GB or more for Linux VM
Linux OS	Ubuntu / CentOS / RHEL (64-bit)

1.5 b) Cygwin - Perl, Python, TCL

- Cygwin is a Linux-like environment for Windows.
- Provides a Unix-like command line with many common Linux utilities.
- Useful when we cannot install full Linux or VMware.
- Supports scripting languages like Perl, Python and TCL, which are widely used in EDA flows.

How to Install Cygwin ?

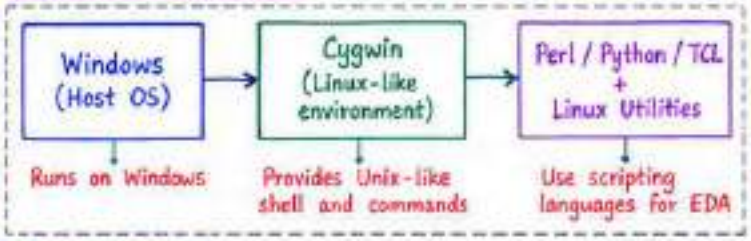
1. Download Cygwin setup from https://www.cygwin.com/
2. Run setup.exe
3. Select packages to install (e.g. gcc, make, perl, python, tcl)
4. Complete installation and open Cygwin terminal.

Using Perl, Python and TCL in Cygwin

- During installation, select the required packages.
- After installation, verify using :

```
$ perl -v          # check Perl version
$ python --version # check Python version
$ tclsh            # start TCL shell
```

Cygwin Environment



Advantages :

- Easy to install and lightweight
- Provides many Linux commands on Windows
- Supports Perl, Python, TCL (useful for EDA scripting)
- No need for a separate virtual machine

Limitations :

- Not a full Linux system
- Limited compatibility with some EDA tools
- Performance is lower compared to native Linux

KEY POINT

- Use VMware if you want a complete Linux environment for EDA tool installation and practice.
- Use Cygwin if you only need Linux-like commands and scripting (Perl / Python / TCL) on Windows.
- Linux knowledge is essential for EDA tools and VLSI design flows.

VIVA / INTERVIEW POINT

Q1. Why is Linux used in EDA industry?
→ Most EDA tools are developed and certified on Linux.

Q2. What is the advantage of using VMware?
→ Allows running Linux without affecting host OS.

Q3. What is Cygwin?
→ A Linux-like environment for Windows with support for Unix commands and scripting tools (Perl, Python, TCL).



Think
Learn
Use Linux
In EDA ...

LINUX FOR ECE / VLSI & EDA

Module 2

OS & Linux Fundamentals

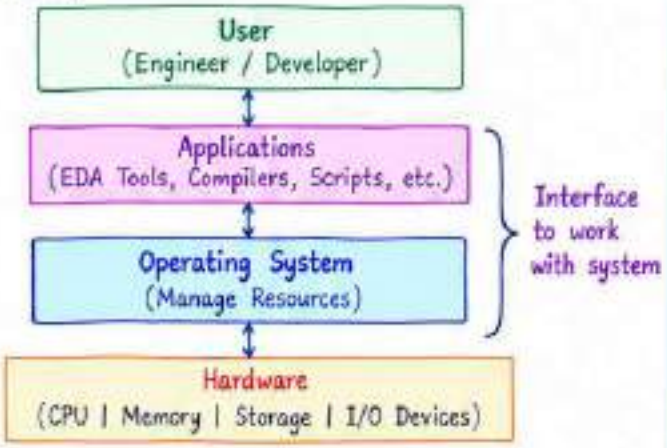
```
$ ls      ~/project
$ cd      /home/user
$ echo $PATH
```

Linux → Tools → VLSI

2. WHAT IS OS ?

2.1 What is an Operating System (OS) ?

- An Operating System (OS) is system software that acts as an interface between the user and the computer hardware.
- It manages hardware resources (CPU, memory, storage, I/O devices) and provides services to run application programs.
- Examples : Linux, Windows, macOS, UNIX.
- In EDA / VLSI, Linux is the most widely used OS due to its stability, scripting support and multi-user capability.



2.2 Interfaces to Work in Linux

We interact with Linux using different interfaces :

① **Command Line Interface (CLI)**
- Most common in EDA environments.
- Uses terminal / shell to execute commands.
- Lightweight and powerful.

② **Graphical User Interface (GUI)**
- Desktop environment (e.g. GNOME, KDE).
- Useful for beginners, but less used in EDA.

③ **Remote Interface**
- Accessing Linux servers using ssh (secure shell).
- Common in companies for working on remote compute servers.

Typical Linux Work Interface



Key Point
- OS provides an interface to manage files, permissions, users, devices and run applications.
- In EDA, we mostly work through command line or SSH on Linux servers.
- Good knowledge of OS concepts makes working with EDA tools easier and more efficient.

2.3 Files and Directory Management

- Linux organizes data in a hierarchical filesystem structure.
- Files store data and directories (folders) are used to group files.
- Common commands for file and directory management :

Command	Purpose	Example
pwd	Print current directory	pwd
ls	List files and directories	ls -ltr
cd	Change directory	cd project
mkdir	Create a new directory	mkdir run1
rmdir	Remove an empty directory	rmdir temp
cp	Copy file or directory	cp file1 file2
mv	Move / Rename	mv old.log new.log
rm	Remove file or directory	rm file1
touch	Create an empty file	touch myfile.txt

Directory Structure (Example)

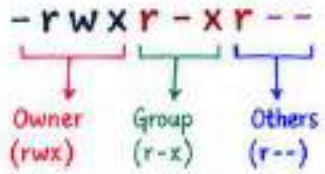
```
/ (root)
├── home
│   └── user
│       ├── project
│       └── reports
├── tools
├── tmp
└── etc
```

Path Types
- Absolute Path : Starts from root '/'
  Example : /home/user/project
- Relative Path : Relative to current directory
  Example : project/run1

2.4 Permissions

- Permissions control who can read, write or execute files/directories.
- Each file has permissions for three categories :
  ① Owner ② Group ③ Others

Permission Representation



Permission Values

r = 4	(read)		
w = 2	(write)		
x = 1	(execute)		
7 = rwx		5 = r-x	
6 = rw-		4 = r--	
0 = ---			

Example :

chmod 755 script.sh → Owner : rwx , Group : r-x , Others : r-x

2.5 Sending Mails

- Used to communicate with team members in Linux environment.
- Common command : mail or mailx

Example :

```
mail -s "Run Completed" user@company.com
echo "Check the reports in /home/user/run1" | mail -s "Log" user@company.com
```

2.6 Submitting Jobs

- In EDA, long running tasks are submitted to compute servers using a job scheduler (e.g. LSF).
- Common command : bsub

Example :

```
bsub -q normal -J my_run -o run.log ./run.sh
```

Flow : User → bsub → LSF → Queue → Execute → Result

? VIVA / INTERVIEW POINT

Q. Why is an Operating System important in EDA ?

→ It provides an interface, manages resources, supports multi-user environment and allows running EDA tools, sending mails and submitting jobs.

LINUX FOR ECE / VLSI & EDA

Module 2

OS & Linux Fundamentals

```
$ ls      ~/project
$ cd      /home/user
$ echo $PATH
```

Learn • Practice • Use

3. LINUX WORKING , ASPECTS , LOGIN AND PASSWORD - SU , USERS , CHANGING PASSWORD , CASE SENSITIVITY

3.1 Linux Working

- Linux is a multi-user, multi-tasking operating system.
- Multiple users can work on the same system simultaneously with their own environments.
- The kernel manages hardware resources (CPU, memory, storage, I/O) and provides services to users.
- Users interact with Linux through a shell using commands.
- Common in EDA for running tools, scripts, managing files, submitting jobs, etc.

Working Flow of Linux



Key Aspects

- Multi-user
- Multi-tasking
- Security (login)
- File management
- User management
- Permission control
- Supports automation & job execution (EDA tools)

3.2 Aspects of Linux (Important for EDA Users)

Aspect	Description
Multi-user	Multiple users can work at the same time.
Multi-tasking	Run many processes simultaneously.
Command line	Powerful shell for automation and scripting.
File system	Organized directory structure to store data.
Security	User authentication, permissions, controlled access.
Stability	Typically runs for long periods without reboot.
Scalability	Used on workstations, servers and compute clusters.
Widely used	Most EDA tools are developed and certified on Linux.

3.3 Login and Password

- To access a Linux system, each user must login with a valid username and password.
- Authentication verifies the user and loads the user's environment (home directory, shell, etc).
- After successful login, a shell prompt is displayed.

Login Process

Enter Username and Password → Authentication by System → If Valid → Shell Prompt

Example (terminal login)

```
login: user1
Password: *******
[user1@server ~]$
```

Failed Login

```
login: user1
Password: ******
Login incorrect
```

3.4 Users in Linux

- Each person has a unique user account.
- Every user has a home directory (usually /home/username).
- Users have different permissions based on their role (normal user / admin).
- User information is stored in files like /etc/passwd (user details).

User Type	Description
Normal User	Can run tools, manage own files, submit jobs.
Admin User (root)	Has full system privileges.
Multiple Users	Common in EDA teams (design, verification, PD, etc).

Useful Commands to See Users

Command	Purpose
who	Shows currently logged in users
w	Shows logged in users with more details
id	Shows user id, group id and groups
cat /etc/passwd	Shows all user accounts
cat /etc/group	Shows all groups

3.5 SU Command (Switch User)

- su (switch user) is used to change from current user to another user, typically to root.
- After running su, you need to enter the target user's password.

Syntax and Example

```
$ su                # switch to root (you will be asked for root password)
$ su username       # switch to another user
$ su -              # switch to root with login environment
$ su - username     # switch to user with login environment
```

Example :

```
[user1@server ~]$ su
Password: *******
[root@server ~]# → Now you are root user
```

To come back to original user:

```
[root@server ~]# exit
                or
[root@server ~]# su - user1
```

3.6 Changing Password

- Each user can change their own password using the passwd command.
- It is recommended to change passwords periodically for security.

Syntax and Example

```
$ passwd            # change current user's password
$ passwd username   # (only root can change other user's password)
```

Example :

```
[user1@server ~]$ passwd
Current password: *******
New password: *******
Retype new password: *******
passwd: password updated successfully
```

Points to Remember

- Password is not displayed while typing.
- Use strong password (letters + numbers + special characters).
- Only root can change another user's password.

3.7 Case Sensitivity in Linux

- Linux is case sensitive, meaning uppercase and lowercase letters are treated differently.
- Examples : file, File and FILE are three different files.
- Commands, directory names, file names and usernames are case sensitive.

Example :

```
[user1@server ~]$ touch file File FILE
[user1@server ~]$ ls
FILE  File  file
```

Note

- Use consistent naming conventions to avoid confusion.
- In EDA scripts and directories, maintain proper case.



LINUX FOR ECE / VLSI & EDA

Module 2

OS & Linux Fundamentals

```
$ ls        ~/project
$ cd        /eda/design
$ echo $PATH
```

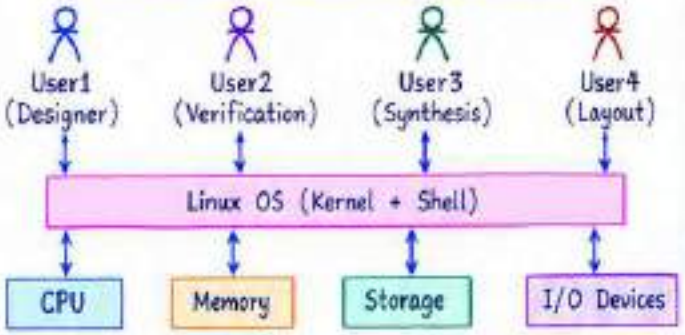
Think Linux → Work Smart

4. LINUX AS MULTI USER AND MULTI TASKING , LINUX OPEN SOURCE.

4.1 Linux as a Multi-User System

- Linux is designed to support multiple users at the same time.
- Each user has a separate account, home directory, environment and permissions.
- Users can work independently without affecting each other.
- The kernel manages user access, resource allocation and process isolation.
- Common in EDA environments where many engineers share the same server for design, verification, synthesis etc.

Multi-User System (Shared Linux Server)



Multiple users → Same system → Isolated environment

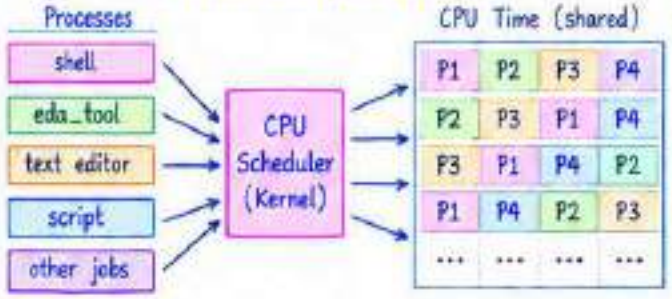
User Related Commands (examples)

Command	Purpose
who	Shows currently logged in users
w	Shows users and their activities
id	Shows user id, group id and groups
users	Shows list of logged in users

4.2 Linux as a Multi-Tasking System

- Linux can run multiple tasks (processes) at the same time.
- CPU time is shared among processes using scheduling.
- Users can run multiple commands, tools and applications simultaneously.
- Supports both interactive and background execution.
- In EDA, we can run synthesis, simulation, verification jobs in parallel.

Multi-Tasking in Linux



Multiple processes → CPU scheduling → Efficient resource usage

4.3 Advantages of Multi-User and Multi-Tasking

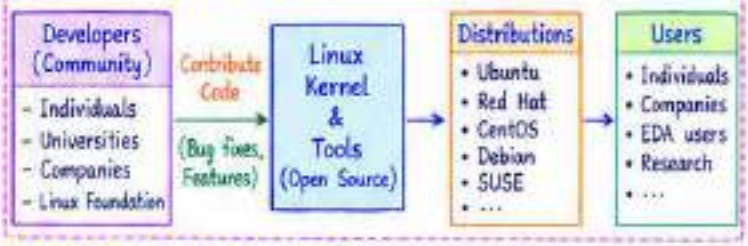
- Better resource utilization (CPU, memory, storage).
- Multiple engineers can work on the same server.
- Supports large scale EDA projects.
- Improves productivity (run many jobs in parallel).
- Provides stability, security and isolation between users.
- Allows both foreground and background execution.

Feature	Benefit in EDA Environment
Multi-user	Team members share tools and data
Multi-tasking	Run multiple EDA jobs in parallel
Resource sharing	Efficient use of CPU, memory, licenses
Isolation	Each user's work is independent
Stability	Long running jobs without reboot
Scalability	Supports large compute servers/clusters

4.4 Linux Open Source

- Linux is an open source operating system.
- The source code is freely available for anyone to view, modify and distribute (mainly under GNU General Public License - GPL).
- Developed and maintained collaboratively by a global community.
- This openness leads to:
  - Faster development and bug fixes
  - High reliability and security
  - Wide support and customization
  - No license cost (compared to proprietary OS)
- Widely used in servers, supercomputers, cloud and EDA environments.

Open Source Development Model



Open Source vs Proprietary

Feature	Open Source (Linux)	Proprietary (e.g., Windows)
Source code	Available	Not available
License cost	Free	Paid
Modification	Allowed	Not allowed
Development	Community driven	Vendor driven
Flexibility	High	Limited
Widely used in EDA	Yes (industry standard)	Limited

Key Point

- Linux = Multi-user + Multi-tasking + Open Source → ideal for EDA environments.
- Enables multiple engineers to work efficiently on shared servers.
- Open source nature makes it reliable, customizable and cost effective.

LINUX FOR ECE / VLSI & EDA

Module 3

Linux Architecture & Linux vs Windows

```
$ ls      ~/project
$ cd      /eda/tools
$ echo $PATH
```

Linux → EDA → Innovation

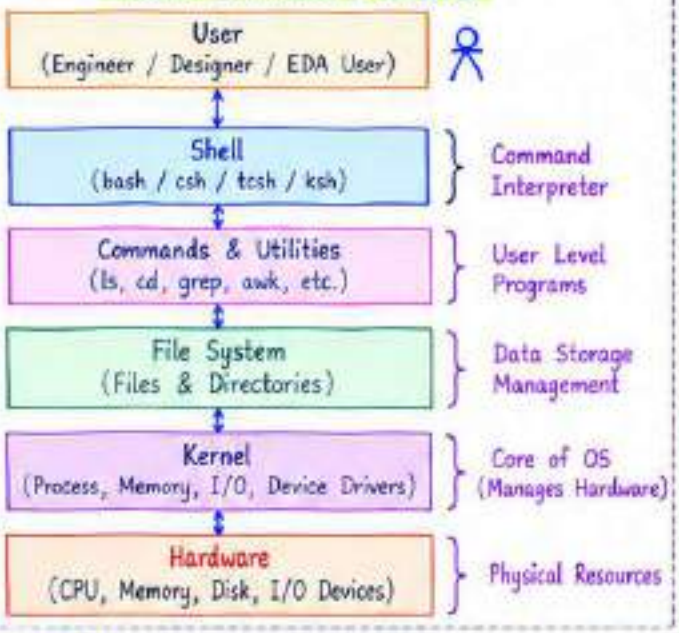
5. LINUX ARCHITECTURE LAYERS

- KERNEL , SHELL , COMMANDS AND UTILITIES , FILES AND DIRECTORIES

5.1 Overview

- Linux is organized in a layered architecture.
- Each layer provides specific functionality and interacts with the layer below it.
- Main layers : Kernel , Shell , Commands & Utilities , Files & Directories.
- This architecture makes Linux stable, secure, multi-user and suitable for EDA environments.

Linux Architecture Layers



5.2 Kernel

- Kernel is the core of Linux operating system.
- It manages hardware resources and provides services to the upper layers.
- Handles : process management, memory management, device management, file system, security etc.
- Users do not directly interact with the kernel.

Kernel Functions

- Process scheduling
- Memory management
- Device drivers
- File system management
- Security and access control

Example (view kernel version)

```
$ uname -r
        → 5.15.0-91-generic
$ uname -a
        # complete info
```

5.3 Shell

- Shell is a command interpreter between user and kernel.
- It reads user commands, interprets them and executes by calling appropriate programs.
- Common shells : bash (default), csh, tcsh, ksh, sh.
- Shell also supports scripting, environment variables, history, aliases etc.

Example (check current shell)

```
$ echo $SHELL
        → /bin/bash
```

Common Shells

```
bash - Bourne Again Shell
csh  - C Shell
tcsh - Enhanced C Shell
ksh  - Korn Shell
sh   - Bourne Shell
```

Key Point

- Linux architecture is layered : User → Shell → Commands → Kernel → Hardware.
- Each layer has a specific role and interacts with adjacent layers.
- This modular design makes Linux powerful, stable and widely used in EDA.

5.4 Commands and Utilities

- Commands are user level programs that perform specific tasks.
- They use services provided by the kernel.
- Examples : file management, text processing, system information, process control etc.
- These are available in /bin, /usr/bin, /usr/local/bin etc.

Common Commands and Their Purpose

Command	Purpose
ls	List files and directories
cd	Change directory
pwd	Print current directory
cp	Copy files / directories
mv	Move / Rename files
rm	Remove files / directories
mkdir	Create a new directory
rmdir	Remove an empty directory
man	Display manual of a command
which	Show path of a command

Example

```
$ ls -ltr      # list files
$ cd project   # change dir
$ which gcc    # path of gcc
$ man ls       # manual
```

Note

- Commands are just programs (executables).
- They run in user space.
- They request services from the kernel.

5.5 Files and Directories

- Linux stores data in a hierarchical file system structure.
- Everything is treated as a file (normal files, directories, devices).
- A directory is a special file used to group files.
- Starts from root directory '/'.

Linux Directory Structure (Example)

```
/ (root)
├── home
│   └── user
│       ├── project
│       └── reports
├── etc
├── bin
├── usr
└── tmp
```

Important Directories

```
/      - Root directory
/home  - User home directories
/etc   - Configuration files
/bin   - Essential commands
/usr   - User utilities
/var   - Logs and variable data
/tmp   - Temporary files
```

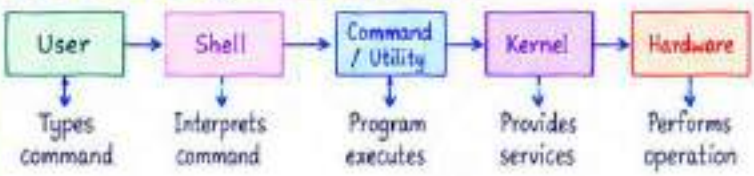
Example :

```
$ cd /home/user/project   # absolute path
$ ls reports              # relative path
```

Path Types

- Absolute Path : /home/user/project
- Relative Path : project/reports

5.6 How the layers work together ?



VIVA / INTERVIEW POINT

Q1. What is the role of kernel in Linux ?
→ It manages hardware resources and provides services to user programs.

Q2. What is the difference between shell and command ?
→ Shell interprets the command, command is an executable program.

Q3. What is the root directory in Linux ?
→ '/' is the top-most directory in the file system.

```
$ ls      ~/project
$ cd      /eda/tools
$ echo $PATH
```


6. LINUX VS WINDOWS

6.1 What is Windows ?

- Windows is a proprietary operating system developed by Microsoft.
- Provides a Graphical User Interface (GUI) with user-friendly environment.
- Commonly used in personal computers, office applications and general purpose usage.
- Many EDA tools are not natively available or have limited support on Windows.

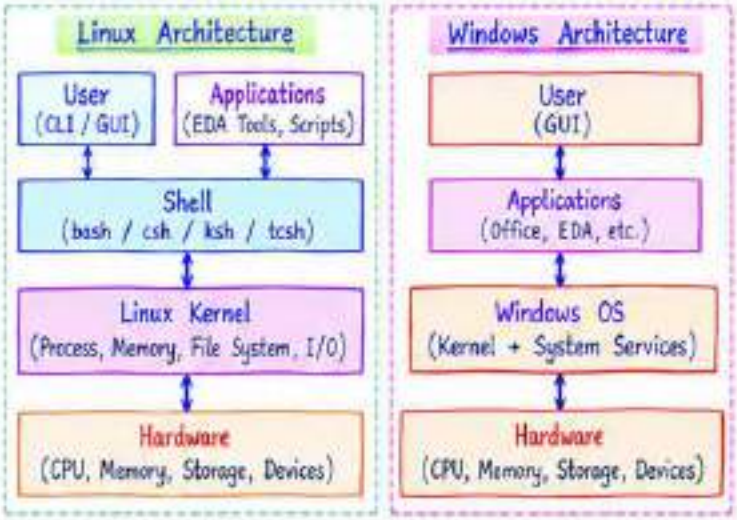
6.2 What is Linux ?

- Linux is an open source, Unix-like operating system based on the Linux kernel.
- Provides both Command Line Interface (CLI) and GUI.
- Widely used in servers, cloud, research and EDA / VLSI industry.
- Known for stability, security, flexibility and customization.

6.3 Key Differences (Overview)

Feature	Linux	Windows
Developer	Community (Open Source)	Microsoft (Proprietary)
License	Free (GPL)	Paid (Proprietary)
User Interface	CLI + GUI (optional)	GUI (default)
Customization	Highly customizable	Limited customization
Security	More secure (less viruses)	More prone to malware
Stability	High (long uptime)	Needs frequent updates
Performance	Efficient, lightweight	Resource intensive
File System	/ (root based)	Drive based (C:, D:)
Case Sensitivity	Case sensitive	Case insensitive
Software Install	Package managers (apt, yum, etc)	Installers (.exe, .msi)
Used in EDA	Industry standard	Limited / via VM

6.4 Architecture Comparison



6.5 File System Comparison

	Linux	Windows
Structure	Single root /	Multiple drives (C:, D:)
Path separator	/	\
Example path	/home/user/project	C:\Users\user\project
Case sensitivity	Yes (file ≠ File)	No (file = File)
Hidden files	Start with . (e.g. .bashrc)	Hidden attribute
Key directories	/home, /etc, /bin, /usr	C:\Program Files, C:\Windows

6.6 Command Comparison

Task	Linux Command	Windows Equivalent
List files	ls	dir
Change directory	cd	cd
Create directory	mkdir	mkdir
Remove file	rm file	del file
Remove directory	rmdir / rm -r	rmdir
Copy file	cp file1 file2	copy file1 file2
Move / Rename	mv old new	move old new
View file	cat file	type file
Clear screen	clear	cls

6.7 Case Sensitivity

- Linux is case sensitive → file, File and FILE are different files.
- Windows is case insensitive → file, File and FILE are treated as the same file.

Linux Example:

```
$ touch file
$ touch File
$ touch FILE
$ ls
  file    File    FILE
```

Windows Example:

```
> type NUL > file
> type NUL > File
> type NUL > FILE
> dir
  file
```

6.8 Why Linux is Preferred in EDA ?

- Most EDA tools (Synopsys, Cadence, Mentor etc.) are developed and certified on Linux.
- Better support for scripting, automation and large scale runs.
- Stable for long running jobs on servers and clusters.
- Flexible environment with package managers and command-line tools.
- Widely used in industry and research.

Key Point

- Linux = Open source, stable, flexible and widely used in EDA.
- Windows = User friendly GUI, good for general use, but limited support for EDA.
- Major differences include file system, case sensitivity, security, customization and tool support.

VIVA / INTERVIEW POINT

Q1. Why is Linux preferred over Windows in EDA ?
→ Most EDA tools are developed and certified on Linux and it provides better stability, scripting and server support.

Q2. What is the main difference in file system structure ?
→ Linux uses a single root (/) while Windows uses multiple drives (C:, D:...).

Q3. What does case sensitivity mean in Linux ?
→ file, File and FILE are treated as three different files.

7. WHAT WE ACTUALLY DO IN WINDOWS VS LINUX ?

7.1 Overview

- Both Windows and Linux are operating systems, but the way we work, tools we use and target users are different.
- In daily use, Windows is mostly GUI driven, while Linux is mostly command line driven.
- In EDA / VLSI, Linux is the industry standard for design, verification, and large scale compute environments.
- Here we see what we actually do in both environments.

7.2 General Usage

Activity	In Windows (What we do)	In Linux (What we do)
Login	User account (GUI)	Login in terminal (ssh/local)
Navigation	Click through folders	Use cd, ls, pwd
Open files	Double click (GUI apps)	Use editors (vi, gvim, nano)
Install software	.exe / installer	package manager (apt/yum) or compile from source
Run tools	GUI based tools	Command line tools & scripts
Manage files	Copy/Paste (mouse)	cp, mv, rm, mkdir, etc.
System settings	Control Panel	Configuration files (text based)
Automation	Limited (GUI / batch)	Shell scripts, Makefiles
Remote work	RDP / GUI tools	ssh, scp, remote shells
Used in VLSI	Less common	Industry standard

7.3 Windows vs Linux (Side by Side)

Feature	Windows	Linux
Developer	Microsoft	Open Source (Community)
License	Proprietary (Paid)	Open Source (Free - GPL)
User Interface	GUI (Graphical)	CLI + GUI (optional)
Ease of Use	Easy for general users	Steeper learning curve (but powerful)
Customization	Limited	Highly customizable
File System	Multiple drives (C:, D:)	Single root (/)
Path separator	\ (backslash)	/ (forward slash)
Case sensitivity	No (file = File)	Yes (file ≠ File)
Software install	.exe / installer	Package manager or compile from source
Used in EDA	Rare	Standard (Synopsys, Cadence, Mentor, etc.)
Stability	Reboot needed frequently	Very stable (uptime long)
Security	More prone to malware	More secure (permissions)
Cost	Paid	Free

7.4 What We Do in Windows ? (Daily Work)



- Use mouse and GUI to perform most tasks.
- Open applications (browser, office tools, etc.).
- Manage files using File Explorer (copy, paste, rename).
- Install tools using setup.exe or MSI installer.
- Used mainly for documentation, presentations, browsing, light coding etc.
- In VLSI, sometimes used for waveform viewing or GUI tools (e.g., DVT tools), but not for full flow.

Typical Windows Workflow

Start Menu → Open App (GUI) → Work → Save (using GUI)

7.5 What We Do in Linux ? (Daily Work)



- Login through terminal (local or remote using ssh).
- Navigate using commands (cd, ls, pwd).
- Edit files using vi / gvim / nano.
- Install tools using package managers (apt, yum) or build from source.
- Run EDA tools and scripts from command line.
- Manage large projects, run simulations, submit jobs to servers.
- Automate tasks using shell scripts, Makefiles.
- Used in VLSI for synthesis, place & route, signoff, verification and large scale compute jobs.

Typical Linux / EDA Workflow

Login (ssh) → Setup Environment → Run Tools (scripts) → Logs / Reports (analysis)

7.6 File and Directory Operations - Example

Windows (GUI)

1) Open File Explorer
2) Double click folder
3) Copy / Paste files
4) Rename using right click

Linux (CLI)

```
$ pwd            # current directory
$ ls -ltr        # list files
$ cd project     # change directory
$ cp file1 file2 # copy file
$ mv old new     # rename / move
$ mkdir run      # create directory
$ rm file        # remove file
```

Key Point

- Windows → User friendly, good for general purpose use.
- Linux → Powerful, flexible, open source and widely used in EDA.
- In VLSI industry, most of the design work is done in Linux environment.
- Learn Linux to work efficiently with industry standard EDA tools.

VIVA / INTERVIEW POINT

Q. Why is Linux mostly used in VLSI instead of Windows ?
→ It is open source, stable, supports command-line automation, works well on servers/clusters and is the standard platform for EDA tools (Synopsys, Cadence, Mentor, etc.).

LINUX FOR ECE / VLSI & EDA

MODULE 3

Linux Architecture & Linux vs Windows

8) WHY TO LEARN LINUX OS ?

1) Introduction

- Linux is a powerful, stable and open-source operating system widely used in industry, research and EDA environments.
- For ECE / VLSI engineers, Linux is the primary platform to run EDA tools, manage large design data, automate flows and work on compute servers.
- Learning Linux helps in better productivity, flexibility and career growth in semiconductor industry.

2) Key Reasons to Learn Linux

(1) Industry Standard for EDA
- Almost all EDA tools (Synopsys, Cadence, Siemens) run on Linux/UNIX.
- Required for place & route, STA, LVS, DRC etc.

(2) Better Control and Flexibility
- Command-line access gives fine control over files, processes and environments.

(3) Automation with Shell Scripting
- Helps to automate repetitive EDA tasks, run flows, parse logs and generate reports.

(4) Efficient Handling of Large Files
- Linux tools (grep, awk, sed, find) are powerful for analyzing big log and report files.

(5) Remote Work and Server Access
- Most EDA work is done on remote Linux servers (using ssh, lsf, bsub).

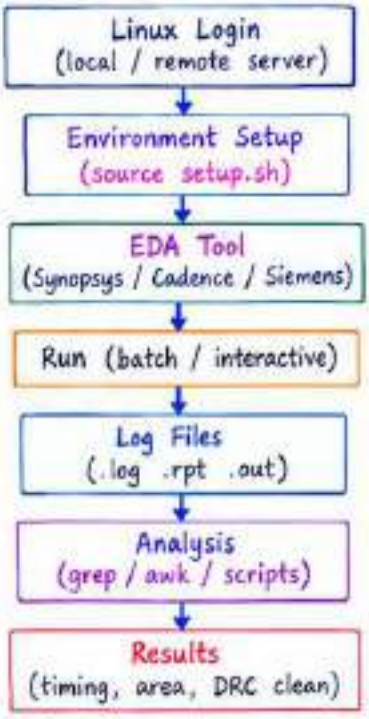
(6) Cost Effective and Open Source
- Linux is open source and widely available.

(7) Widely Used in Industry
- Used in semiconductor, networking, HPC, cloud and research environments.

(8) Improves Problem Solving Skills
- Helps understanding of OS, shell, scripting, processes and system level concepts.

KEY POINT
→ Linux knowledge is a must for VLSI / EDA engineers.
→ Helps in tool usage + automation + server management
→ Improves productivity and employability.
→ Most real-world EDA flows run on Linux.

3) Linux in EDA Design Flow



Why Linux ?
- ✓ Tool availability
- ✓ Large data handling
- ✓ Scripting and automation
- ✓ Remote server access
- ✓ Stable and reliable
- ✓ Used in industry
- ✓ Better performance

4) Linux vs Windows (for EDA)

Feature	Linux / UNIX	Windows
Primary Use	EDA, servers, HPC	Desktop, general use
EDA Tool Support	Native support	Very limited
Command Line	Powerful (bash, csh)	Limited (PowerShell, CMD)
Scripting	Shell scripts (.sh)	Batch (.bat, .ps1)
Stability	High (24x7 servers)	Moderate
Cost	Open source (free)	Licensed (paid)
Remote Access	SSH (standard)	RDP (GUI based)
Use in Industry	Very high	Limited in EDA

5) Practical Observation

- In VLSI companies, design data (RTL, netlists, libraries, logs) are handled on Linux servers.
- Day to day work involves command line, scripts, log analysis and job submission (e.g. bsub, qsub, sbatch).
- Good Linux skills save time and help in quick debugging of tool issues.

6) Viva / Interview Points

Q1. Why is Linux important for EDA engineers ?
↳ Most EDA tools run on Linux and it provides command-line control, scripting, server access and efficient file handling.

Q2. Can we run EDA tools on Windows ?
↳ Generally No. Most industry EDA tools are supported only on Linux/UNIX.



Open Terminal
Type Commands
Get Things Done ...

LINUX FOR ECE / VLSI & EDA

Module 4

Terminal, Shell & Shell Scripting

```
$ ls      ~/project
$ cd      /eda/tools
$ echo $PATH
```

Same Terminal Bigger Possibilities !

9. LINUX TERMINAL - HOW TO OPEN, COMMANDS, EXAMPLES, LISTING FORMAT

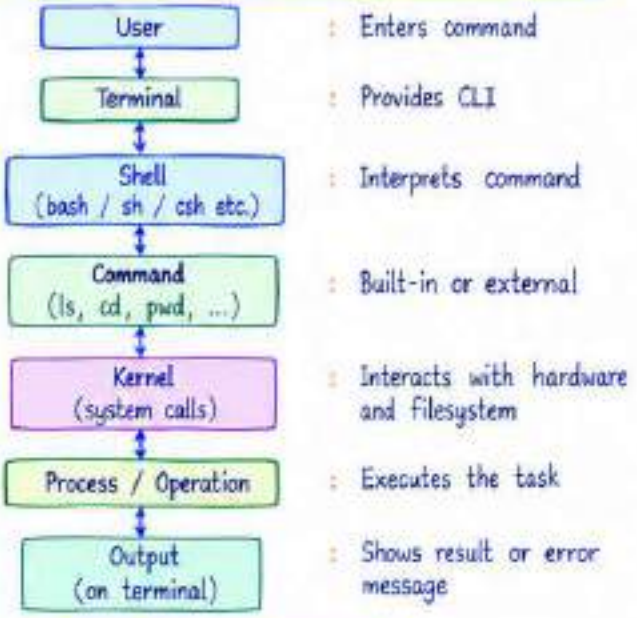
9.1 What is a Linux Terminal ?

- Terminal is a text based interface to interact with the Linux system using commands.
- It provides a command line interface (CLI) to the shell.
- We type commands, shell interprets them and kernel executes them.
- In EDA / VLSI, most of the work (tool runs, scripts, log analysis) is done using terminal.

9.2 How to Open Terminal ?

Method	Steps
Linux GUI (Ubuntu etc.)	Click on Activities → Search "Terminal" → Open Terminal
Shortcut	Use Ctrl + Alt + T (in many distros)
From menu	Applications → Accessories → Terminal
Remote server	Use ssh login (e.g.) $ ssh user@server
From current folder	Right click in file manager → Open in Terminal

9.3 Terminal → Shell → Kernel (Execution Flow)



9.4 Common Terminal Commands

Command	Purpose
pwd	Print current working directory
ls	List files and directories
cd	Change directory
mkdir	Create directory
rmdir	Remove empty directory
touch	Create empty file
cp	Copy files / directories
mv	Move / Rename files
rm	Remove files / directories

9.5 Examples

Command	Example	Result / Usage
pwd	$ pwd	/home/user/project
ls	$ ls	file1 file2 dir1
ls -ltr	$ ls -ltr	Long listing (time sorted)
cd	$ cd project	Change to project directory
mkdir	$ mkdir reports	Create reports directory
touch	$ touch run.log	Create empty file
cp	$ cp file1 file2	Copy file1 to file2
mv	$ mv old new	Rename / move file
rm	$ rm file1	Remove file
ls -a	$ ls -a	Show hidden files
ls -lh	$ ls -lh	Show size in human readable form

9.6 Listing Format (ls options)

Option	Description	Example	Sample Output
-l	Long format	$ ls -l	-rw-r--r-- 1 user ... file1
-t	Sort by time	$ ls -lt	Latest files first
-r	Reverse order	$ ls -ltr	Oldest files first
-a	Show all files (including hidden)	$ ls -a	. .. .bashrc file1
-h	Human readable sizes	$ ls -lh	4.0K 1.2M 10G
-F	Show type indicator	$ ls -F	dir/ file script.sh*

Example : ls -ltr Output

```
$ ls -ltr
total 12
-rw-r--r--  1 user group  1024 Jan 10 10:15 file1
-rw-r--r--  1 user group  2048 Jan 11 09:20 file2
drwxr-xr-x  2 user group  4096 Jan 12 14:30 reports/
```

Type Indicators

/ → directory
* → executable
@ → link
(no symbol) → file

9.7 Practical Observation (EDA)

- We use terminal to set up environment, run EDA tools, check logs, parse reports and submit jobs.
- Command line is faster and more powerful for large projects.
- Many EDA scripts (TCL/Shell/Makefile) are executed from terminal.
- Remote servers are usually accessed using ssh terminal.

Key Point

✓ Terminal gives command line access to the system.
✓ Learn basic commands (pwd, ls, cd, etc.)
✓ Use listing formats (-l, -a, -h, -t).
✓ Essential for EDA / VLSI workflows.

VIVA / INTERVIEW POINT

Q1. How to open terminal in Linux ?
→ From application menu or using Ctrl+Alt+T or via ssh.

Q2. What does ls -ltr do ?
→ Long format, time sorted, oldest files first.

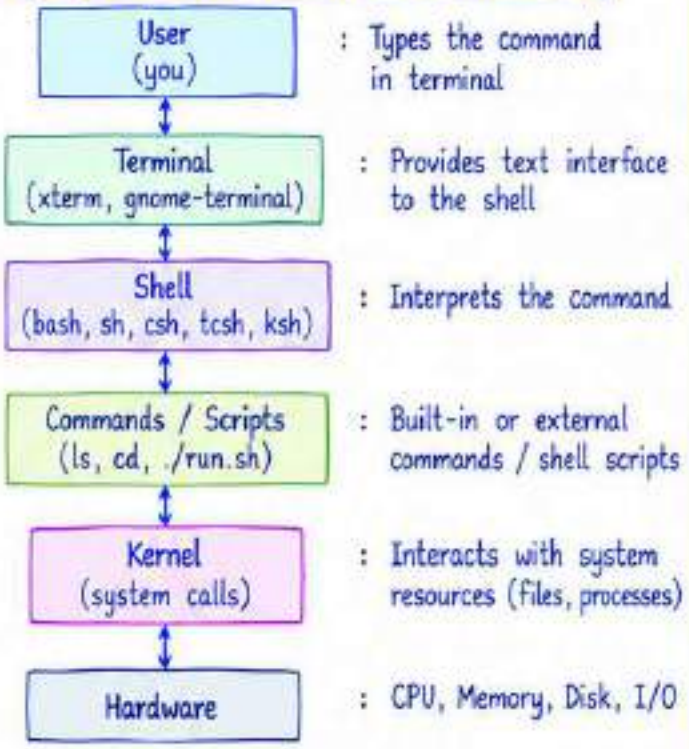
Q3. Why terminal is important in EDA ?
→ It is used to run tools, scripts, manage files, analyse logs and work on remote servers.

10. WHAT IS SHELL, TYPES OF SHELLS, EXAMPLES AND APPLICATIONS

10.1 What is a Shell ?

- Shell is a command line interpreter (CLI) which sits between the user and the Linux kernel.
- It reads user commands, interprets them and executes them by requesting services from the kernel.
- Provides features like command history, scripting, variables, pipes, redirection etc.
- Different shells provide different syntax and features.
- In EDA / VLSI, we use shell to run tools, setup environment, execute scripts and automate flows.

10.2 Shell in Overall Linux Environment



10.3 Types of Shells

Shell	Full Form / Origin	Key Features	Common Usage
sh	Bourne Shell (AT&T)	Original UNIX shell (simple)	Basic shell, portable scripts
bash	Bourne Again Shell (GNU)	Most common, rich features, command history	Linux default, EDA environments
csh	C Shell (BSD)	C-like syntax, interactive	Older systems, easy for beginners
tcsh	TENEX C Shell (enhanced csh)	csh + command history, completion	Used in many EDA tools/servers
ksh	Korn Shell (AT&T)	Advanced features, scripting, performance	Enterprise UNIX systems

10.4 Example Usage

Check current shell

```
$ echo $SHELL
  /bin/bash
$ ps -p $$
  PID  TTY      TIME   CMD
 1234  pts/0    00:00:00 bash
```

Change shell temporarily

```
$ csh
$ bash
```

Change shell permanently

```
$ chsh
(need proper permission)
```

10.5 Applications of Shells

- Interact with Linux system through commands.
- Run and manage EDA tools (e.g., synthesis, P&R, verification).
- Write shell scripts to automate repetitive tasks.
- Set and manage environment variables (PATH, LD_LIBRARY_PATH).
- Process log and report files using command line utilities.
- Used in remote servers (ssh) for batch runs and job submission.
- Customize working environment (aliases, history, prompt).

10.6 Common Built-in Shell Commands

Command	Purpose
echo	Display text / variable value
pwd	Print current working directory
cd	Change directory
history	Show command history
alias	Create shortcut for a command
set	Set shell options / variables
exit	Exit from shell

Key Point

- ✓ Shell provides interface between user and kernel.
- ✓ Different shells have different syntax and features.
- ✓ bash is most widely used in Linux and EDA environments.
- ✓ Shell scripting is essential for automation in VLSI flows.

VIVA / INTERVIEW POINT

Q1. What is a shell ?
→ A command line interpreter that interprets and executes user commands.

Q2. Which shell is most commonly used in Linux ?
→ bash.

Q3. Why are shells important in EDA ?
→ To run tools, automate flows, manage environment and process logs using command line.

11. SHELL SCRIPTING - INTRODUCTION, USAGES, APPLICATIONS, EXAMPLES WITH QUESTIONS

11.1 Introduction to Shell Scripting

- A shell script is a text file containing a sequence of Linux commands executed by the shell.
- Used to automate repetitive tasks, save time and reduce human errors.
- Scripts are interpreted (no compilation needed).
- Commonly written for shells like bash, sh, csh, tcsh, ksh.
- Very useful in EDA / VLSI flows for tool runs, environment setup, log processing etc.

11.2 Basic Structure of a Shell Script

```
#!/bin/bash          → Shebang (specify shell)
# This is a comment  → Comments (ignored by shell)
echo "Hello Linux"
pwd                  → Linux commands
ls -ltr                (executed in order)
```

11.3 How to Create and Run a Script ?

1. Create a file and edit
   `$ vi myscript.sh`
2. Add commands
   (as shown above)
3. Make it executable
   `$ chmod +x myscript.sh`
4. Run the script
   `$ ./myscript.sh`

Note:
- Use ./ to run script in current directory.
- File must have execute permission.
- Check with `ls -l myscript.sh`

11.4 Common Examples

Example	Description
echo "Start run"	Print a message
pwd	Show current directory
ls -ltr	List files
date	Show current date & time
mkdir reports	Create a directory
cd project	Change directory
echo "Done"	End message

11.5 Usages of Shell Scripting

- Automate repetitive commands / tasks.
- Setup environment variables (e.g., PATH, LD_LIBRARY_PATH).
- Run EDA tools (synthesis, P&R, verification) in batch mode.
- Process log / report files (e.g., extract errors/warnings).
- Manage files and directories.
- Schedule runs (using cron).
- Useful for large VLSI projects and regression runs.

11.6 Applications in EDA / VLSI

- Tool setup scripts (e.g., source EDA tool environment).
- Run synthesis / P&R with different corners.
- Automate regression test runs.
- Parse log files to find ERROR / WARNING.
- Generate reports and summary.
- Clean old run directories.
- Submit jobs to farm / server.
- Used in daily engineering workflows.

11.7 Example: Run a Simple EDA Flow Script

```
#!/bin/bash
echo "===== Starting Synthesis Run ====="
source ./setup.sh   # setup tool environment
cd run
echo "Running synthesis ..."
dc_shell -f synth.tcl > synth.log 2>&1
echo "Run completed"
```

11.8 Sample Questions (Viva / Interview)

Q1. What is a shell script ?
→ A text file containing a set of Linux commands executed by a shell.

Q2. How do you make a script executable ?
→ chmod +x script.sh

Q3. Which shell is commonly used for scripting ?
→ bash (Bourne Again Shell).

Q4. Give one application of shell scripting in EDA.
→ Automating synthesis / P&R runs and log processing.

Key Point

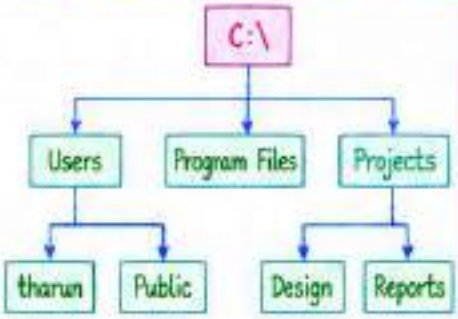
- ✓ Shell scripting improves productivity and reduces manual effort.
- ✓ It is widely used in EDA / VLSI for automation.
- ✓ Learn basic scripting to handle real project workflows.

12. LINUX FILES AND DIRECTORY MANAGEMENT - HOW WINDOWS ANALOGY vs LINUX ANALOGY

12.1 Introduction

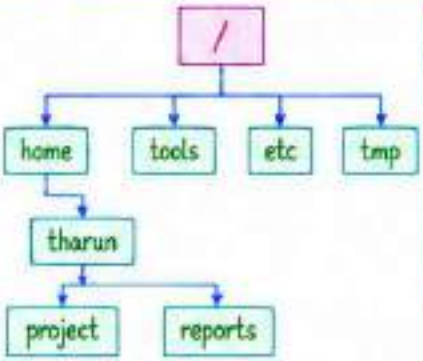
- Linux stores data in files and directories arranged in a hierarchical tree structure.
- Everything is a file (regular file, directory, device, etc.) under a single root directory /.
- We use command line utilities to manage files and directories (create, list, move, copy, delete, etc.).
- Linux file management is powerful, flexible and is essential in EDA/VLSI projects to handle design files, logs, reports, scripts and large datasets.

12.2 Windows File System (Analogy)



Windows points:
- Multiple drives (C:, D:, E:)
- Uses \\ as separator
- GUI based (Explorer)
- File types by extension (.txt, .docx, .exe)

12.3 Linux File System (Analogy)



Linux points:
- Single root /
- Uses / as separator
- Mostly command line
- No extension required
- Everything is a file

12.4 Windows vs Linux Analogy

Feature	Windows (Analogy)	Linux (Analogy)
Structure	Drives (C:, D:)	Single root (/)
Path separator	\\ (backslash)	/ (forward slash)
Access	GUI (Explorer)	Command line (shell)
File type	Uses extensions (file.txt)	No extension required
Case sensitivity	Usually not case sensitive	Case sensitive
Typical tools	Explorer, GUI apps	ls, cd, mkdir, cp, mv, rm
Usage in EDA	Limited	Widely used (scripts, logs, tools, automation)

12.5 Common File and Directory Commands

Command	Purpose
pwd	Print current working directory
ls	List files and directories
ls -l	Long listing format
ls -a	Show hidden files (starting with .)
ls -ltr	Long format, time sorted (latest last)
cd [dir]	Change directory
mkdir [dir]	Create a new directory
rmdir [dir]	Remove an empty directory
touch [file]	Create an empty file
rm [file]	Remove a file
cp src dst	Copy file/directory
mv src dst	Move / rename file/directory

12.6 Listing Format (ls options)

Example:

```
$ ls -ltr
```

permissions | links | owner | group | size | date | time | name

```
total 12
-rw-r--r--  1 tharun users  234 Apr 10 10:15 run.log
drwxr-xr-x  2 tharun users 4096 Apr 11 09:20 reports
-rwxr-xr-x  1 tharun users 1024 Apr 12 16:45 run.sh
```

12.7 Practical Usage in EDA / VLSI

- Create project directory structure for design runs.
- Organize RTL, netlists, scripts, logs and reports.
- Use ls -ltr to check latest run outputs.
- Write scripts to clean old files and manage runs.
- Essential for automation and regression flows.

Key Point

✓ Linux file system is a hierarchical tree starting from /
✓ Command line tools make file/directory management efficient.
✓ Essential skill for EDA / VLSI project workflows.

Viva / Interview Questions

Q1. What is the root directory in Linux? → /
Q2. How to list all files including hidden files? → ls -a
Q3. Difference between mkdir and rmdir?
→ mkdir : create directory
→ rmdir : remove empty directory

13) CREATE AND REMOVE FILES

– touch , rm , rm -f – explain these commands and examples

1) Introduction

- Files are used to store data, scripts, logs, reports in Linux.
- We can create empty files or update their time stamp using touch command.
- We can remove (delete) files using rm command.
- rm -f is used to forcefully remove files without prompting for confirmation.
- These commands are very useful in day to day EDA work (e.g., creating run scripts, logs, report files, cleanup old files etc.).

2) touch Command

Purpose

- Create an empty file.
- If the file already exists, it updates the file's access and modification time (timestamp).

Syntax

```
touch [options] filename
```

Common Options

-c	Do not create the file if it does not exist
-t TIME	Specify a particular timestamp

Examples

```
$ touch run.sh              # creates an empty file
$ touch report.log          # creates an empty file
$ touch file1 file2         # create multiple files
$ touch -c existing.txt     # do not create if not exists
$ touch -t 202401011200 old.log  # set time
```

Verify

```
$ ls -l run.sh
-rw-r--r-- 1 user group 0 Sep 13 10:20 run.sh
```

Observation

- touch is commonly used to create empty log files before running EDA tools.
- It is also used in shell scripts to create marker files or update timestamps.

3) rm Command

Purpose

- Remove a (delete) files from the filesystem.
- By default, rm asks for confirmation when removing write-protected files.

Syntax

```
rm [options] filename
```

Common Options

-i	Prompt before every removal
-f	Force removal (no prompt, ignore errors)
-r	Remove directories and their contents (not used for single files)
-v	Verbose mode (show what is being removed)

Examples

```
$ rm run.sh              # remove a file
$ rm -i report.log       # ask for confirmation
$ rm -f temp.log         # force remove without prompt
$ rm -v data.txt         # show removed file
$ rm file1 file2         # remove multiple files
$ rm *.log               # remove all .log files
$ rm -f *.rpt            # force remove all .rpt files
```

4) rm -f Command (Force Remove)

- The -f option tells rm to forcefully remove the file without asking for confirmation.
- It also ignores non-existent files (no error message).
- Useful in scripts where we don't want any manual intervention.

Syntax

```
rm -f filename
```

Examples

```
$ rm -f run.log          # remove without prompt
$ rm -f old_file.txt     # remove even if write-protected
$ rm -f not_exist.log    # no error even if file not present
```

KEY POINT

→ touch → create empty file or update timestamp.
→ rm → remove files (may ask for confirmation).
→ rm -f → forcefully remove files without confirmation.
→ Use rm -f carefully (files once deleted cannot be recovered).

5) Viva / Interview Points

Q1. What is the difference between touch and rm ?
↳ touch creates/updates a file. rm deletes a file.

Q2. Why use rm -f in scripts ?
↳ To remove files without manual confirmation and to avoid script stopping when file is not found.

14. CREATE AND REMOVE DIRECTORIES - MKDIR , RMDIR (RM DIR)

14.1 Introduction

- In Linux, a directory (also called (folder) is a special file that stores other files and directories.
- Directories help in organizing project files, logs, scripts, tools etc.
- We use mkdir to create directories.
- We use rmdir or rm -r to remove directories.
- Directory management is very important in EDA workflows (to maintain clean and organized runs).

14.2 mkdir command

- mkdir is used to create a new directory.
- By default, it creates only one directory.
- Use -p option to create multiple levels of directories at once (including intermediate directories).

Syntax :	mkdir dirname mkdir -p dir1/dir2/dir3

Examples :	$ mkdir project
	$ mkdir run1 run2
	$ mkdir -p /home/user/eda/run1/logs

Result :	New directory/directories are created.

14.3 rmir command

- rmdir is used to remove an empty directory.
- It will fail if the directory contains files or subdirectories.

Syntax :	rmdir dirname
Examples :	$ rmdir old_dir
Result :	Removes the empty directory. If directory is not empty → Error.

14.4 rm -f (Remove Directory)

- rm -r is used to remove a directory and all its contents (files and subdirectories).
- Use with caution, as deleted data is not recoverable.

Syntax :	rm -f dirname
Examples :	$ rm -r project $ rm -r /home/user/tmp/run1
Result :	Directory and all its contents are removed.

14.5 rm -rf (Force Remove)

- rm -rf removes directory and its contents without asking for confirmation.
- Useful for scripts and automated cleanups.

Syntax :	rm -rf dirname
Examples :	$ rm -rf old_run $ rm -rf ./temp/*
Result :	Directory is removed forcefully (no prompt).

14.6 Examples and Output

Command	Description	Example	Output / Result
mkdir	Create a directory	$ mkdir project	project/ created
mkdir -p	Create multiple levels	$ mkdir -p run1/logs/reports	run1/logs/ reports/ created
rmdir	Remove empty directory	$ rmdir empty_dir	empty_dir removed
rmdir	Remove non-empty directory	$ rmdir data	rmdir: failed Directory not empty
rm -r	Remove directory with contents	$ rm -r data	data/ removed (with all files)
rm -rf	Force remove (without prompt)	$ rm -rf temp	temp/ removed (no message)

14.7 Windows vs Linux Analogy

Windows (GUI)	Linux (Command Line)
Folder	Directory
Create folder	mkdir folder_name
Create nested folders	mkdir -p dir1/dir2
Delete empty folder	rmdir folder_name
Delete folder with files	rm -r folder_name
Force delete	rm -rf folder_name
Navigation — Use File Explorer	Use cd, ls





Practical EDA Example

```
$ mkdir -p my_chip/{scripts,logs,reports}   # create project structure
$ ls my_chip                                # list directories
$ rm -r my_chip/logs                        # remove logs directory
$ rm -rf my_chip                            # cleanup entire project
```

Questions (Viva / Interview)

Q1. Which command is used to create a directory ?
→ mkdir

Q2. How to create multiple directories at once ?
→ mkdir -p dir1/dir2

Q3. What is the difference between rmdir and rm -r ?
→ rmdir removes only empty directory, rm -r removes with contents.

Q4. Which command removes a directory without prompt ?
→ rm -rf

Key Point

- ✓ Use mkdir to create directories.
- ✓ Use rmdir for empty directories.
- ✓ Use rm -r for non-empty directories.
- ✓ Use rm -rf with caution.
- ✓ Organize directories in EDA flows.

15. LIST FILES AND DIRECTORIES - ls , ls -l , ls -lt , ls -ltr , ls -d , ls -f , ls .*v , etc.

15.1 Introduction

- ls (list) is used to display the contents of a directory (files and subdirectories).
- It shows file names by default and supports various options to view additional information like permissions, size, modification time etc.
- Widely used in EDA environment to check available files, logs, reports, scripts, directories etc.

15.2 Basic ls command

- Displays the list of files and directories in current directory.

Syntax :	ls [options] [path]
Example :	$ ls
Result :	Lists files and directories.

15.3 ls -l (Long listing format)

- Shows detailed information : permissions, owner, group, size, modification time and file name.

Syntax :	ls -l [path]
Example :	$ ls -l
Result :	

```
-rw-r--r--  1 user group  234  Aug 10  10:45  run.log
drwxr-xr-x  3 user group 4096  Aug 10  11:02  scripts
```

15.4 ls -lt (Sort by modification time)

- Lists files with latest modified files on top.

Syntax :	ls -lt
Example :	$ ls -lt
Result :	Files sorted by time (newest first).

15.5 ls -ltr (Long + time + reverse)

- Lists in long format, sorted by time with oldest first.

Syntax :	ls -ltr
Example :	$ ls -ltr
Result :	Files sorted by time (oldest first).

Sample Output (ls -l)

```
-rw-r--r--  1 user group  1045  Aug 10  10:45  run.log
-rw-r--r--  1 user group  2380  Aug 09  21:12  report.rpt
drwxr-xr-x  3 user group  4096  Aug 08  18:30  scripts
drwxr-xr-x  5 user group  4096  Aug 07  14:20  results
```

15.6 Other useful options

Command	Description	Example	Output / Result
ls	List files in current dir	$ ls	file1 file2 dir1
ls -l	Long listing format	$ ls -l	detailed info
ls -lt	Sort by time (latest first)	$ ls -lt	newest on top
ls -ltr	Long format, oldest first	$ ls -ltr	oldest on top
ls -d	List directories only (do not show contents)	$ ls -d */	dir1/ dir2/
ls -f	Do not sort (as in FS order)	$ ls -f	list in FS order
ls .*v	List files matching pattern	$ ls .*v	top.v alu.v ...
ls -a	List all files (including hidden)	$ ls -a	. .. .bashrc ...
ls -la	Long format + all files	$ ls -la	detailed list incl. hidden files

15.7 Metacharacters with ls

Symbol	Meaning	Example	Result
*	matches any sequence of characters	$ ls *.log	all .log files
?	matches a single character	$ ls file?.txt	file1.txt file2.txt
[]	match any one from set	$ ls file[0-9].txt	file1.txt ... file9.txt
.	current directory	$ ls .	list current dir
..	parent directory	$ ls ..	list parent dir
.*	hidden files (start with .)	$ ls .*	.bashrc .profile ...
/	dir identifier	$ ls -d */	only directories

Practical Example (EDA)

```
$ ls              # list files in run directory
$ ls -ltr         # check oldest log file
$ ls *.rpt        # list all report files
$ ls -d */        # list subdirectories only
$ ls -a           # show hidden config files
$ ls *.v          # list all verilog files
$ ls run??        # match run1, run2 etc
```

Questions (Viva / Interview)

Q1. What is the use of ls -l ?
→ To display detailed information of files and directories.

Q2. What does ls -d */ do ?
→ Lists only directories without showing their contents.

Q3. How to list all .v files in a directory ?
→ ls *.v

Key Point

- ✓ ls → list files
- ✓ -l → detailed info
- ✓ -t → sort by time
- ✓ -r → reverse order
- ✓ -d → directories only
- ✓ -f → no sorting
- ✓ * ? [] → pattern matching
- ✓ Very useful in EDA flows to view logs, reports etc.

16. HIDDEN FILES IN LINUX OS - ls -a , ls -s - (with meta characters and examples)

16.1 What are Hidden Files ?

- In Linux, files or directories whose names start with a dot (.) are called hidden files.
- They are not shown by default with normal ls command.
- Hidden files are commonly used for configuration, history, cache, environment settings etc.
- Example hidden files : .bashrc .profile .config .git .vimrc
- These files are important in Linux and EDA environments.

16.2 Viewing Hidden Files using ls -a

- The -a option shows all files including hidden files.
- It displays entries starting with '.' (including . and ..).

Syntax :	ls -a [path]
Example :	$ ls -a
Result :	Shows both normal and hidden files.

Example Output :

```
$ ls
run.log    scripts    reports
$ ls -a
.    ..    .bashrc    .config    .vscode    run.log    scripts    reports
```

. = current directory; .. = parent directory; .bashrc .config .vscode = hidden files (start with .)

16.3 Viewing Size of Hidden Files using ls -s

- The -s option displays the size (in blocks) of each file.
- Useful to find which hidden files are consuming space.

Syntax :	ls -s [path]
Example :	$ ls -s -a
Result :	Shows size of all files including hidden files.

Example Output :

```
$ ls -s -a
4  .          4  ..          4  .bashrc      8  .config
16 .vscode    28 run.log     12 scripts      8  reports
```

↳ size in 1K blocks (approx)

16.4 Common Combinations for Hidden Files

Command	Purpose	Example
ls -a	show all (including hidden)	$ ls -a
ls -al	long listing of hidden files	$ ls -al
ls -s	show size	$ ls -s
ls -s -a	size of all (including hidden)	$ ls -s -a
ls -d .*	list only hidden files/dirs	$ ls -d .*
ls -s .*	size of only hidden files	$ ls -s .*

16.5 Listing Only Hidden Files

- Use meta character * with dot to list only hidden files.

Syntax :	ls .*
Example :	$ ls .*
Result :	Lists only hidden files and directories.

Example Output :

```
$ ls .*
.bashrc    .config    .vscode    .profile    .gitignore
```

16.6 Listing Only Hidden Directories

- Use -d option with .* to list hidden directories only.

Syntax :	ls -d .*/
Example :	$ ls -d .*/
Output :	.config/ .vscode/ .git/

16.7 Metacharacters with Hidden Files

Pattern	Meaning	Example	Result
.*	matches all hidden files	$ ls .*	all hidden files
.?*	hidden files with one character after dot	$ ls .?*	.a .b .c
.???*	hidden files with 3 or more characters	$ ls .???*	.abc .xyz ...
.*.log	hidden files with .log	$ ls .*.log	.run.log .err.log
.*rc	hidden files ending with rc	$ ls .*rc	.bashrc .vimrc
.*[0-9]	hidden files with digit	$ ls .*[0-9]	.file1 .data2
.*/*	hidden subdirectories	$ ls -d .*/	.config/ .git/

Example: Pattern Matching for Hidden Files

```
$ ls .*          # all hidden files
$ ls .*.log      # hidden .log files
$ ls .?*         # hidden files with one character (e.g. .a)
$ ls .[a-z]*     # hidden files starting with a-z
$ ls -s .*       # size of all hidden files
```

Practical Observation (EDA)

- Hidden files store tool configurations (e.g. .synopsys, .cadence, .mentor).
- Useful to check .bashrc, .profile for EDA environment setup.
- Large hidden log/cache files can consume disk space, use ls -s -a to identify.
- Be careful while deleting hidden files.

Key Points

- ✓ Files starting with . are hidden.
- ✓ Use ls -a to view hidden files.
- ✓ Use ls -s to see size.
- ✓ Use .* to list only hidden files.
- ✓ Use -d with .* for directories.
- ✓ Hidden files store important configurations (EDA tools).
- ✓ Meta characters help to filter hidden files.

Viva / Interview Questions

Q1. Why are hidden files used in Linux ?
→ To store configuration, environment and temporary files without cluttering the directory listing.

Q2. What does ls -a display ?
→ All files including hidden files (starting with .).

Q3. How to list only hidden directories ?
→ ls -d .*/

17. LISTING FILES IN MULTIPLE DIRECTORIES - directory , find (with examples)

17.1 Introduction

- Often we need to list files present in multiple directories (current directory, subdirectories, or different paths).
- Linux provides commands like ls and find to achieve this.
- The directory command gives a simple view of files when multiple directories are given as arguments.
- The find command is more powerful and searches recursively across directories with various conditions.
- Useful in EDA projects to locate logs, reports, netlists, scripts etc across large directory structures.

17.2 directory command

- directory is similar to ls but displays files in multiple directories side by side under directory headings.
- It is mainly used to quickly list contents of multiple directories (non-recursive).

Syntax :	directory dir1 dir2 ...
Example :	$ directory run1 run2 reports
Result :	Lists files in each specified directory.

Example Output :

```
$ directory run1 run2
run1:                 run2:
  run.log               run.log
  script.sh             synth.rpt
  timing.rpt            netlist.v
```

17.3 find command

- find is used to search for files and directories in one or more directories (recursively).
- It can search based on name, type, size, modification time etc.
- Very useful to locate files across large directory trees.

Syntax :	find [path] [options] [expression]

Examples :	$ find . -name "*.log"
	$ find /home/user -type f -name "*.rpt"
	$ find . -type d -name "run*"

Result :	Lists files/directories matching the given criteria.

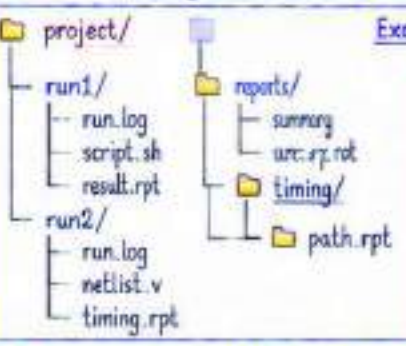
17.4 Common find options

Option	Meaning	Example
-name	search by file name	find . -name "*.log"
-type f	search only files	find . -type f -name "*.rpt"
-type d	search only directories	find . -type d -name "run*"
-size	search by size	find . -size +10M
-mtime	search by modification time	find . -mtime -7
-path	search specific path	find /home -path "*/logs/*"

17.5 Examples and Output

Command	Description	Example	Output (Sample)
directory	List files in multiple dirs (non-recursive)	$ directory dir1 dir2	dir1: a.txt dir2: b.txt
find	Search files recursively by name	$ find . -name "*.log"	./run1/run.log ./run2/debug.log
find	Search only files	$ find . -type f	./run1/a.txt ./run2/b.txt
find	Search only directories	$ find . -type d	./run1 ./run2
find	Search by size	$ find . -size +10M	./run1/large.log
find	Search by modification time (last 7 days)	$ find . -mtime -7	./reports/timing.rpt

17.6 Directory structure example



Example : $ directory run1 run2

```
$ find project -name "*.rpt"
project/run1/result.rpt
project/run2/timing.rpt
project/reports/summary.rpt
project/reports/drc.rpt
project/reports/timing/path.rpt
```

17.7 Metacharacters for file matching

Symbol	Meaning	Example	Matches
*	any sequence of characters	*.log	run.log, a.log
?	single character	file?.txt	file1.txt, fileA.txt
[ ]	any one character from set	file[0-9].txt	file1.txt, file5.txt
[a-z]	range of characters	log_[a-z].txt	log_a.txt, log_b.txt
.	literal dot (hidden files)	.*	.bashrc, .config
..	parent directory	ls ..	parent directory
/	directory separator	run1/*.rpt	all .rpt in run1

Practical Example (EDA Flow)

```
# list all .rpt files in current project and subdirectories
$ find . -name "*.rpt"
# list only log files larger than 1MB
$ find . -type f -name "*.log" -size +1M
# list all directories named run*
$ find . -type d -name "run*"
# use directory to quickly view multiple folders
$ directory run1 run2 reports
```

Key Points

- ✓ directory → list multiple directories (non-recursive)
- ✓ find → powerful recursive search
- ✓ Use -name with * for pattern matching
- ✓ Use -type to filter files/dirs
- ✓ Hidden files start with .
- ✓ Metacharacters help in matching multiple files
- ✓ Very useful in EDA projects to locate logs/reports.

Questions (Viva / Interview)

Q1. How do you list all .log files in current and subdirectories?
→ find . -name "*.log"

Q2. What is the difference between directory and find ?
→ directory lists given dirs (non-recursive), find searches recursively.

Q3. How do you list only hidden files in a directory ?
→ ls -a or ls .*

18. LISTING FILES IN MULTIPLE DIRECTORIES - du , find

18.1 Introduction

- In real projects, files are spread across multiple directories (logs, reports, scripts, netlists etc).
- We use du to check disk usage of files/directories and find to search for files/directories recursively across multiple directories.
- These commands are very useful in EDA flows to find large files, specific report files or to analyze project directory structure.

18.2 du command (Disk Usage)

- du (disk usage) is used to estimate the disk space used by files and directories.
- Useful to find large directories/files in your project.

Syntax :	du [options] [path]
Example :	$ du -sh ./project
Result :	Shows total size of project directory.

18.2.1 Common du options

Option	Meaning	Example
-s	summary (total of directory)	du -s dir
-h	human readable (KB, MB, GB)	du -h dir
-sh	summary + human readable	du -sh dir
-a	show files as well as directories	du -ah dir
-c	show grand total	du -shc dir1 dir2
--max-depth=N	limit directory level	du --max-depth=1 dir
-x	stay on same filesystem	du -shx /home

Example Output (du -sh)

```
$ du -sh *
4.0K    scripts/
12M     logs/
3.1G    results/
28K     reports/
```

Observation:
- Helps to identify large directories consuming space.
- Useful to clean unnecessary files in EDA projects.

18.3 Practical use-cases of du

- Find which directory is consuming more disk space.
- Check log directory size after tool runs.
- Identify large report or waveform files.
- Monitor disk usage in shared servers.
- Clean temporary directories (e.g. ./temp, ./INCA).

18.4 find command (Search Files/Directories)

- find is used to search for files and directories recursively across a directory tree based on various criteria like name, type, size, modification time etc.
- It is very powerful for locating files in multiple directories.

Syntax :	find [path] [options] [expression]
Example :	$ find . -name "*.rpt"
Result :	Lists all .rpt files in current directory and subdirectories.

18.4.1 Common find options

Option	Meaning	Example
-name	search by file name (case sensitive)	find . -name "*.rpt"
-type f	search only files	find . -type f
-type d	search only directories	find . -type d
-size +N	files larger than N (size)	find . -size +100M
-size -N	files smaller than N	find . -size -10M
-mtime N	modified N days ago	find . -mtime 7
-iname	case insensitive search	find . -iname "*.LOG"
-path	search specific path	find ./logs -name "*.log"
-exec	execute command on result	find . -name "*.log" -exec rm {} \;
-print	(default) print the results	find . -name "*.rpt" -print

18.4.2 Examples

```
$ find . -name "*.rpt"          # find all .rpt files
$ find ./logs -name "*.log"     # find .log in logs dir
$ find . -type f -size +100M    # find files > 100MB
$ find . -type d -name "INCA"   # find directories named INCA
$ find . -mtime 7 -name "*.rpt" # find .rpt modified 7 days ago
$ find /home/user -iname "*.LOG" # case insensitive search
```

18.4.3 Example Output

```
$ find . -name "*.rpt"
./run1/timing.rpt
./run1/summary.rpt
./run2/placement.rpt
./reports/final.rpt
```

Directory structure



Viva / Interview Questions

Q1. What is the use of du ?
→ To check disk usage of files/directories.

Q2. How to find all .log files in a directory tree ?
→ find . -name "*.log"

Q3. How to find files larger than 1GB ?
→ find . -type f -size +1G

Key Points

- ✓ du → find disk usage.
- ✓ find → search files/dirs.
- ✓ Use -name with * for pattern matching
- ✓ -type f → files only.
- ✓ -type d → directories only.
- ✓ Use -size, -mtime for advanced search.
- ✓ Very useful in EDA projects to locate logs, reports, large files etc.

19. FILES AND DIRECTORIES - PROPERTIES AND PERMISSIONS (chmod , 10 characters)

19.1 File/Directory Properties

- Every file and directory in Linux has properties such as name, size, owner, group, permissions and modification time.
- These can be viewed using commands like ls -l and stat.
- Permissions control who can read (r), write (w) and execute (x) a file/directory.
- In Linux, each file/directory has a 10 character permission string (e.g: -rwxr-x---).

19.2 The 10 character permission string

```
-rwxr-x---
1 2 3 4 5 6 7 8 9 10
```

1	→ file type (- regular file, d directory, l link etc.)
2-4	→ owner permissions (r w x)
5-7	→ group permissions (r w x)
8-10	→ others permissions (r w x)

19.3 File type (1st character)

Symbol	Type	Meaning
-	Regular file	Normal file (text, log, script etc.)
d	Directory	Directory
l	Symbolic link	Link to another file/directory
c	Character device	Device file (e.g. terminal)
b	Block device	Device file (e.g. disk)
p	Named pipe	Pipe for IPC
s	Socket file	Inter-process communication

19.4 Permission characters

Symbol	Meaning	Value
r	read permission	4
w	write permission	2
x	execute permission	1
-	no permission	0

Combinations

7 = rwx
6 = rw-
5 = r-x
4 = r--
3 = -wx
2 = -w-
1 = --x
0 = ---

19.5 chmod command

- chmod is used to change the permissions of a file/directory.
- Permissions can be specified using either numeric or symbolic mode.

Syntax :	chmod [options] mode file/dir
Example :	$ chmod 755 run.sh
Result :	Sets owner: rwx, group: r-x, others: r-x

19.6 chmod examples

Command	Meaning	Permission (rwx)	Example
chmod 755	file.sh	rwx r-x r-x	Executable script
chmod 644	file.txt	rw- r-- r--	Normal file
chmod 700	dir/	rwx --- ---	Private directory
chmod 777	temp/	rwx rwx rwx	Full access (not recommended)
chmod +x script.sh	Add execute permission		
chmod -w file.txt	Remove write permission		
chmod u+r file.txt	Add read for owner		
chmod go-w file.txt	Remove write for group & others		

19.7 ls -l output fields

```
-rwxr-x---  1  user  group  4096  Aug 10 10:45  run.sh
```

permissions | links | owner | group | size (bytes) | date (date time) | name

19.8 stat command (more properties)

- stat provides detailed information about a file/directory.
- Useful to view inode number, permissions, size, timestamps etc.

Example : $ stat run.sh

```
  File: run.sh
  Size: 4096        Blocks: 8        IO Block: 4096   regular file
Device: 802h/2050d  Inode: 123456    Links: 1
Access: (0755/-rwxr-xr-x)  Uid: (1000/user)  Gid: (1000/group)
Access: Aug 10 10:44:12 2026
Modify: Aug 10 10:45:01 2026
Change: Aug 10 10:45:01 2026
```

19.9 Using find with permissions

- find can search files/directories based on permissions.

```
Examples :  $ find . -perm 755              # find files with 755
            $ find /project -type f -perm 644  # find files with 644
            $ find . -type d -perm 700      # find dirs with 700
            $ find . -executable            # find executable files
```

Viva / Interview Questions

Q1. What does -rwxr-x--- mean ?
→ Regular file, owner: rwx, group: r-x, others: ---.

Q2. What is the use of chmod ?
→ To change file/directory permissions.

Q3. How do you give execute permission to a script ?
→ chmod +x script.sh

Key Points

- ✓ First character → file type.
- ✓ Next 9 characters → permissions.
- ✓ r = 4, w = 2, x = 1
- ✓ chmod → change permissions.
- ✓ Use numeric (755) or symbolic (+x).
- ✓ stat → detailed properties.
- ✓ find → search based on permissions.
- ✓ Permissions are very important in EDA environments for scripts, logs, reports and tool execution.

20. ALL USERS IN LINUX - ROOT , PRIMARY , GROUP , OTHERS

20.1 Users in Linux

- Linux is a multi-user operating system. Multiple users can work on the same system with different privileges.
- Each user is identified by a username and a unique User ID (UID).
- Users have a home directory, shell and set of permissions.
- Users can be classified as : root user, normal user (primary user), members of groups and others.

20.2 Types of Users

1) Root User (Superuser)

- Username : root UID : 0
- Has complete access to the system.
- Can perform any operation (install, configure, modify, delete).
- Use with care. Usually accessed using sudo or su.

2) Primary User (Normal User)

- Created for regular work.
- Has limited permissions.
- Each user has a primary group (same name as username by default).

3) Group

- A group is a collection of users.
- Used to manage permissions on shared files/directories.
- A user can belong to one primary group and multiple secondary groups.

4) Others

- Refers to all users who are not the owner and not in the group.
- They have the most limited access by default.

20.3 User and Group Information Commands

Command	Purpose
who	Shows currently logged in users
whoami	Shows current username
id	Shows UID, GID and group information
groups	Shows groups of current user
cat /etc/passwd	List all users in the system
cat /etc/group	List all groups in the system

Example Outputs :

```
$ who
user1    pts/0    Aug 10 10:15
user2    pts/1    Aug 10 11:02
$ whoami
user1
$ id
uid=1001(user1) gid=1001(user1)
groups=1001(user1),2001(eda),3001(vlsi)
$ groups
user1 eda vlsi
```

```
$ cat /etc/passwd (sample)
root:x:0:0:root:/root:/bin/bash
user1:x:1001:1001::/home/user1:/bin/bash
user2:x:1002:1002::/home/user2:/bin/bash
...
```

```
$ cat /etc/group (sample)
root:x:0:
user1:x:1001:
eda:x:2001:user1,user2
vlsi:x:3001:user1
...
```

20.4 File/Directory Permission Structure (10 characters)



1 : file type
2-4 : owner (user)
5-7 : group
8-10 : others

File type (1st character)

Symbol	Meaning
-	regular file
d	directory
l	symbolic link
c	character device
b	block device
p	named pipe
s	socket file

Permission characters

Symbol	Meaning	Value
r	read	4
w	write	2
x	execute	1
-	no permission	0

Permission combinations

7 = rwx 5 = r-x 3 = -wx 1 = --x
6 = rw- 4 = r-- 2 = -w- 0 = ---

20.5 chmod Command

- chmod is used to change file or directory permissions.
- Permissions can be specified using numeric mode or symbolic mode.
- Syntax : `chmod [options] mode file/dir`

Examples :

```
$ chmod 755 script.sh      # rwxr-xr-x
$ chmod 644 file.txt       # rw-r--r--
$ chmod +x run.sh          # add execute permission
$ chmod -w file.txt        # remove write permission
```

20.6 Examples with Different Users

Permission	Owner (user)	Group	Others	Usage Example
rwxr-x---	user1	eda	---	private file
rw-r--r--	user1	eda	r--	readable by all
rwxrwxr-x	user1	vlsi	r-x	shared in group
rwx------	user1	---	---	highly confidential
drwxr-xr-x (dir)	user1	vlsi	r-x	directory access

20.7 Practical Observation (EDA)

- In EDA projects, directories are usually shared among team members using group permissions.
- Use groups (e.g. eda, vlsi) to allow collaboration.
- Avoid giving unnecessary write/execute permission to others.
- Use chmod and groups to protect important scripts and reports.

Key Points

- ✓ root : UID 0, full control
- ✓ primary user : normal user
- ✓ group : collection of users
- ✓ others : all remaining users
- ✓ 10 characters show type and permissions
- ✓ chmod changes permissions
- ✓ Use groups for project collaboration

Viva / Interview Questions

Q1. What is the UID of root user ? → 0
Q2. Explain -rwxr-x--- ? → owner: rwx, group: r-x, others: ---
Q3. How do you give execute permission to a file ? → chmod +x file
Q4. Why are groups important in EDA environments ? → for shared access

21. CHMOD - CHANGING USERS - U , G , O , +, R , W , X , - , = , +

21.1 Linux Users Overview

- Linux is a multi-user operating system. Each user is identified by a unique User ID (UID) and belongs to a group (GID).
- File permissions decide who can read (r), write (w) or execute (x) a file/directory.
- Users are classified as : root, primary (normal) user, group members and others.
- We can change file permissions using chmod.

21.2 Types of Users

1) Root User (Superuser)

- Username : root UID : 0
- Has full control over the system.
- Can perform any operation (install, configure, modify, delete).
- Use with care. Usually accessed using sudo or su.

2) Primary User (Normal User)

- Created for regular work.
- Has limited permissions.
- Each user has a primary group (same name as username by default).

3) Group

- A group is a collection of users.
- Used to manage permissions on shared files/directories.
- A user can belong to one primary group and multiple secondary groups.

4) Others

- Refers to all remaining users who are not the owner and not in the group.
- They have the most limited access by default.

21.3 The 10 Character Permission String

```
-rwxr-x---
1 2 3 4 5 6 7 8 9 10
```

1	→	file type (- regular, d directory, l link etc.)
2-4	→	owner permissions (r w x)
5-7	→	group permissions (r w x)
8-10	→	others permissions (r w x)

21.4 Permission Characters

Symbol	Meaning	Value
r	read permission	4
w	write permission	2
x	execute permission	1
-	no permission	0

Numeric Combinations

7 = rwx
6 = rw-
5 = r-x
4 = r--
3 = -wx
2 = -w-
1 = --x
0 = ---

21.5 chmod Command

- chmod is used to change file or directory permissions.
- Permissions can be specified using either symbolic mode or numeric mode.

Syntax :	chmod [options] mode file/dir
Example :	$ chmod 755 script.sh
Result :	Sets owner: rwx, group: r-x, others: r-x

21.6 Symbolic Mode (u, g, o, +, -, =, r, w, x)

Symbol	Meaning	Example	Effect
u	user/owner	chmod u+x file.sh	add execute for owner
g	group	chmod g-w file.txt	remove write for group
o	others	chmod o+r file.txt	add read for others
a	all (u+g+o)	chmod a+r file.txt	add read for everyone
+	add permission	chmod u+x file.sh	add execute permission
-	remove permission	chmod g-w file.txt	remove write permission
=	set exact permission	chmod o=r file.txt	set only read for others
r	read	chmod u+r file.txt	read permission
w	write	chmod g+w file.txt	write permission
x	execute	chmod a+x script.sh	execute permission

21.7 Numeric Mode Examples

Command	Permission (rwx)	Meaning
chmod 755 file.sh	rwxr-xr-x	owner: rwx, group: r-x, others: r-x
chmod 644 file.txt	rw-r--r--	owner: rw-, group: r--, others: r--
chmod 700 dir/	rwx------	owner: rwx, group: ---, others: ---
chmod 777 dir/	rwxrwxrwx	full access (not recommended)
chmod 600 file	rw-------	owner read/write only

21.8 Examples (Symbolic Mode)

```
$ chmod u+x run.sh          # add execute for owner
$ chmod g+w data.txt        # add write for group
$ chmod o-r secret.txt      # remove read for others
$ chmod a=rw file.txt       # set read & write for all
$ chmod ug+x tool.sh        # add execute for owner & group
$ chmod go-rw file.txt      # remove read & write for group and others
```

Viva / Interview Questions

Q1. What does the first character in -rwxr-x--- indicate ?
→ File type (- file, d directory etc.)

Q2. Explain chmod 755.
→ owner: rwx, group: r-x, others: r-x.

Q3. What is the difference between u, g and o?
→ u: owner, g: group, o: others.

Q4. Why is 777 not recommended ?
→ It gives full access to everyone (security risk).

Key Points

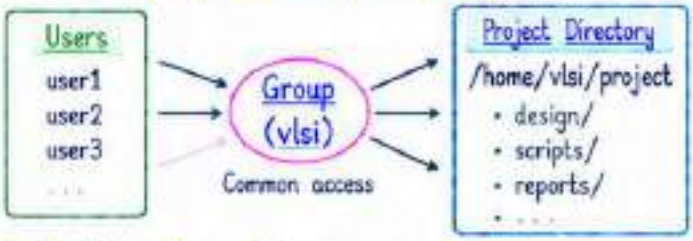
- ✓ Root user has UID 0 and full access.
- ✓ Each user has a primary group.
- ✓ Use chmod to change permissions.
- ✓ u, g, o represent owner, group and others.
- ✓ + adds, - removes, = sets exact permissions.
- ✓ r, w, x represent read, write and execute.
- ✓ Use appropriate permissions in EDA projects for security.

22. SIGNIFICANCE OF GROUPS IN PROJECTS, USING CHMOD (0-7), CHANGING OWNERS AND GROUPS

22.1 Significance of Groups in Projects

- A group is a collection of users.
- Groups allow multiple users to share files and directories in a controlled manner.
- In EDA / VLSI projects, team members are added to a common project group (e.g. pd, vlsi, design).
- Files/directories can be given group permissions so that all team members can read/write/execute based on need.
- Helps in collaboration, data sharing and security.
- Avoids giving full permissions to others.
- Example : Project directory owned by a user but group is set to 'vlsi' so that all vlsi team members can access it.

Project Sharing using Group



22.2 User Types Recap

Type	Description	Example
Root user	Superuser, full control	root (UID 0)
Primary user	Normal user, has a primary group	user1
Group	Collection of users	vlsi, pd
Others	All remaining users	...

22.3 chmod - Absolute (Numeric) Permissions (0 - 7)

- Permissions are represented using 3 digits (owner, group, others).
- Each digit is a combination of read (4), write (2) and execute (1).

Octal	Permission	Meaning
0	---	no permission
1	--x	execute only
2	-w-	write only
3	-wx	write + execute
4	r--	read only
5	r-x	read + execute
6	rw-	read + write
6	rw-	read + write
7	rwx	read + write + execute

Calculation

e.g. 755
7 = 4 + 2 + 1 (rwx)
5 = 4 + 1 (r-x)
5 = 4 + 1 (r-x)
→ Owner : rwx
→ Group : r-x
→ Others : r-x

22.4 chmod - Symbolic Permissions

Symbol	Meaning	Example
u	user (owner)	chmod u+x file
g	group	chmod g-w file
o	others	chmod o+r file
a	all (u+g+o)	chmod a=r file
+	add permission	chmod u+x file
-	remove permission	chmod g-w file
=	set exact permission	chmod o=r file
r	read	-
w	write	-
x	execute	-

22.5 chmod Examples

Command	Result / Meaning
chmod 755 script.sh	Owner:rwx Group:r-x Others:r-x
chmod 644 file.txt	Owner:rw- Group:r-- Others:r--
chmod 700 dir/	Owner:rwx Group:--- Others:---
chmod 777 temp/	Everyone full access (not recommended)
chmod u+x run.sh	Add execute permission for owner
chmod g+w file.txt	Add write permission for group
chmod a-r file.txt	Remove read permission for others
chmod a=r file.txt	Set only read permission for all
chmod go-rw file.txt	Remove read & write for group and others

22.6 Changing Owners and Groups

- Owner of a file/directory can be changed using chown command.
- Group ownership can be changed using chgrp command.
- Syntax and examples are given below.

Command	Meaning	Example
chown user file	Change owner	chown user1 file.txt
chown user:group file	Change owner and group	chown user1:vlsi file.txt
chgrp group file	Change group	chgrp vlsi file.txt
chown -R user:group dir/	Recursive change	chown -R user1:vlsi project/

Example Scenario

```
$ ls -l project/
drwxr-x--- 5 user1 vlsi 4096 Aug 10 10:30 project/
$ chown -R user2:vlsi project/
$ ls -l project/
drwxr-x--- 5 user2 vlsi 4096 Aug 10 10:35 project/
```

22.7 Practical Use in EDA Projects

- Create a common group (e.g. vlsi) for all team members.
- Set group ownership for project directories.
- Use chmod to give appropriate permissions.
- Avoid using 777 (security risk).
- Example: /home/vlsi/my_design with group vlsi and permission 775 so that all team members can work on the project.

Viva / Interview Questions

Q1. What is the UID of root ?
→ 0

Q2. What does 755 mean ?
→ rwx r-x r-x

Q3. Difference between chown and chgrp ?
→ chown changes owner (and group), chgrp changes only group.

Q4. Why use groups in projects ?
→ To share files among team members with controlled permissions.

Key Points

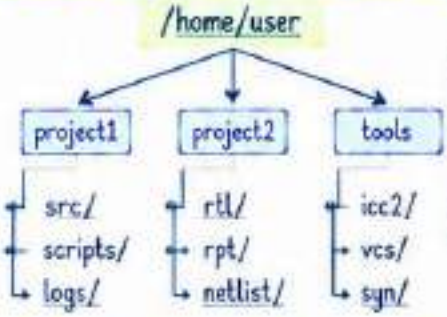
- ✓ Root user has full control.
- ✓ Groups enable collaboration.
- ✓ Numeric permissions (0-7) are commonly used.
- ✓ Use chmod, chown, chgrp to manage access.
- ✓ Avoid 777 permissions.
- ✓ Set group ownership for EDA project directories.
- ✓ Use -R for recursive changes.

23. MOVING DIRECTORY - CD , MV , CP

23.1 Introduction

- In Linux, we often need to navigate between directories and move or copy files/directories.
- cd is used to change the current working directory.
- mv is used to move (rename) files/directories.
- cp is used to copy files/directories.
- These commands are very important in EDA projects to organize scripts, logs, reports and design files.

23.2 Directory Structure (Example)



Absolute Path :
/home/user/project1

Relative Path :
project1/scripts

23.3 cd Command (Change Directory)

- Used to move from one directory to another.
- Changes the current working directory.

Syntax : cd [directory]

Command	Meaning	Example
cd dir	go to a directory	cd project1
cd ..	go to parent directory	cd ..
cd /	go to root directory	cd /
cd ~	go to home directory	cd ~
cd -	go to previous directory	cd -

Example :

```
$ pwd
 /home/user
$ cd project1      # move to project1
$ pwd
 /home/user/project1
$ cd ..            # go back to parent
$ pwd
 /home/user
```

Special Directories

. → current directory
.. → parent directory
/ → root directory
~ → home directory
- → previous directory

23.4 mv Command (Move / Rename)

- Used to move files/directories to another location.
- Also used to rename a file/directory.

Syntax : mv [source] [destination]

```
$ mv file.txt report.txt      # rename file
$ mv report.txt ../backup/    # move to another dir
$ mv dir1 dir2                # rename directory
$ mv dir1 /home/user/         # move directory
```

23.5 cp Command (Copy)

- Used to copy files or directories.
- Original file remains unchanged.

Syntax : cp [options] [source] [destination]

Command	Meaning	Example
cp file1 file2	copy file	cp a.txt b.txt
cp file /path/	copy file to dir	cp run.sh scripts/
cp -r dir1 dir2	copy directory (recursive)	cp -r project1 project2
cp -i file dir/	ask before overwrite	cp -i file.txt backup/
cp -v file dir/	show what is copied	cp -v file.txt logs/

Example :

```
$ cp script.sh scripts/      # copy file
$ cp -r reports/ backup/     # copy directory
$ cp -v *.rpt logs/          # copy all .rpt files (with details)
```

23.6 Differences Between cd, mv and cp

Command	Purpose	Changes original?	Works for
cd	Change directory	No	Directories
mv	Move / Rename	Yes (moved/renamed)	Files & Directories
cp	Copy	No (keeps original)	Files & Directories

23.7 Practical Examples in EDA Project

```
$ cd /home/user/vlsi_project    # go to project directory
$ mkdir new_run                 # create a run directory
$ cp -r scripts/ new_run/       # copy scripts
$ mv old_run/ archive/          # move old run to archive
$ cd new_run                    # enter the new run
$ pwd
 /home/user/vlsi_project/new_run
```

Viva / Interview Questions

Q1. What does cd - do?
→ It changes to the previous directory.

Q2. What is the difference between mv and cp?
→ mv moves (renames) the file, cp creates a copy and original remains.

Q3. How to copy an entire directory?
→ Use cp -r dir1 dir2

Q4. Does cd change files?
→ No. It only changes the current directory.

Key Points

- ✓ Use cd to navigate between directories.
- ✓ Use mv to move or rename files/directories.
- ✓ Use cp to create a copy.
- ✓ Use cp -r for directories.
- ✓ Use options like -i and -v when required.
- ✓ Organize project files properly in EDA environments.
- ✓ Always verify with pwd and ls after operations.

LINUX FOR ECE / VLSI & EDA

Module 7

File Operations, Navigation & Editors

24. EDITING A FILES - WE CAN USE DOCS AS WELL AS VI , GVIM , NOTEPAD , NOTEPAD++

24.1 Introduction

- In Linux, a file can be created and edited using different text editors.
- Choosing the right editor depends on the environment (terminal/ GUI) and the type of work (small edits / large files).
- Common editors : vi , gvim , nano (often called simple editor), and GUI editors like gedit (docs).
- In Windows environment (e.g., using WSL/Cygwin), Notepad and Notepad++ can also be used.
- Editors are essential for modifying scripts (.sh), config files, logs and reports in EDA projects.

24.2 Using Docs (gedit)

- Docs refers to a simple GUI text editor (gedit) in Linux.
- Easy to use with mouse, useful for small file edits.
- Command : gedit filename & (opens in background)

Example :

```
$ gedit notes.txt &          # open text file in gedit
$ gedit ~/project/run.sh &
```

24.3 gvim Editor (GUI based vi)

- gvim is the graphical version of vi.
- Provides all vi features with a GUI (menus, mouse, syntax highlighting).
- Useful for editing large files (e.g., scripts, configuration).

Example :

```
$ gvim script.sh &           # open file in gvim (background)
$ gvim &                     # open gvim without a file
```

24.5 Notepad and Notepad++ (Windows)

- If you are working in Windows (e.g., through WSL, Cygwin or network share), you can use Notepad or Notepad++.
- Notepad : basic text editor (simple).
- Notepad++ : advanced editor with syntax highlighting, line numbers, search, etc.

Example :

```
# Open a file from Windows explorer
notepad run.sh               # open in Notepad
notepad++ run.sh             # open in Notepad++
```

24.6 Comparison of Editors

Editor	Type	Environment	Features	Use Case
gedit (Docs)	GUI	Linux	Simple GUI	Small edits
vi	CLI	Linux	Powerful, default	Server/remote
gvim	GUI	Linux	vi + GUI	Large files
notepad	GUI	Windows	Basic editor	Simple edits
notepad++	GUI	Windows	Advanced, syntax	Code / scripts

24.4 vi Editor (Terminal based)

- vi is a powerful and widely used text editor in Linux.
- Works in terminal, available on all Linux systems.
- Modes : Command mode , Insert mode , Ex mode.

Mode	How to enter	Common commands	Purpose
Command	default	h j k l , dd , yy	navigate, edit commands
Insert	i / a / o	type text	insert text
Ex	:	:w :q :wq :q!	save / quit

Example :

```
$ vi run.sh                  # open file in vi
  i                          # enter insert mode
  (type text)
  Esc                        # back to command mode
  :wq                        # save and quit
```

24.7 Other Useful Editor Commands

Command	Description
vi filename	open file in vi
gvim filename &	open file in gvim (background)
gedit filename &	open file in gedit
notepad filename	open file in notepad (Windows)
notepad++ filename	open file in notepad++ (Windows)

24.8 Practical Examples

```
$ gedit config.txt &         # edit using docs
$ vi run.sh                  # edit using vi
$ gvim design.tcl &          # edit using gvim
  notepad notes.txt          # open in Notepad (Windows)
  notepad++ script.sh        # open in Notepad++ (Windows)
```

Viva / Interview Questions

Q1. Which editor is available by default in all Linux systems ?
→ vi

Q2. What is the difference between vi and gvim ?
→ vi (terminal) , gvim (GUI)

Q3. When do you prefer Notepad++ ?
→ For editing code/scripts with syntax highlighting in Windows.

Q4. How to save and exit in vi ?
→ :wq

Key Points

- ✓ Use vi for remote servers.
- ✓ Use gvim or gedit (docs) for GUI editing.
- ✓ Notepad/Notepad++ can be used in Windows environment (WSL/Cygwin).
- ✓ Editors are essential for EDA scripts, config files, logs and reports.
- ✓ Choose editor based on convenience and file size.

25. DISPLAYING THE FILE CONTENTS - CAT , CAT -b AND HISTORY COMMAND

25.1 Introduction

- In Linux, we can display the contents of a file using cat command.
- cat -b option displays non-empty lines with line numbers.
- history command shows previously executed commands in the terminal.
- These are useful for quick viewing, debugging logs, scripts and checking past commands in EDA workflows.

25.2 cat Command

- cat (concatenate) is used to display the entire content of a file on the terminal.
- It can also be used to display multiple files.

Syntax : `cat [options] [file ...]`

Example :

```
$ cat file1.txt                  # display file content
  VLSI Design Flow
  Linux commands are useful
  Good Practice
$ cat file1.txt file2.txt        # display multiple files
$ cat /home/user/notes.txt       # display using
                                   absolute path
```

25.3 cat -b Command

- cat -b displays non-empty lines with line numbers.
- Empty lines are not numbered.
- Useful for viewing scripts or log files with meaningful line numbers (ignoring blank lines).

Syntax : `cat -b [file]`

Example :

```
$ cat -b script.sh
  1   #!/bin/bash
  2   echo "Starting run"      # non-empty
  3   # run synthesis            lines numbered
  4   make                     # blank line
                               # not numbered
  5   echo "Done"
```

25.4 history Command

- history shows the list of previously executed commands in the current shell session.
- Very useful to recall long commands in EDA flows.
- Each command is shown with a line number.

Syntax : `history [n]` # n = number of commands

Example :

```
$ history
  987  cd project          # show all previous commands
  988  ls -ltr
  989  vi run.sh
  990  cat log.txt
  991  make all
$ history 5                # show last 5 commands
$ !990                     # re-execute command number 990
```

Note :
- Use ↑ (up arrow) to recall previous command.
- Use !n to run a specific command.
- Use history -c to clear history.

25.5 Other Editors (Quick Overview)

Editor	Environment	Purpose	Command / Example
vi	Linux (CLI)	Text editor (default)	vi file.txt
gvim	Linux (GUI)	Graphical vi	gvim file.txt
gedit (docs)	Linux (GUI)	Simple GUI editor	gedit file.txt
notepad	Windows	Basic text editor	notepad file.txt
notepad++	Windows	Advanced editor (syntax highlight)	notepad++ file.txt

Practical Examples in EDA

```
$ cat run.log              # view log file
$ cat -b run.sh            # view script with line nos
$ history                  # check previous commands
$ gvim design.tcl          # edit tcl file
$ gedit notes.txt          # open using docs
$ notepad++ report.rpt     # open in notepad++ (win)
```

Key Points

- ✓ cat displays file content.
- ✓ cat -b numbers non-empty lines.
- ✓ history helps to recall commands.
- ✓ Use suitable editor based on environment (CLI/GUI).
- ✓ Useful for viewing and editing EDA scripts, logs and reports.

26. LINKING GVIM TO CYGWIN

26.1 Introduction

- Cygwin provides a Unix-like environment in Windows.
- GVIM is the graphical version of vi (Vim).
- By linking GVIM to Cygwin, we can open and edit files from the Cygwin terminal using the gvim command.
- This is very useful in EDA projects where we work on remote / local files through Cygwin but want a GUI editor.

26.2 Prerequisites

- Cygwin installed and working.
- GVIM (from Vim) installed in Windows.
- Note the installation path of gvim.exe (e.g., C:\Program Files\Vim\gvim.exe)

26.3 Method 1 : Add Windows gvim to PATH

- 1. Find the gvim.exe path in Windows.
- .Convert Windows path to Cygwin path.
- .Add it to your .bashrc or .bash_profile.

Example :

```
$ cygpath "C:\Program Files\Vim"      # convert to
  /cygdrive/c/Program Files/Vim         Cygwin path
$ echo 'export PATH=$PATH:/cygdrive/c/Program\ Files
  /Vim' >> ~/.bashrc                  # add to PATH
$ source ~/.bashrc                    # apply changes
$ which gvim                          # check
  /cygdrive/c/Program Files/Vim/gvim
```

26.4 Method 2 : Create an Alias

- Instead of modifying PATH, we can create an alias.
- Add the following line in ~/.bashrc

Example :

```
$ alias gvim='/cygdrive/c/Program\ Files/Vim/gvim.exe'
$ source ~/.bashrc
$ gvim &                    # open gvim
```

26.5 Verifying the Setup

- Run gvim from Cygwin terminal.
- It should open the GVIM editor with the specified file.

Example :

```
$ gvim test.txt &       # open file in gvim
$ gvim &                # open gvim without file
```

26.6 How it Works?



- Cygwin translates the command to the Windows executable using the provided path/alias.
- GVIM opens as a separate GUI window to edit the file.

26.7 Useful Commands

Command	Purpose
cygpath <win_path>	Convert Windows path to Cygwin path
echo $PATH	Display current PATH (Cygwin)
which gvim	Show gvim path in Cygwin
alias gvim='...'	Create alias for gvim
gvim filename &	Open file in GVIM (background)
gvim &	Open GVIM without a file

26.8 Example Session

```
$ which gvim
  /cygdrive/c/Program Files/Vim/gvim   # verify path
$ gvim design.tcl &                    # open file
$ gvim &                               # open gvim
$ ls
  design.tcl  run.sh  notes.txt
$ gvim notes.txt &                     # edit notes
```

26.9 Alternative Editors (also usable in Cygwin)

Editor	Command (in Cygwin)	Description
vi	vi file.txt	Terminal based editor
gvim	gvim file.txt &	GUI version of vi
gedit	gedit file.txt &	Simple GUI editor (Linux, if available)
notepad	notepad file.txt &	Windows Notepad (from Cygwin)
notepad++	notepad++ file.txt &	Advanced editor (Windows)

Key Points

- ✓ Linking GVIM with Cygwin provides a GUI editing capability.
- ✓ Use cygpath to convert Windows paths.
- ✓ Add to PATH or create an alias.
- ✓ Useful for editing large scripts, config files in EDA projects.
- ✓ Other editors like notepad and notepad++ can also be used.

27. .BASHRC & .TCSHRC - EXPLAIN THIS IMPORTANCE . HOW TO SET ALIAS IN BASHRC & TCSHRC .

27.1 Introduction

- .bashrc and .tcshrc are startup configuration files for bash and tcsh shells respectively
- These files are read when a new shell is started (interactive shell)
- They allow us to set environment variables, define aliases, functions and customize the shell behavior
- Using alias we can create short, user-friendly commands to save time and reduce typing errors

27.2 Importance

- Helps to configure the working environment
- Makes repetitive commands easier with aliases
- Ensures a consistent setup across sessions
- Useful in EDA flows to set tool paths and aliases for frequently used commands

27.3 .bashrc (Bash Shell)

- Location : ~/.bashrc
- Executed for interactive bash shells
- Commonly used to set PATH, prompt, aliases etc

Example :

```
$ gvim ~/.bashrc
# add alias
  alias ll='ls -ltr'
  alias run='cd ~/project && ls'
$ source ~/.bashrc          # apply changes
$ ll                        # uses alias
```

27.4 .tcshrc (C Shell)

- Location : ~/.tcshrc
- Executed for interactive tcsh shells
- Used to set environment variables, aliases, prompt etc

Example :

```
$ gvim ~/.tcshrc
# set alias (tcsh syntax)        # cont use (tcsh syntax)
  alias ll 'ls -ltr'
  alias proj 'cd ~/project; ls'
$ source ~/.tcshrc          # apply changes
$ proj                      # uses alias
```

27.5 Setting Aliases

(a) In .bashrc (Bash)

Command	Purpose
alias name='command'	Create an alias
unalias name	Remove an alias
alias -p	Show all aliases

Example :

```
$ alias ll='ls -ltr'
$ ll                        # same as ls -ltr
$ unalias ll
$ ll                        # original command
```

(b) In .tcshrc (C Shell)

Command	Purpose
alias name 'command'	Create an alias
unalias name	Remove an alias
alias	Show all aliases

Example :

```
$ alias ll 'ls -ltr'
$ ll                        # same as ls -ltr
$ unalias ll
$ ll                        # original command
```

27.6 Combined Example (Both Shells)

.bashrc (bash)

```
alias ll='ls -ltr'
alias proj='cd ~/project'
```

.tcshrc (tcsh)

```
alias ll 'ls -ltr'
alias proj 'cd ~/project'
```

Usage :

```
$ proj
$ ll                        <- uses alias
$ . .                       # opens project directory
                            # lists files (long format)
```

KEY POINT

- .bashrc → bash shell | .tcshrc → tcsh shell
- Used to set environment, aliases, functions, prompt
- alias helps to create short and convenient commands
- Use source to apply changes immediately

VIVA / INTERVIEW POINT

Q. What is the difference between .bashrc and .profile?

→ .bashrc is for interactive shells, .profile is for login shells

28. PUSHD , POPD - EXPLAIN THESE COMMANDS ALONG EXAMPLES TOO

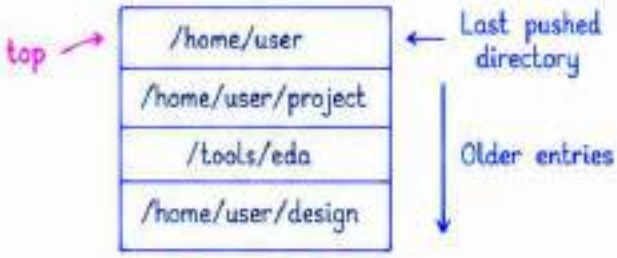
28.1 Introduction

- pushd and popd are shell built-in commands used for directory stack management.
- They allow us to temporarily change directories and return back easily.
- Very useful when working on multiple project directories in EDA flows (e.g., rtl, synth, pnr, reports).
- Available in bash, csh, tcsh and other Unix shells.

28.2 Directory Stack Concept

- A stack maintains the list of directories you visit using pushd.
- pushd <dir> → changes to <dir> and adds current directory to stack.
- popd → returns to previous directory.
- dirs → displays the current directory stack.

Directory Stack (LIFO)



pushd → push to stack
popd → pop from stack

28.3 pushd Command

- Syntax : pushd <directory>
- Changes to the specified directory and pushes the current directory onto the stack.
- If no directory is specified, it swaps the top two directories in the stack.

Examples :

```
$ pwd
  /home/user
$ pushd /tools/eda              # go to /tools/eda
  /tools/eda /home/user
$ pushd ~/project               # go to project
  ~/project /tools/eda /home/user
$ pushd                         # swap top two
  /tools/eda ~/project /home/user
```

28.4 popd Command

- Syntax : popd [+n | -n]
- Removes a directory from the stack and changes to that directory.
- popd alone → goes to the previous directory.
- popd +n → removes n^{th} entry (count from left).
- popd -n → removes n^{th} entry (count from right).

Examples :

```
$ popd                          # back to /tools/eda
  /tools/eda ~/project /home/user
$ popd                          # back to /home/user
  /home/user
$ pushd /data
  /data /home/user
$ pushd /tmp
  /tmp /data /home/user
$ popd +1                       # remove /data and go there
  /data /home/user
```

28.5 dirs Command

- Shows the current directory stack.
- Options: -v (numbered list), -c (clear stack).

Examples :

```
$ dirs                          # current stack
  /home/user
$ pushd /tools
  /tools /home/user
$ pushd /proj
  /proj /tools /home/user
$ dirs -v
  0   /proj
  1   /tools
  2   /home/user
$ dirs -c                       # clear the stack
```

Note :
pushd, popd and dirs are shell built-in commands (not external executables).

Key Points

- ✓ pushd → change directory and push current dir to stack.
- ✓ popd → return to previous directory.
- ✓ dirs → display directory stack.
- ✓ Useful for navigating between multiple project directories in EDA workflows.
- ✓ Works in bash, csh, tcsh (commonly used in EDA servers).

VIVA / INTERVIEW POINT

Q. What is the use of pushd and popd in Linux ?

→ They help in quick navigation between multiple directories using a directory stack, very useful in EDA projects.

29. ENVIRONMENT VARIABLES - $PATH , $TERM - EXPLAIN THESE COMMANDS ALONG EXAMPLES TOO.

29.1 What are Environment Variables ?

- Environment variables are name-value pairs used by the shell and applications to store configuration information.
- They control the behavior of shell, tools and programs.
- Set in shell startup files (.bashrc, .tcshrc, etc).
- Useful in EDA environments to set tool paths, license locations, display settings, etc.

29.2 $PATH Variable

- $PATH contains a list of directories separated by : where the shell looks for executable commands.
- When we type a command, shell searches in each directory in $PATH in order.
- Helps to run tools without giving full path.

Syntax / Example :

```
$ echo $PATH                          # display current PATH
$ export PATH=$PATH:/tools/eda/bin    # add a new directory
$ echo $PATH                          # verify
```

Example Output :

```
/usr/bin:/bin:/usr/local/bin:/tools/eda/bin
```

29.3 $TERM Variable

- $TERM specifies the type of terminal being used.
- It tells applications how to control the screen (colors, cursor movement, clear screen, etc).
- Common values: xterm, xterm-256color, vt100, linux.
- EDA tools and text-based applications use $TERM to set display capabilities.

Example :

```
$ echo $TERM                          # display terminal type
  xterm-256color
```

- We can also set or change it (generally not needed):

```
$ export TERM=xterm
$ echo $TERM                          # verify
  xterm
```

29.4 Useful Commands

Command	Purpose
echo $VAR	Display value of a variable
export VAR=value	Set a variable (current shell)
env	Display all environment variables
set	Display shell variables (bash)
unset VAR	Remove a variable
printenv	Display environment variables

29.5 Examples in EDA Context

- Add EDA tool to PATH:

```
$ export PATH=$PATH:/cad/synopsys/bin
$ echo $PATH                          # check
```

- Set license path (commonly used):

```
$ export LM_LICENSE_FILE=/cad/licenses/license.dat
$ echo $LM_LICENSE_FILE
```

- Set terminal type for better display:

```
$ export TERM=xterm-256color
$ echo $TERM
```

- Use in .bashrc for permanent setup (see 29.6).

29.6 Setting in Shell Startup Files

In .bashrc (Bash)

```
# Add tool paths
export PATH=$PATH:/tools/eda/bin
export LM_LICENSE_FILE=~/licenses.dat
export TERM=xterm-256color
# Aliases
alias ll='ls -ltr'
alias syn='dc_shell'
```

In .tcshrc (C Shell)

```
# Add tool paths
setenv PATH $PATH:/tools/eda/bin
setenv LM_LICENSE_FILE ~/licenses.dat
setenv TERM xterm-256color
# Aliases
alias ll 'ls -ltr'
alias syn 'dc_shell'
```

KEY POINTS

- ✓ $PATH → list of directories for executables
- ✓ $TERM → terminal type (screen capabilities)
- ✓ Set in .bashrc / .tcshrc for permanent setup
- ✓ Use export (bash) and setenv (tcsh)
- ✓ Very useful in EDA flows for tool paths and display settings

VIVA / INTERVIEW POINT

Q. What is $PATH?

→ It is an environment variable that contains the list of directories searched for executable commands.

Q. What is $TERM used for?

→ It specifies the terminal type and controls how applications display output on the screen.

30. HOW TO USER CAN CREATE HIS OWN FILE AND MAKE IT AS EXECUTABLE - EXPLAIN THESE COMMANDS ALONG EXAMPLES TOO.

30.1 Introduction

- A user can create his own file (usually a shell script) and make it executable.
- Used to automate repetitive tasks in Linux and EDA flows.
- We write commands in a file and give execute permission using chmod.
- Executable files can be run from current directory or by specifying full/relative path.

30.2 Creating a file

- We can create a file using touch or directly using an editor like vi / gvim / nano.

Examples :

```
$ touch run.sh          # create an empty file
$ vi run.sh             # open and edit
$ gvim run.sh           # open in GUI editor
```

30.3 Writing content (Shell script)

- First line is called shebang (#!) - tells the shell which interpreter to use.
- Then write the commands to be executed.

Example script : (run.sh)

```
#!/bin/bash
echo "Starting EDA run"
pwd
ls -ltr
echo "Run completed"
```

30.4 Making the file executable

- Use chmod to give execute permission.
- Syntax : chmod +x <filename>

Example :

```
$ chmod +x run.sh       # make it executable
$ ls -l run.sh

-rwxr-xr-x  1 user group 120 Aug 17 10:30 run.sh
```

30.5 Executing the file

- Run the script using ./filename (if in current dir) or using full/relative path.
- Syntax : ./run.sh

Examples :

```
$ ./run.sh                  # execute in current dir
$ /home/user/scripts/run.sh # using absolute path
$ scripts/run.sh            # using relative path
```

30.6 Summary of Commands

Command	Purpose
touch file	Create an empty file
vi file	Open and edit in terminal
gvim file	Open and edit in GUI
chmod +x file	Make file executable
ls -l file	Check permissions
./file	Execute the file

30.7 Practical Example (EDA context)

```
$ vi run_eda.sh                    ( used to run EDA tools,
$ chmod +x run_eda.sh                set env, launch scripts
$ ./run_eda.sh                       etc)
Starting EDA run
/home/user/project
total 8
drwxr-xr-x  2 user group 4096 Aug 17 10:30 logs
-rw-r--r--  1 user group  100 Aug 17 10:30 run.tcl
Run completed
```

Key Points

- ✓ Write commands in a file.
- ✓ Use shebang (#!) on first line.
- ✓ Give execute permission using chmod +x.
- ✓ Run using ./filename.
- ✓ Very useful for automating EDA workflows.

VIVA / INTERVIEW POINT

Q. How do you make a file executable in Linux?
→ Using chmod +x filename.

Q. What is a shebang?
→ It specifies the interpreter (e.g., #!/bin/bash) to execute the script.

31. SETTING THE ENVIRONMENT VARIABLES AND ESCAPE SEQUENCES

31.1 Environment Variables - Introduction

- Environment variables are named values that store configuration information for the shell, programs and EDA tools.
- They control the behavior of the shell and applications.
- Each shell session has its own set of environment variables.
- Common variables : $PATH , $TERM , $HOME , $USER , $SHELL , $PWD etc

31.2 Viewing Environment Variables

Examples :

```
$ echo $PATH     # display PATH variable
$ echo $TERM     # display terminal type
$ echo $HOME     # display home directory
$ echo $USER     # display current user
$ env            # list all environment variables
$ printenv PATH  # another way to print
```

31.3 Setting Environment Variables

- Use export to set a variable (bash/sh)
- Syntax : export VAR=value
- To add to existing variable use $VAR
- Example :

```
$ export MYVAR=eda              # set variable
$ echo $MYVAR                   # display value
 eda
$ export PATH=$PATH:/tools/eda/bin
$ echo $PATH                    # append to PATH
 /usr/bin:/bin:/tools/eda/bin  ...
```

Note :

- To make it permanent, add the export command in .bashrc (bash) or .tcshrc (tcsh)
- After editing, reload using : source .bashrc or source .tcshrc

31.4 Important Variables : $PATH , $TERM

$PATH

- $PATH contains a list of directories where the shell searches for executable commands.
- Directories are separated by : (colon)
- When we type a command, Linux searches these directories in order.

Example :

```
$ echo $PATH
 /usr/bin:/bin:/usr/local/bin:/tools/eda/bin
```

$TERM

- $TERM specifies the type of terminal being used.
- It tells applications how to control the display (colors, cursor movement, etc).
- Common values : xterm , xterm-256color , vt100 , linux etc.

Example :

```
$ echo $TERM                    # terminal type
 xterm-256color
```

31.5 Common Ways to Set Variables

Task	Command (bash)	Command (tcsh)
Set variable	export VAR=value	setenv VAR value
Add to PATH	export PATH=$PATH:dir	setenv PATH $PATH:dir
View variable	echo $VAR	echo $VAR
Make permanent	~/.bashrc	~/.tcshrc
Reload file	source ~/.bashrc	source ~/.tsshrc

31.6 Escape Sequences

- Escape sequences start with a backslash (\\) followed by a special character.
- Used in echo/printf to control terminal output formatting.

Sequence	Meaning	Example (echo)
\n	New line	echo -e "Line1\nLine2"
\t	Horizontal tab	echo -e "A\tB"
\b	Backspace	echo -e "AB\bC" → AC
\r	Carriage return	echo -e "Hello\rWorld"
\\	Print backslash	echo -e "\\"
\e or \033	Escape (for colors)	echo -e "\e[31mRed\e[0m"

31.7 Practical Example (EDA context)

```
$ export EDA_HOME=/tools/eda          # set EDA tool path
$ export PATH=$PATH:$EDA_HOME/bin     # update PATH
$ echo $EDA_HOME                      # verify
 /tools/eda
```

Key Points

- ✓ Environment variables control shell and tools
- ✓ Use export (bash) / setenv (tcsh)
- ✓ $PATH → directories for executables
- ✓ $TERM → terminal type
- ✓ Use \n, \t etc for formatted output

VIVA / INTERVIEW POINT

Q. What is $PATH ?
→ It is a list of directories where Linux searches for executable commands.
Q. What is $TERM ?
→ It specifies the type of terminal used (e.g. xterm).

32. TEXT PROCESSING SUCH AS (*, ., ?, ^ , $, \ , \.)

32.1 Introduction

- These are special characters (metacharacters) used in shell and text processing commands (like ls, grep, sed).
- They help in pattern matching, file selection and text search.
- Used to represent patterns instead of exact strings.
- Very useful in EDA flow for filtering log files, reports, netlists etc.

32.2 List of Metacharacters

Symbol	Meaning	Example (use)
*	Matches zero or more characters	ls *.log (all .log files)
.	Matches any single character	ls run?.rpt (run1.rpt)
?	Matches exactly one character	ls file?.txt (file1.txt)
^	Matches beginning of a line (in grep)	grep ^ERROR file.log
$	Matches end of a line (in grep)	grep "end$" file.log
\	Escape character (treat next as literal)	echo * (prints *)
\.	Literal dot (in grep)	grep "\\.v" (match .v files only)

32.3 Examples with Commands

1. Using * (wildcard)

```
$ ls *.log          # list all .log files
Output : run.log  synth.log  place.log
```

2. Using ? (single character)

```
$ ls run?.rpt       # matches run1.rpt, run2.rpt
Output : run1.rpt  run2.rpt
```

3. Using . (any character)

```
$ ls file?.txt      # file1.txt, fileA.txt
Output : file1.txt  fileA.txt
```

4. Using ^ (start of line) in grep

```
$ grep ^ERROR run.log   # lines starting with ERROR
  ERROR : Timing failed
  ERROR : Memory overflow
```

5. Using $ (end of line) in grep

```
$ grep "Done$" run.log   # lines ending with Done
Output :
  Step Completed  Done
  All jobs  Done
```

6. Using \ (escape character)

```
$ echo \\*       # prints * instead of wildcard
Output :  *
```

7. Using \. (literal dot) in grep

```
$ ls *.v        # lists .v and also somexv (not exact)
$ ls \*.v       # (not useful with ls)
$ grep "\\.v" filelist.txt   # match only .v files
Output :
  top.v
  alu.v
```

32.4 Practical Use Cases in EDA

- List all tool log files : ls *.log
- Search for lines starting with WARNING : grep ^WARNING run.log
- Find lines ending with a specific keyword : grep "WNS$" timing.rpt
- Match specific file types : ls *.rpt , ls *.v
- Use escape sequences to search exact characters : grep "\.sdc" filelist.txt

32.5 Important Notes

- Metacharacters are interpreted by the shell.
- Use quotes (" ") when needed to avoid shell expansion.
- Fise \ to escape special characters when matching literally.
- Behaviour may vary slightly between different shells (bash, csh, tcsh).

Key Points

- ✓ * → zero or more characters
- ✓ ? → one character
- ✓ . → any character
- ✓ ^ → start of line (grep)
- ✓ $ → end of line (grep)
- ✓ \\ → escape next character
- ✓ \. → literal dot

VIVA / INTERVIEW POINT

Q. What does * represent ?
→ It matches zero or more characters.

Q. How to search for lines ending with a word ?
→ Use $ (e.g. grep "Done$").

33. GREP - GREP -n , -r , ^ , -i -r -w - v , xargs

33.1 Introduction to grep

- grep (global regular expression print) is used to search for a pattern in files and print matching lines.
- Useful for log files, reports, netlists etc in EDA flow.
- Supports regular expressions and many options.

Syntax : grep [options] "pattern" file(s)

33.2 Metacharacters used with grep

Symbol	Meaning	Example (grep command)
*	Zero or more characters	grep "err*" run.log
.	Any single character	grep "r.n" run.log
?	Exactly one character	grep "gr?y" file.txt
^	Start of a line	grep "^ERROR" run.log
$	End of a line	grep "Done$" run.log
\	Escape character (treat next char literally)	grep "v\.1" file.txt
\.	Literal dot	grep "file\.txt" filelist

33.3 Useful grep options (as per topic)

Option	Meaning	Example
-n	Show line numbers of matches	grep -n "WNS" timing.rpt
-r	Recursive search in directories	grep -r "ERROR" ./logs
^	Pattern at start of line	grep "^WARN" run.log
-i	Case insensitive search	grep -i "error" run.log
-w	Match whole word only	grep -w "WNS" timing.rpt
-v	Invert match (show non-matching)	grep -v "INFO" run.log

33.4 Examples

1. grep -n (line numbers)

```
$ grep -n "Done" run.log    # show line numbers
45: Step Completed Done
87: All jobs Done
```

2. grep -r (recursive search)

```
$ grep -r "ERROR" ./sim    # search in all files
./sim/run1.log: ERROR : Timing failed
./sim/run2.log: ERROR : No convergence
```

3. grep ^ (start of line)

```
$ grep "^WARNING" run.log  # lines starting with WARNING
WARNING : Low slack
WARNING : High temperature
```

4. grep -i (case insensitive)

```
$ grep -i "error" run.log  # match error, ERROR, Error
ERROR : File not found
Error : Cannot open file
```

5. grep -w (whole word)

```
$ grep -w "WNS" timing.rpt  # exact word only
WNS = 0.123
WNS = -0.045
```

6. grep -v (invert match)

```
$ grep -v "INFO" run.log  # show lines not having INFO
WARNING : Less memory
ERROR : Tool crashed
```

7. Using xargs with grep

```
$ find . -name "*.log" | xargs grep "ERROR"
./run1.log: ERROR : failed
./run2.log: ERROR : timeout
```

Key Points

- ✓ * → zero or more characters
- ✓ . → any single character
- ✓ ? → exactly one character
- ✓ ^ → start of line
- ✓ $ → end of line
- ✓ \ → escape next character
- ✓ -n → line numbers
- ✓ -r → recursive search
- ✓ -i → ignore case
- ✓ -w → whole word
- ✓ -v → invert match
- ✓ xargs → pass output as arguments

Practical Use in EDA

- Search for errors in large logs
- Filter specific warnings
- Find patterns across directories
- Useful in regression runs

VIVA / INTERVIEW POINT

Q. What does grep -r do ?
→ Searches recursively in subdirectories

Q. How to ignore case in grep ?
→ Use -i option.

34. SED (STREAMLINE EDITOR)

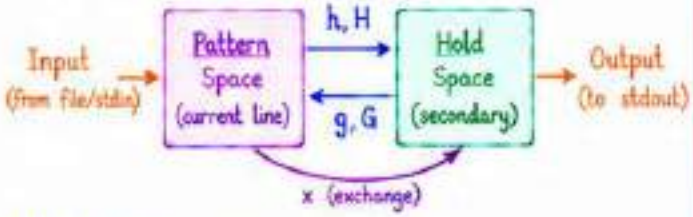
34.1 What is sed ?

- sed (stream editor) is a non-interactive text processing tool.
- Reads input line by line, performs editing based on commands and prints modified output.
- Useful for search, replace, delete, insert, transform text in files or from standard input.
- Very useful in EDA flows to process log files, reports and data files automatically.

Syntax : sed [options] 'command(s)' file(s)

34.2 Pattern space and Hold space

- Pattern space : temporary space that holds the current line (or lines) being processed.
- Hold space : secondary buffer used to store text temporarily. Used with commands like h, H, g, G, x.
- Most sed commands work on the pattern space. Some use hold space for advanced operations.



34.3 Basic usage examples

```
# print file (no change)
  $ sed 'p' file.txt
# substitute first match in each line
  $ sed 's/old/new/' file.txt
# substitute all matches
  $ sed 's/old/new/g' file.txt
# delete lines matching pattern
  $ sed '/ERROR/d' run.log
# insert a line before pattern
  $ sed '/START/i Initialization' run.log
```

34.4 Practical use in EDA

- Modify tool logs (replace paths, filter patterns)
- Extract specific information from reports
- Clean and format data files
- Useful in automation scripts

Key Point

- ✔ sed works on pattern space.
- ✔ Use -e for multiple commands.
- ✔ Very powerful for batch text processing.
- ✔ Combine with grep, awk in EDA flows.

34.5 Common sed commands (25 examples)

No.	Command	Purpose	Example
1	s/old/new/	Replace first match	sed 's/a/b/' file
2	s/old/new/g	Replace all matches	sed 's/a/b/g' file
3	s/old/new/2	Replace 2nd match	sed 's/a/b/2' file
4	p	Print line	sed -n 'p' file
5	d	Delete line	sed '/ERROR/d' file
6	i \text	Insert before line	sed '/PAT/i hello' file
7	a \text	Append after line	sed '/PAT/a hello' file
8	c \text	Change entire line	sed '/PAT/c new line' file
9	n	Read next line	sed 'n' file
10	N	Append next line	sed 'N' file
11	h	Copy to hold space	sed 'h' file
12	H	Append to hold space	sed 'H' file
13	g	Get from hold space	sed 'g' file
14	G	Append from hold space	sed 'G' file
15	x	Exchange pattern & hold	sed 'x' file
16	y/abc/xyz/	Translate chars	sed 'y/abc/xyz/' file
17	=	Print line numbers	sed '=' file
18	l	List with escapes	sed -n 'l' file
19	q	Quit (stop processing)	sed '10q' file
20	r file	Read file and insert	sed '1r header.txt' file
21	w file	Write to file	sed '/ERROR/w err.log' file
22	-e cmd	Multiple commands	sed -e 's/a/b/' -e 'p' file
23	-f script	Read commands from file	sed -f cmds.sed file
24	/pattern/	Address / match pattern	sed '/WARN/p' file
25	!	Negate address	sed '/INFO/!d' file

VIVA / INTERVIEW POINT

Q. What is sed ?
A stream editor used for non-interactive text processing.

Q. What is pattern space ?
It holds the current line being processed.

Q. What is hold space ?
A secondary buffer to store text (h, g, x etc).

35. SED SEARCH AND SUBSTITUTE

35.1 Search and Substitute in sed

- sed (stream editor) is used to perform text transformations on a file or input stream.
- Search and substitute (s) is one of the most commonly used operations in sed.
- Useful to replace strings, modify patterns, clean logs, update configuration files etc.

- Syntax : `sed 's/pattern/replacement/flags' file`

- Example :
  `$ sed 's/old/new/' file.txt`
- It replaces the first occurrence of "old" with "new", in each line.

35.2 Pattern matching and substitution

- sed searches for a pattern (regular expression), in the pattern space and replaces it with the given string.
- By default only the first match in each line is replaced.
- Use 'g' flag to replace all matches.

General form:
`sed 's/REGEX/REPLACE/FLAGS' file`

Common flags:
g → global (all matches)
i → ignore case
number → specific occurrence
e.g. 2 → second match

35.3 Example

file.txt :

```
1 VLSI design flow
2 vlsi verification
3 VLSI Design
4 ASIC vlsi flow
```

Command :
`$ sed 's/VLSI/EDA/g' file.txt`

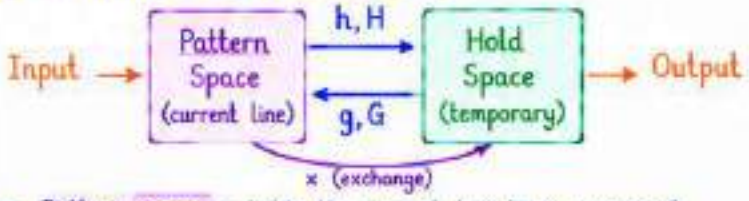
Output :

```
EDA design flow
eda verification
EDA Design
ASIC eda flow
```

35.4 Practical use in EDA

- Modify tool logs (replace paths, update names)
- Clean up reports (remove unwanted strings)
- Automate configuration file changes
- Useful in batch scripts and regression flows

35.5 Pattern and Hold Space



- Pattern space : holds the current line being processed.
- Hold space : secondary buffer to store text temporarily.
- Most search and substitute operations work on pattern space.
- Hold space is useful for advanced text manipulation.

35.6 25 sed commands for Search and Substitute

No.	Command	Description	Example
1	s/old/new/	Replace first match	sed 's/a/b/' file
2	s/old/new/g	Replace all matches	sed 's/a/b/g' file
3	s/old/new/2	Replace 2nd match	sed 's/a/b/2' file
4	s/old/new/Ig	Ignore case, all	sed 's/a/b/Ig' file
5	s/old/new/p	Print changed lines	sed 's/a/b/p' file
6	/pattern/s//new/	Search pattern then replace	sed '/error/s//WARN/' file
7	s/^old/new/	Replace at line start	sed 's/^a/b/' file
8	s/old$/new/	Replace at line end	sed 's/a$/b/' file
9	s/[0-9]/#/g	Use regex in replace	sed 's/[0-9]/#/g' file
10	s/\t/ /g	Replace tab with space	sed 's/\t/ /g' file
11	s/ */ /g	Remove extra spaces	sed 's/ */ /g' file
12	s/./_/g	Replace dot	sed 's/\./_/g' file
13	s/\/-/g	Replace slash	sed 's/\/-/g' file
14	s/old/new/gI	Case-insensitive (I)	sed 's/a/b/gI' file
15	s#old#new#g	Use # as delimiter	sed 's#/#-#g' file
16	s|old|new|g	Use | as delimiter	sed 's|:|-|g' file
17	y/abc/xyz/	Translate chars	sed 'y/abc/xyz/' file
18	s/old//g	Delete a pattern	sed 's/ERROR//g' file
19	s/.*/new/	Replace whole line	sed 's/.*/new/' file
20	n;s/old/new/	Next line then replace	sed 'n;s/a/b/' file
21	1,5s/old/new/g	Range of lines	sed '1,5s/a/b/g' file
22	s/old/new/w out	Write changed lines	sed 's/a/b/w out.txt' file
23	/pat/!s/old/new/	Negate pattern	sed '/test/!s/a/b/' file
24	s/\<old\>/new/g	Whole word replace	sed 's/\<vlsi\>/eda/g' file
25	s/\n/ /g	Join lines (newline→space)	sed 's/\n/ /g' file

Key Point

- ✓ Use 's' command for substitution.
- ✓ Use 'g' for all matches.
- ✓ sed works on pattern space.
- ✓ Useful for log and report processing in EDA.

VIVA / Interview Point

Q. What does sed s command do ?
→ It searches for a pattern and substitutes it with a new string.
Q. How to replace all matches ?
→ Use the 'g' flag : sed 's/old/new/g' file

36. SED TO UPDATE TEXT FILES

36.1 Introduction

- sed (stream editor) is a powerful tool used to search, modify, delete, insert or transform text in files or input streams.
- It reads input line by line, performs the specified editing commands and writes the output.
- Very useful in log file processing, report editing and EDA flow automation.

36.2 Substitution for Updating Text

- The "s" (substitute) command is most commonly used to update/replace text in a file.
- Syntax : `sed "s/pattern/replacement/flags" file`
- Common flags :
  - g → replace all occurrences in a line
  - i → ignore case
  - number → replace only nth occurrence
  - p → print modified lines
  - w file → write result to a file

36.3 Updating File (In-place edit)

- Use -i option to directly update the file.
- Syntax : `sed -i "s/old/new/g" file`
- Create a backup : `sed -i.bak "s/old/new/g" file`
- Useful for configuration files, scripts, EDA reports.

36.4 Example

Input file (config.txt) :

```
1  TOOL=old_tool
2  MODE=fast
3  PATH=/usr/bin
4  TOOL=old_tool
```

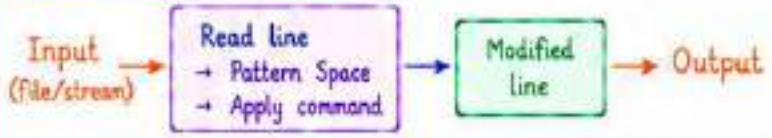
Command :

```
$ sed "s/old_tool/new_tool/g" config.txt
```

Output :

```
1  TOOL=new_tool
2  MODE=fast
3  PATH=/usr/bin
4  TOOL=new_tool
```

36.5 sed processing flow



36.6 Common use cases to update text files

- Update tool paths, version numbers in scripts.
- Modify configuration parameters.
- Replace environment variables.
- Clean and standardize log files.
- Automate text changes in batch flows.

36.7 Useful sed commands for updating text files

No.	Sed Command	Description	Example
1	s/old/new/	Replace first occurrence	sed "s/a/b/" file
2	s/old/new/g	Replace all occurrences	sed "s/a/b/g" file
3	s/old/new/2	Replace 2nd occurrence	sed "s/a/b/2" file
4	s/old/new/3	Replace 3rd occurrence	sed "s/a/b/3" file
5	s/old/new/ig	Replace all (ignore case)	sed "s/a/b/ig" file
6	s/^old/new/	Replace at line start	sed "s/^a/b/" file
7	s/old$/new/	Replace at line end	sed "s/a$/b/" file
8	s/old/new/p	Print modified lines	sed "s/a/b/p" file
9	s/old/new/w out	Write result to file	sed "s/a/b/w out" file
10	/pattern/s//new/	Search pattern then replace	sed "/err/s//WARN/" file
11	1,5s/old/new/g	Replace in line range	sed "1,5s/a/b/g" file
12	/pat1/,/pat2/s/a/b/g	Replace between patterns	sed "/START/,/END/s/a/b/g"
13	s/old/new/gI	Case-insensitive (I)	sed "s/a/b/gI" file
14	s/a/b/;s/c/d/	Multiple substitutions	sed "s/a/b/;s/c/d" file
15	y/abc/xyz/	Translate characters	sed "y/abc/xyz/" file
16	d	Delete matching line	sed "/temp/d" file
17	1d	Delete first line	sed "1d" file
18	$d	Delete last line	sed "$d" file
19	i\text	Insert line before	sed "i\# new line" file
20	a\text	Append line after	sed "a\# end" file
21	c\text	Change the line	sed "2c\MODE=slow" file
22	r file	Read and insert file	sed "2r extra.txt" file
23	w file	Write pattern space to file	sed "/WARN/w warn.log" file
24	n	Read next line	sed "n" file
25	p	Print pattern space	sed -n "p" file

Practical Observation

- sed is widely used in EDA to update run scripts, tool config files and reports.
- Helps in automating repetitive edits.
- Use -i.bak for safety.

Key Point

- ✓ Use "s" command for substitution.
- ✓ Use -i to update files directly.
- ✓ Regular expressions can be used.
- ✓ Very useful for log/report processing in EDA flows.

VIVA / Interview Point

Q. What does sed "s/old/new/g" do ?
→ Replaces all occurrences of "old" with "new" in each line.

Q. How to update a file in-place ?
→ sed -i "s/old/new/g" file

37. AWK COMMANDS , SHORT NOTATIONS

37.1 What is AWK ?

- AWK is a powerful text processing language used to search, pattern match, and process structured text files (line by line)
- It splits each input line into fields (columns) and allows processing based on patterns and actions
- Widely used in EDA for log file analysis, report parsing and data extraction
- AWK is more powerful than basic text utilities like grep, sed for column based processing

37.2 AWK Structure / Syntax

```
awk 'pattern { action }' file
```

- pattern → condition to match (optional)
- action → what to do with matching lines
- file → input file (optional)

37.3 Important built-in variables

- $0 → entire line
- $1, $2, ... → fields (columns)
- NF → number of fields
- NR → current record (line) number
- FNR → line number in current file
- FS → field separator (default space/tab)
- OFS → output field separator (default space)

37.4 Example Input File (data.txt)

```
1  VLSI     85  A
2  EDA      92  A+
3  Linux    78  B
4  Scripts  90  A
```

Fields:
$1 = ID
$2 = Subject
$3 = Marks
$4 = Grade

37.5 Common AWK commands (with examples)

No.	AWK Command	Description	Example
1	{print}	Print entire line	awk '{print}' file
2	{print $1}	Print 1st field	awk '{print $1}' file
3	{print $1,$3}	Print multiple fields	awk '{print $1,$3}' file
4	NF	Number of fields	awk '{print NF}' file
5	NR	Line number	awk '{print NR,$0}' file
6	FNR	File line number	awk '{print FNR,$0}' file
7	/pattern/	Match pattern	awk '/VLSI/ {print}' file
8	!/pattern/	Not match pattern	awk '!/Linux/ {print}' file
9	BEGIN{ }	Action before input	awk 'BEGIN{print "Start"}' file
10	END{ }	Action after input	awk 'END{print "Done"}' file
11	-F ":"	Set field separator	awk -F: '{print $2}' file
12	-v var=val	Define variable	awk -v t=90 '$3>t {print $1}' file
13	{if(cond) print}	Conditional processing	awk '{if($3>80) print $1}' file
14	{sum+=$3}	Arithmetic operation	awk '{sum+=$3} END{print sum}' file
15	{print tolower($0)}	Convert to lower case	awk '{print tolower($0)}' file
16	{print toupper($0)}	Convert to upper case	awk '{print toupper($0)}' file
17	{print length($0)}	Length of line	awk '{print length($0)}' file
18	{printf "fmt",$1}	Formatted output	awk '{printf "%d-%s\n", $1,$2}' file
19	{OFS=","}	Output field separater	awk 'BEGIN{OFS=","} {print $1,$3}' file
20	{count++}	Count matching lines	awk '/A/ {count++} END{print count}'
21	{min=$3}	Find minimum	awk 'NR==1{min=$3} $3<min{min=$3} END{print min}' file
22	{max=$3}	Find maximum	awk 'NR==1{max=$3} $3>max{max=$3} END{print max}' file
23	{sum+=$3; c++}	Average	awk '{sum+=$3; c++} END{print sum/c}' file
24	{print $0 > "out"}	Write to another file	awk '$3>85 {print > "top.txt"}' file
25	{getline}	Read next line	awk '{print; getline; print}' file

37.6 Practical Examples

```
awk '{print $2}' data.txt                    # Print subject column
awk '$3>=90 {print $1,$2}' data.txt          # Marks >= 90
awk -F: '{print $1}' /etc/passwd             # Use : separator
awk '{sum+=$3} END{print sum}' data.txt      # Sum of marks
awk 'BEGIN{print "ID Sub Mark Grade"}
     {print $1,$2,$3,$4}' data.txt           # Add header
```

37.7 Practical Observation (EDA)

- AWK is widely used to parse tool reports (timing, area, DRC)
- Helps extract required columns from large log files
- Useful for generating custom summaries and CSV reports

Key Point

- ✓ AWK processes text line by line and field by field
- ✓ Use $1, $2 ... to access columns
- ✓ BEGIN and END are very useful
- ✓ Powerful for log file analysis in EDA flows

VIVA / Interview Point

Q. What is AWK used for ?
→ Text processing and pattern matching.

Q. What does $0, $1 represent ?
→ $0 = entire line, $1 = first field.

Q. How to find number of fields in a line ?
→ awk '{print NF}' file

38. SED AND AWK

1. Overview

- Both sed and awk are powerful text processing tools in Linux.
- sed → stream editor (works line by line, mainly for editing / substituting text).
- awk → pattern scanning and processing language (more powerful, works on fields/columns).
- Often used together in EDA for log file processing, report parsing, data extraction and automation.

2. Comparison : SED vs AWK

Feature	SED	AWK
Type	Stream editor	Pattern scanning language
Processing unit	Line by line	Record (line) & Fields
Main use	Search, replace, edit text	Pattern match, field processing, calculations
Syntax	sed 'command' file	awk 'pattern {action}' file
Field handling	Not easy	Built-in ($1, $2, ...)
Variables	Limited	Yes (variables, arrays)
Best for	Simple text edits	Reports / structured data
Example	sed 's/a/b/g' file	awk '{print $2}' file

3. SED Basics (Quick Recap)

- Syntax : sed [options] 'command' file
- Common use : search and substitute, delete, insert, modify lines.
- Example : sed -i 's/ERROR/WARNING/g' run.log
  (replace in file)

4. AWK Basics

- Syntax : awk 'pattern { action }' file
- Each line is a record and split into fields (columns) based on FS (default : space/tab).
- Built-in variables : $0, $1, $2, ..., NF, NR, FNR, FS, OFS
- Example : awk '{print $1,$3}' data.txt

5. When to use ?

- Use sed for simple search & replace or small edits.
- Use awk when you need to work with columns, perform calculations or generate reports.
- In EDA flows, both are used in scripts to parse log files, timing reports, area reports, etc.

Key Point

- ✓ sed → edit text | awk → process fields
- ✓ awk is more powerful for structured data
- ✓ Combine with pipes for complex processing
- ✓ Very useful in EDA log/report analysis.

6. Common SED and AWK commands with examples

No.	Tool	Command / Syntax	Description	Example
1	sed	s/old/new/	Substitute first match	sed 's/foo/bar/' file
2	sed	s/old/new/g	Substitute global	sed 's/foo/bar/g' file
3	sed	s/old/new/2	Replace 2nd occurrence	sed 's/a/b/2' file
4	sed	/pattern/p	Print matching lines	sed '/ERROR/p' file
5	sed	/pattern/d	Delete matching lines	sed '/DEBUG/d' file
6	sed	n	Read next line	sed 'n' file
7	sed	p	Print current line	sed -n 'p' file
8	sed	i text	Insert text above line	sed '1i Header' file
9	sed	a text	Append text below line	sed '1a End' file
10	sed	c text	Change line with text	sed '2c NewLine' file
11	sed	-i	Edit file in place	sed -i 's/a/b/g' file
12	sed	-n	Suppress default print	sed -n '/ERROR/p' file
13	sed	=	Print line numbers	sed '=' file
14	sed	y/abc/xyz/	Translate characters	sed 'y/abc/xyz/' file
15	sed	s/^/prefix/	Add prefix to line	sed 's/^/>>/' file
16	awk	{print $1}	Print first field	awk '{print $1}' file
17	awk	{print $1,$3}	Print multiple fields	awk '{print $1,$3}' file
18	awk	NF	Number of fields	awk '{print NF}' file
19	awk	NR	Line number	awk '{print NR,$0}' file
20	awk	/pattern/	Pattern matching	awk '/WNS/{print $0}' file

7. Practical Examples

- Replace ERROR with WARNING (in file)
  sed -i 's/ERROR/WARNING/g' run.log
- Extract 2nd column using AWK
  awk '{print $2}' report.rpt
- Print lines containing 'viol' using AWK
  awk '/viol/ {print $0}' timing.rpt
- Add line numbers using sed
  sed '=' file.txt
- Get total area from report (3rd column)
  awk '/Total/ {print $3}' area.rpt

EDA Example

log / rpt file
↓
sed / awk
↓
extract info
↓
generate summary

ViVA / Interview Points

Q. What is the main difference between sed and awk ?
→ sed is a stream editor (line based), awk is a pattern scanning language (field based).

Q. Which tool is better for extracting columns from a report ?
→ awk is more suitable because it provides field variables like $1, $2, ...

39. CHANGE DMA TO CMA (USING AWK)

1. Problem Statement

- In many EDA log / report files, the term "dma" needs to be changed to "cma".
- We can use AWK for pattern matching and field level updates.
- This can be done selectively (specific field) or globally (all fields) with various options.

2. Sample Input File (design.rpt)

```
1   read_dma      OK     10ns
2   write_dma     FAIL   20ns
3   total_dma     15     100ns
4   dma_enable    YES    -
5   no_change     OK     5ns
```

3. Basic Replacement (All occurrences)

```
awk '{gsub("dma","cma"); print}' design.rpt
```

- gsub() → global substitution in the whole line.
- Replaces every occurrence of "dma" with "cma".

4. Output

```
1   read_cma      OK     10ns
2   write_cma     FAIL   20ns
3   total_cma     15     100ns
4   cma_enable    YES    -
5   no_change     OK     5ns
```

5. Replace only in specific field

```
awk '{$1="cma"}{print}' design.rpt  # change 1st field
```

6. Using match condition

```
awk '{ if($0 ~/dma/) {gsub("dma","cma")} print }' design.rpt
```

- Only lines containing dma will be modified.

7. Case insensitif replacement

```
awk '{gsub(/dma/i, "cma"); print }' design.rpt
```

- Replaces DMA, Dma, dma etc.

8. Replace and print to new file

```
awk '{gsub("dma","cma"); print }' design.rpt > design_cma.rpt
```

- Original file is unchanged. Output is written to new file.

9. Useful AWK Commands for this task

No.	Command	Purpose	Example
1	gsub()	Global replace	awk '{gsub("dma","cma");print}' file
2	sub()	Replace first	awk '{sub("dma","cma");print}' file
3	$1="cma"	Change 1st field	awk '{$1="cma";print}' file
4	$2="cma"	Change 2nd field	awk '{$2="cma";print}' file
5	if(/dma/)	Conditional	awk '/dma/{gsub("dma","cma")}1' file
6	/dma/	Pattern match	awk '/dma/ {print}' file
7	!/dma/	Inverse match	awk '!/dma/{print}' file
8	toupper()	Convert to upper	awk '{print toupper($0)}' file
9	tolower()	Convert to lower	awk '{print tolower($0)}' file
10	NF	No. of fields	awk '{print NF}' file
11	NR	Line number	awk '{print NR,$0}' file
12	FNR	File line number	awk '{print FNR,$0}' file
13	print	Print fields	awk '{print $1,$3}' file
14	printf	Formatted print	awk '{printf "%s %s\n",$1,$2}'
15	split()	Split string	awk '{split($1,a,"_"); print a[1]}'
16	index()	Find position	awk '{print index($1,"dma")}'
17	length()	String length	awk '{print length($1)}'
18	substr()	Substring	awk '{print substr($1,1,3)}'
19	$0	Whole line	awk '{gsub("dma","cma");print $0}'
20	BEGIN	Initialization	awk 'BEGIN{print "Start"} {gsub("dma","cma");print}'
21	END	Final action	awk 'END{print "Done"}'
22	FS	Input separator	awk 'BEGIN{FS=":"} {print $1}'
23	OFS	Output separator	awk 'BEGIN{OFS="-"} {$1="cma";print}'
24	RS	Record separator	awk 'BEGIN{RS=""} {gsub("dma","cma");print}'
25	next	Skip line	awk '/skip/{next} {gsub("dma","cma");print}'

10. EDA Use Cases

- Change tool specific keywords in log files (e.g. dma → cma).
- Modify report fields before feeding to other scripts.
- Clean up names, paths or tags in large files.
- Works well with pipes: cat file | awk '{gsub("dma","cma");print}' > new.rpt

11. Practical Observation

- AWK is very useful for field-based replacements in structured EDA logs.
- For simple line based replace, sed can also be used, but AWK gives more flexibility (field operations, conditions).
- Always verify changes using diff or less.

Key Point

- ✓ Use gsub() → replace all occurrences.
- ✓ Use sub() → replace first occurrence.
- ✓ AWK works on fields, very useful for EDA reports.
- ✓ Combine with conditions for selective changes.
- ✓ Verify output after modification.

Viva / Interview Points

Q1. How to replace dma with cma using AWK ?
→ awk '{gsub("dma","cma");print}' file

Q2. What is the difference between sub() and gsub() ?
→ sub(): first match
gsub(): all matches

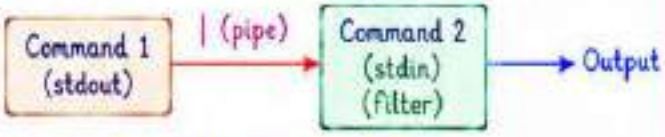
40. PIPES , FILTERS , SORT

1. Introduction

- Pipes (|) connect the output of one command to the input of another command.
- Filters are commands that read from standard input, process the data and produce output (e.g., grep, awk, sed, sort, uniq, head, tail, wc)
- Useful to process large log / report files in EDA flows.

2. Pipe (|)

- Syntax : command1 | command2
- command1 output → passed to command2 input.
- Can chain multiple commands



3. Common Filters (examples)

Command	Purpose
grep	search for pattern
sort	sort lines
uniq	remove duplicate lines
head	first N lines
tail	last N lines
wc	count lines/words/characters
cut	extract columns
awk	field based processing
sed	stream editor (text replacement)

4. Pipes Examples

```
1. ls -ltr | less          # page through long output
2. ps -ef | grep vim       # search process
3. cat run.log | grep "ERROR" | wc -l   # count errors
4. ls *.rpt | grep -i timing | sort     # chain commands
5. bsub -n 10 | grep JOBID  # filter output
```

5. Useful Filter Examples

```
1. cat file | head -5      # first 5 lines
2. cat file | tail -5      # last 5 lines
3. cat file | uniq         # remove duplicates
4. cat file | sort         # sort lines
5. cat file | wc -l        # count lines
```

6. sort Command

- sort arranges lines of text in ascending or descending order.
- Syntax : sort [options] file
- If no file is given, it reads from standard input (use with |).

Common sort options

Option	Purpose	Example
-n	numeric sort	sort -n file
-r	reverse order	sort -r file
-k N	sort by Nth field	sort -k 2 file
-t c	field separator	sort -t, -k2 file
-u	unique (remove duplicates)	sort -u file
-f	ignore case	sort -f file
-g	general numeric	sort -g file
-h	human readable (1K 10K)	sort -h file
-M	month name sort	sort -M file
-o file	write output to file	sort file -o out.txt

sort Examples

```
1. sort names.txt              # alphabetical
2. sort -n numbers.txt         # numeric
3. sort -r numbers.txt         # descending
4. sort -k 2 report.txt        # sort by 2nd column
5. sort -t, -k 3 data.csv      # CSV, sort 3rd field
6. sort -u file                # remove duplicates
7. sort -h sizes.txt           # 1K 10K 2M ...
8. sort file -o sorted.txt     # write to new file
```

7. Practical EDA Examples

- Find top 10 worst paths from timing report:
  `grep "WNS" timing.rpt | sort -n | head`
- Count unique cells from a log file:
  `grep "Inst:" design.log | awk '{print $2}' | sort | uniq | wc -l`
- List largest files:
  `du -h | sort -h | tail`
- Check errors in multiple logs:
  `grep -i "error" *.log | sort | uniq`

Key Points

- ✓ Pipe (|) connects commands
- ✓ Filters process text (grep, sort, etc)
- ✓ sort has many useful options
- ✓ Combining commands saves time in EDA log analysis
- ✓ Useful for large files and reports

Viva / Interview Points

Q1: What does | do?
→ Sends stdout of one command to stdin of another command.

Q2: How to sort a file numerically in reverse order ?
→ sort -nr file

41. FUNCTIONS IN GVIM , GVIM VISUAL MODE , INSERT MODE

1. What is GVIM ?

- GVIM (Graphical Vim) is the GUI version of vi/vim editor.
- Provides same powerful features as vim with a graphical interface (menus, toolbar, mouse support).
- Widely used in EDA for editing scripts, configuration files, logs and reports.

2. Modes in GVIM



- navigation
- commands
- editing ops

- enter text
- edit text
- like any editor

- select text
- perform ops
- cut / copy / delete

3. Insert Mode

- Used for entering and editing text.
- Press i, a, o etc. to enter Insert mode.
- Press Esc to return to Normal mode.

Key	Action	Example / Result
I	insert before cursor	IHello
I	insert at line start	I# comment
a	insert after cursor	a text
A	insert at line end	A ;
o	open new line below	o <text>
O	open new line above	O <text>
Esc	return to normal mode	Esc

4. Visual Mode

- Used to select text and perform operations on it.
- Types: character (v), line (V), block (Ctrl+v).

Key	Action	Example / Result
v	character wise select	v + move
V	line wise select	V + move
Ctrl+v	block/column select	Ctrl+v + move
y	copy selected text	vy
d	delete selected text	vd
>	indent right	v>
<	indent left	v<

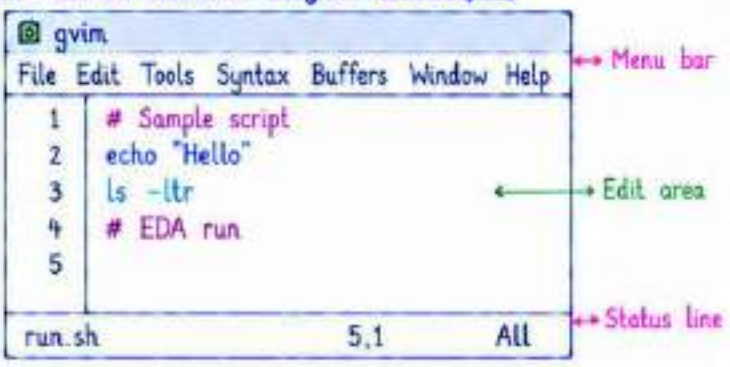
5. Important GVIM Functions / Commands

Command	Purpose	Example
:w	save file	:w
:q	quit	:q
:wq	save and quit	:wq
:q!	quit without saving	:q!
:e file	open file	:e run.sh
:new	new window	:new
:split file	horizontal split	:split log.txt
:vsplit file	vertical split	:vsplit design.v
:tabnew	new tab	:tabnew
:help keyword	open help	:help visual
:set number	show line numbers	:set number
:set syntax=on	enable syntax	:set syntax=on
:colorscheme	change color theme	:colorscheme desert

6. Examples

1. Open a file in gvim
   $ gvim run.sh # open file with GUI
2. Edit text using insert mode
   Press i → type text → Esc # enter and exit
3. Select and copy using visual mode
   v move → y # copy selected text
4. Delete a line using visual mode
   V + move → d # delete selected lines
5. Split windows
   :split file1 :vsplit file2 # multiple files
6. Enable line numbers and syntax
   :set number :set syntax=on

7. GVIM Window Layout (Example)



Key Points

- ✓ GVIM provides a user-friendly interface for vi.
- ✓ Insert mode is for entering text (i, a, o).
- ✓ Visual mode is for selecting text (v, V, Ctrl+v).
- ✓ GVIM supports multiple windows, tabs and customization.
- ✓ Very useful for editing scripts, logs and configuration files in EDA flows.

Viva / Interview Points

Q1: How to enter insert mode in gvim ?
→ Press i, a or o.

Q2: What is the difference between v and V ?
→ v = character wise selection
→ V = line wise selection

42. GVIM ADVANCED CONCEPTS - . , Ctrl + P , Ctrl + N

1. Recap: What is GVIM ?

- GVIM (Graphical Vim) is aon GUI version of vi/vim editor.
- Provides syntax highlighting, multiple windows, mouse support and powerful editing features.
- Same editing commands as vim with an enhanced visual interface, useful for EDA scripts, logs and config files.

2. GVIM Modes (Quick View)

Normal Mode ⟷ Insert Mode ⟷ Visual Mode

Normal Mode	Insert Mode	Visual Mode
• navigation	• enter text	• select text
• commands	• edit text	• operate on selection
• use . , Ctrl+p/n	• press Esc to exit	• character (v)
• copy / paste	• text input	• line (V)
• save / quit	• auto indent	• block (Ctrl+V)

3. The . (dot) Command

- Repeats the last editing command.
- Very useful for doing the same change multiple times.
- Works for insert, delete, change, yank, etc.

Action	Example / Result
Repeat delete	dd (delete line) . (deletes next line)
Repeat change	cwhello <Esc> (change word) . (repeat on next word)
Repeat insert	A ; <Esc> (insert at end) . (repeat insert on next line)

4. Ctrl + P and Ctrl + N

- Used for navigating previously entered commands / text (completion).
- In GVIM, useful in:
  - command line mode (:) → history search
  - insert mode → keyword / file name completion (similar to shell)

Command	Purpose	Example
Ctrl + P	previous command / suggestion	type : then Ctrl + P
Ctrl + N	next command / suggestion	type : then Ctrl + N

5. Examples - . (dot command)

1. Delete multiple lines

```
1  line1
2  line2
3  line3
4  line4
```

Command: dd (deletes line1)
Press . (deletes line2)

Result:

```
1  line3
2  line4
```

2. Repeat a change

```
1  net1
2  net2
3  net3
```

Command: cwdata <Esc> (changes net1 → data)
Press . (changes net2 → data)

Result:

```
1  data
2  data
3  net3
```

3. Repeat insert

```
1  port1
2  port2
3  port3
```

Command: A ; <Esc> (add ; at end)
Press . (adds ; to next line)

Result:

```
1  port1;
2  port2;
3  port3
```

6. Examples - Ctrl + P / Ctrl + N

1. In command line mode (history)

```
:open run.sh
:open design.v
:set number
:wq
```

Press : then Ctrl + P
→ shows previous command (e.g. :set number)
Press Ctrl + N
→ next command in history

2. In insert mode (file name / keyword completion)

In insert mode, type: #include "

- Press Ctrl + P / Ctrl + N
  → cycles through file name suggestions

7. Practical GVIM Tips (EDA Usage)

- Use . to repeat common edits in large netlists, scripts, logs.
- Use Ctrl + P / Ctrl + N to quickly reuse previous commands.
- Use visual mode for editing multiple lines, e.g. commenting.
- GVIM is useful for editing configuration files like .synrc, .lib, .tcl etc.

Key Points

- ✓ . repeats the last command.
- ✓ Ctrl + P → previous command / suggestion.
- ✓ Ctrl + N → next command / suggestion.
- ✓ Insert mode for text input, Visual mode for selection.
- ✓ Very useful in EDA for quick and repeated edits.

Viva / Interview Points

Q1: What does . (dot) do in vim/gvim ?
→ It repeats the last editing command.

Q2: What is the use of Ctrl + P and Ctrl + N ?
→ They are used for browse previous and next commands / completion suggestions.

43. GVIM CODE INDENTATION - , .cshrc file , Guidelines

1. What is Code Indentation ?

- Indentation means adding proper spaces or TABs at the beginning of a line to improve readability of code and scripts.
- In GVIM, indentation can be done manually or automatically using editor settings.
- Proper indentation is very useful for shell scripts like .cshrc and for reading complex code.

2. Indentation in GVIM

- TAB → insert one level of indentation (based on settings)
- Shift + TAB → remove one level of indentation
- Use auto-indent features for consistent formatting.
- Useful while writing shell scripts (.cshrc, .sh) and EDA flow scripts.
- Indentation does not change functionality, but improves maintainability.

3. Useful GVIM Settings for Indentation

Command	Purpose	Example
:set tabstop=4	TAB width	:set tabstop=4
:set shiftwidth=4	indent size	:set shiftwidth=4
:set expandtab	use spaces instead of TAB	:set expandtab
:set autoindent	auto indent next line	:set autoindent
:set smartindent	smart indent for C-like code	:set smartindent
:set paste	disable auto indent (paste)	:set paste

4. .cshrc File

- .cshrc is a C Shell configuration file executed at shell startup.
- It is used to set environment variables, aliases, paths, prompts etc.
- Located in home directory: ~/.cshrc
- Edit using: gvim .cshrc

Example .cshrc snippets

```
# Set project path
setenv PROJECT_HOME /home/user/project
set path = ( $PROJECT_HOME/bin $path )

# Alias for convenience
alias ll 'ls -ltr'
alias v 'gvim'
```

5. Example: Indentation in a C program (GVIM)

```
int main()
{
printf("Hello\n");
if(1)
{
printf("Inside if\n");
}
return 0;
}
```

Indent →

```
int main()
{
    printf("Hello\n");
    if (1)
    {
        printf("Inside if\n");
    }
    return 0;
}
```

6. Working with .cshrc in GVIM

$ gvim .cshrc

- Make changes with proper indentation.
- Save and exit : :wq
- Apply changes : source .cshrc
- Verify : echo $PATH

Before	After (formatted)
setenv PATH \ /tools/bin:\ /eda/bin:$PATH	setenv PATH \ /tools/bin:\ /eda/bin:\ $PATH

7. Guidelines for Writing .cshrc

1. Use consistent indentation (2 or 4 spaces).
2. Add comments using # to explain each setting.
3. Keep related settings together (PATH, aliases, environment).
4. Avoid unnecessary blank lines.
5. Use meaningful variable names.
6. Test changes using source .cshrc.
7. Avoid hardcoding user specific paths (use $HOME).

8. Practical Example - Indentation in .cshrc

```
# Environment setup
setenv VLSI_HOME /eda/tools
set path = ( $VLSI_HOME/bin \
             $VLSI_HOME/scripts \
             $path )

# Aliases
alias run 'source run.sh'
alias rehash 'rehash'
```

Key Points

- ✓ Indentation improves readability.
- ✓ Use GVIM settings.
- ✓ Keep .cshrc organized.
- ✓ Source the file after modifications.
- ✓ Very useful in EDA environments.

9. Viva / Interview Points

Q1: What is .cshrc ?
→ It is a C shell configuration file executed at login.

Q2: Which GVIM settings control indentation ?
→ tabstop, shiftwidth, expandtab, autoindent, smartindent.

44. CONNECTING TO SERVER , PROCESS MANAGEMENT - Bsub , KILLING USING PID , LSF , SLEEP

1. Connecting to a Server

- Servers are used for running EDA tools, large simulations and shared projects.
- We usually connect to remote servers using SSH (Secure Shell).
- Provides secure, encrypted communication.

1.1 SSH Login Syntax

```
ssh [options] user@hostname
```

Example :

```
$ ssh tharun@eda-server01      → Login to server
```

Useful options :

Option	Purpose	Example
-p <port>	Specify port number	ssh -p 2222 user@host
-X	Enable X11 forwarding (for GUI tools)	ssh -X user@host
-Y	Trusted X11 forwarding	ssh -Y user@host
-v	Verbose mode (debug)	ssh -v user@host
-l <user>	Specify username	ssh -l user host

2. Process Management in Linux

- Each running program is a process with a unique PID (Process ID).
- We can view, control and terminate processes.

2.1 Common Process Commands

Command	Purpose	Example
ps	Display running processes	ps -ef | grep tool
top	Real-time process monitoring	top
kill	Terminate a process	kill 12345
kill -9	Force kill (if not terminating)	kill -9 12345
jobs	Show background jobs	jobs
fg	Bring job to foreground	fg %1
bg	Continue job in background	bg %1

Note :
- Use kill -9 only if normal kill doesn't work.
- Always verify PID before killing : ps -ef | grep <name>

3. LSF and bsub (Job Submission)

- Most EDA environments use LSF (Load Sharing Facility) for batch job management on compute servers.
- bsub is used to submit jobs to LSF queue.

3.1 bsub Syntax

```
bsub [options] <command | script>
```

Example :

```
$ bsub run.sh          → submit script to default queue
```

Common options :

Option	Purpose	Example
-q <queue>	Specify queue	bsub -q normal run.sh
-n <num>	Number of CPU cores	bsub -n 8 run.sh
-R <rusage>	Resource requirement	bsub -R "rusage[mem=4G]" run.sh
-o <file>	Output log file	bsub -o run.log run.sh
-e <file>	Error log file	bsub -e run.err run.sh
-J <name>	Job name	bsub -J synthesis run.sh

3.2 Check Job Status

```
$ bjobs          → show submitted jobs
$ bjobs -u tharun   → show your jobs
$ bjob -l <jobid>   → detailed info
```

4. Killing Processes using PID

- Find the PID using ps or top.
- Use kill <PID> to terminate gracefully.
- If not terminated, use kill -9 <PID>.

Example :

```
$ ps -ef | grep tool    → find PID of tool
$ kill 12345            → terminate process
$ kill -9 12345         → force kill
```

Caution :
- Make sure you are killing correct PID.
- Force kill (-9) may cause data loss.

5. sleep Command

- sleep pauses execution for specified time (in seconds).
- Useful in scripts, waiting for jobs, timing control etc.
- Syntax : sleep <seconds>

Examples :

```
$ sleep 10      → wait for 10 seconds
$ sleep 60      → wait for 1 minute
$ sleep 1h      → wait for 1 hour (if supported)
```

6. Practical Use in EDA Flow

- Login to server → setup environment → submit jobs (bsub)
- Monitor jobs using bjobs / top
- Kill stuck jobs using kill.
- Use sleep in scripts to wait for file generation or tool completion.

Key Points

- ✓ SSH is used to connect to remote servers.
- ✓ bsub submits jobs to LSF queue.
- ✓ Use ps / top to find PID and kill to terminate.
- ✓ sleep is useful in automation scripts.
- ✓ Process management is critical in EDA environments.

Viva / Interview Points

Q1: What is bsub ?
→ Command to submit jobs to LSF scheduler.

Q2: How to kill a process ?
→ kill <PID> or kill -9 <PID>

Q3: Why use sleep ?
→ To wait/delay execution in scripts.

45. RUNNING MULTIPLE COMMANDS AS A SCRIPT

1. Why use a shell script ?

- Run multiple commands sequentially.
- Automate repetitive tasks (EDA runs, log collection).
- Ensure consistent environment setup.
- Save time and reduce manual errors.
- Useful for long flows (synthesis, P&R, STA, etc).

2. Basic shell script structure

```
#!/bin/bash                → shebang (interpreter)
# My first script
echo "Starting run ..."
pwd
ls -ltr
echo "Run completed"
```

3. Creating and running a script

① Create a file : vi run.sh
② Add multiple commands
③ Give execute permission : chmod +x run.sh
④ Run the script : ./run.sh

4. Example : EDA flow script

```
#!/bin/bash
# setup environment
source /tools/eda/setup.csh
cd ~/project/chip1
echo "Running synthesis ..."
bsub -q normal -o syn.log synth.tcl
sleep 5
echo "Checking log ..."
grep "ERROR" syn.log
echo "Done"
```

Multiple commands executed sequentially

5. Types of shell scripts

Shell	Extension	Shebang line	Example
Bash	.sh	#!/bin/bash	run.sh
C Shell	.csh	#!/bin/csh	run.csh
TC Shell	.tcsh	#!/bin/tcsh	run.tcsh
Korn Shell	.ksh	#!/bin/ksh	run.ksh

6. Useful commands inside scripts

- echo → print messages
- cd → change directory
- source → load environment
- bsub → submit job to LSF
- sleep → wait for specified time
- grep → check logs

7. Environment setup in script

```
#!/bin/bash
# set EDA tools
export PATH=$PATH:/tools/eda/bin
export LM_LICENSE_FILE=27000@server
cd $WORK_DIR
```

8. Running script in background

- Add & to run in background.
- Useful for long EDA jobs.
- Example :

```
$ ./run.sh &      → runs in background
$ jobs            → check background jobs
```

9. Monitor and control script execution

- Check running processes using ps or top.
- Terminate using kill <PID> if required.
- Check job status from log files.

10. Practical example : script for multiple EDA steps

```
#!/bin/bash
# Full flow
echo "Starting full flow"
source /tools/eda/setup.csh
bsub -q normal synth.tcl
bsub -q normal place.tcl
bsub -q normal route.tcl
sleep 10
echo "Check logs in logs/ directory"
```

Submit multiple jobs as a script

11. Guidelines / Best Practices

- ✓ Use meaningful script name (e.g., run_synth.sh).
- ✓ Add comments for each step.
- ✓ Set required environment variables inside script.
- ✓ Use echo to print progress messages.
- ✓ Check return status ($?) if needed.
- ✓ Use log files to capture output.
- ✓ Use sleep for waiting between steps (job completion, file gen).
- ✓ Make script executable (chmod +x script.sh).
- ✓ Test script with a small run before full flow.

Key Point

Shell script = set of Linux commands executed sequentially to automate tasks like environment setup, job submission, log checking and report generation.

Viva / Interview Points

Q1: Why use a script ?
→ Automate multiple commands.

Q2: How to run in background ?
→ ./run.sh &

Q3: What does sleep do ?
→ Waits for specified time.

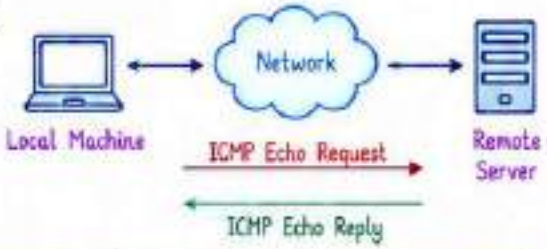
46. COMMUNICATION COMMAND - PING , FTP AND ITS COMMANDS , TELNET

1. PING COMMAND

- ping checks the reachability of a remote host using ICMP echo request/reply.
- Useful to test network connectivity, latency and packet loss.
- Works with hostname or IP address.

Syntax :

```
ping [options] <hostname | IP>
```



Common options

Option	Meaning
-c N	send N packets
-i sec	interval between packets
-s size	packet size in bytes
-W sec	timeout to wait for reply

Example 1 : Ping a server

```
$ ping chipserver
PING chipserver (192.168.10.25) 56(84) bytes of data.
64 bytes from 192.168.10.25: icmp_seq=1 ttl=64 time=0.8 ms
64 bytes from 192.168.10.25: icmp_seq=2 ttl=64 time=0.7 ms
64 bytes from 192.168.10.25: icmp_seq=3 ttl=64 time=0.8 ms
^C
--- chipserver ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
rtt min/avg/max = 0.7/0.8/0.8 ms
```

Example 2 : Limited count

```
$ ping -c 5 192.168.1.10
→ sends 5 packets and exits
```

Use cases (EDA) :

- Check if license server is reachable.
- Debug network issues.
- Measure latency between local and compute server.

Key Point

- Ping checks connectivity (ICMP), not service availability.
- If ping works, server might be up, but specific services (SSH, FTP, etc.) may still be down.

2. FTP (FILE TRANSFER PROTOCOL)

- FTP is used tho transfer files between local machine and remote server.
- Works on client-server model (default port 21).
- Useful for uploading/downloading files (logs, scripts, reports, etc.).
- Interactive commands are used inside ftp shell.

Basic connection :

```
$ ftp <hostname | IP>
```

Common FTP commands

Command	Purpose
open <host>	connect to server
user <name>	enter username
pass <pwd>	enter password
ls	list files/directories
cd <dir>	change remote directory
pwd	print current remote dir
get <file>	download file (remote → local)
put <file>	upload file (local → remote)
mget <files>	download multiple files
mput <files>	upload multiple files
bye / quit	exit ftp

Example session :

```
$ ftp server1
Connected to server1.
220 Welcome to FTP server
Name (server1:user): eceuser
331 Password required.
Password: *******
230 Login successful.
ftp> pwd
257 "/home/eceuser"
ftp> ls
200 PORT command successful.
150 Here comes the directory listing.
setup.sh
run.log
reports/
226 Directory send OK.
ftp> get run.log    # download
ftp> put script.sh  # upload
ftp> cd reports
ftp> ls
ftp> bye
221 Goodbye.
```

FTP in EDA flow :

- Transfer large log files, reports, netlists.
- Get/put scripts between local and remote workstations.
- Useful when GUI file transfer is not available.

Note :

- Use sftp/scp for secure file transfer (instead of ftp).
- FTP credentials may not be allowed on some servers.

3. TELNET COMMAND

- telnet is used to connect to a remote server using TCP.
- Useful to check if a specific port/service is open.
- Interactive terminal session to remote host.
- Default port is 23.
- (Now mostly replaced by ssh for security).

Syntax :

```
telnet <hostname | IP> [port]
```

Example 1 : Connect to server

```
$ telnet server1
Trying 192.168.1.20 ...
Connected to server1.
Escape character is '^]'.
login: eceuser
Password: *******
Last login: Tue Nov 12 10:22:01
$
# now you are on remote server
```

Example 2 : Check a port

```
$ telnet server1 27000
Trying 192.168.1.20 ...
Connected to server1.
# If connection fails:
# telnet: Unable to connect
```

Use cases :

- Check if license server port is open (e.g. 27000).
- Debug network connectivity.
- Access simple services when ssh is not allowed.

Viva / Interview Points

Q1: What does ping do?
→ Checks host reachability using ICMP.

Q2: What are FTP main commands?
→ ls, cd, get, put, bye.

Q3: What is default port for telnet?
→ 23.

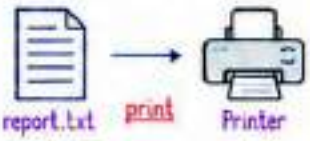
Practical Observation

In EDA environments, ping, ftp and telnet are useful for checking server reachability, transferring tool files/logs and verifying service ports.

47. BASIC UTILITIES - PRINT , LT , VI EDITOR , GVIM

1. PRINT COMMAND

- print is used to send a file or text to a printer.
- Converts text file to printer readable format.
- Useful for printing documents, reports, logs etc.
- Depending on system, it may use lpr or other backends.



Syntax :

```
print [options] <file(s)>
```

Common options

Option	Meaning
-d <printer>	specify printer
-n <copies>	number of copies
-h	no banner page
-o <option>	set printer options

Example 1 : Print a file

```
$ print report.txt
        # send file to default printer
```

Example 2 : Multiple files

```
$ print file1.txt file2.txt
        # print multiple files
```

Note :
print command may not be available on all Linux systems. If not, use lpr instead.

2. lt COMMAND

- lt (long listing with pagination) is similar to ls -l but with page-by-page display.
- Useful when directory has many files.
- Allows scrolling (space → next page, q → quit).
- Often available as an alias or shell function.
- Provides detailed file information.

Syntax :

```
lt [directory]
```

Example : List files using lt

```
$ lt                # list files in current dir
$ lt /home/user     # list files in given dir
```

Sample Output (like ls -l)

```
-rw-r--r-- 1 user user  1234 Jan 10 10:20 report.txt
drwxr-xr-x 2 user user  4096 Jan 11 09:15 scripts
-rwxr-xr-x 1 user user   856 Jan 12 14:30 run.sh
-rw-r--r-- 1 user user 10240 Jan 13 11:05 timing.rpt
```

Navigation keys (in lt)

space → next page
b → previous page
q → quit

3. VI EDITOR

- vi is a powerful text editor available on every Unix/Linux system.
- Used to create and edit text files, scripts, configuration files etc.
- Works in different modes (Normal, Insert, Command).
- Lightweight and available in terminal.

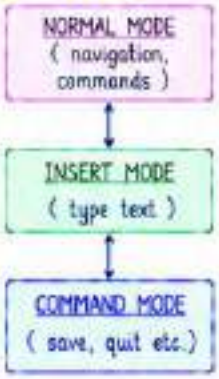
Syntax :

```
vi <filename>
```

Example : Open a file

```
$ vi run.sh      # open file in vi
```

Modes in vi



Some common vi commands

Command	Purpose
i	enter insert mode
Esc	back to normal mode
:w	save file
:q	quit
:wq	save and quit
:q!	quit without saving
/pattern	search forward
?pattern	search backward
dd	delete a line
yy	copy a line
p	paste after cursor

4. GVIM EDITOR

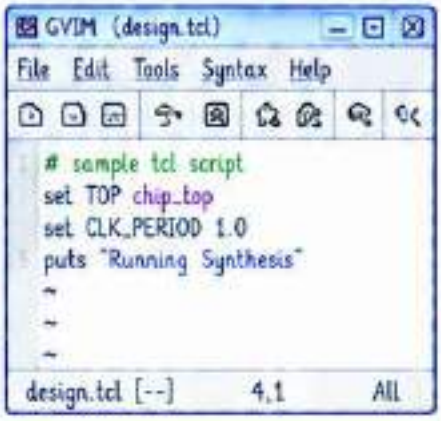
- gvim is the graphical version of vi.
- Provides GUI with menus, toolbar and syntax highlighting.
- Useful for editing large files comfortably.
- Supports mouse operations.

Syntax :

```
gvim <filename>
   or
gvim &    # open gvim in background
```

Example :

```
$ gvim design.tcl  # open file in gvim
$ gvim &           # open gvim (no terminal block)
```



When to use vi vs gvim ?

vi	gvim
Terminal based	Graphical (GUI)
Lightweight	Feature rich
Available on all systems	May not be available everywhere
Good for remote servers	Good for local editing
No mouse (usually)	Supports mouse
Simple	Syntax highlighting, toolbar etc.

Practical Observations (EDA)

- Use vi on remote servers (no GUI).
- Use gvim on local machine for editing large scripts.
- Useful for editing .tcl, .sh, .cshrc, .conf files in EDA flow.
- print is used to take hard copy of reports when needed.
- lt is handy to browse directories with many files on remote servers.

Viva / Interview Points

Q1: What is the use of print command ? → Sends a file to printer.
Q2: What is the difference between vi and gvim ? → vi is terminal based, gvim is GUI.
Q3: In lt, how to go to next page ? → Press space bar.

Key Point

- print → print files to a printer (use lpr if not available)
- lt → long listing with pagination
- vi → powerful terminal text editor
- gvim → graphical text editor with more features

48. CTAG'S (GENERATE TAGS FOR SOURCE CODE)

1. INTRODUCTION TO CTAGS

- ctags (exuberant ctags) is a utility to generate an index (tags file) of symbols found in source code files.
- It scans source files and records the location of definitions such as functions, variables, macros etc.
- Mainly used with editors like vi/vim to quickly jump to the definition of a symbol.
- Very useful in large projects (EDA / VLSI code bases).

2. BASIC SYNTAX

```
ctags [options] file1 file2 ...
```

- ctags → command
- [options] → options (optional)
- file1 file2 ... → source files or directories

3. COMMON OPTIONS

Option	Purpose
-o <file>	specify tags file name
-R	recursive search in sub directories
--languages=<list>	specify languages (e.g. C,C++,Verilog)
--exclude=<pattern>	exclude files / directories
--fields=+n	include line number in tags
--extra=+q	include qualified tags (class::func)
--c-kinds=+p	include function prototypes
--help	show help information

4. EXAMPLES

Example 1 : Create tags for current directory

```
$ ctags *      # creates file 'tags' for all source files in current directory
```

Example 2 : Recursive tags for project

```
$ ctags -R     # scan all subdirectories
```

Example 3 : Specify tags file name

```
$ ctags -o project.tags *.c *.h
                # create tags file with custom name
```

Example 4 : C, C++ and Verilog files

```
$ ctags -R --languages=C,C++,Verilog
                # useful for VLSI projects
```

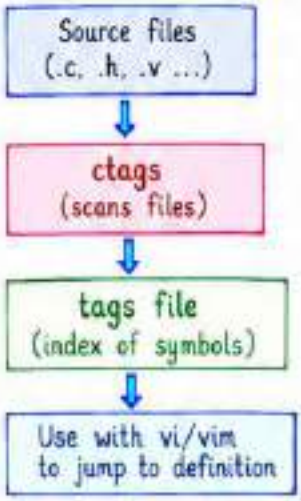
Example 5 : Exclude certain directories

```
$ ctags -R --exclude=build --exclude=.git
                # skip unnecessary directories
```

Example 6 : Include line numbers and prototypes

```
$ ctags -R --fields=+n --c-kinds=+p *.c
                # more information in tags
```

5. HOW CTAGS WORKS ?



tags file contains:

- symbol name
- file name
- line number
- search pattern

Supported Languages (may vary with version):

- C, C++
- Verilog / SystemVerilog
- Tcl, Python, Shell
- Others ...

6. SAMPLE tags FILE FORMAT

```
main        test.c      /^int main()$/;
add         util.c      /^int add(int a, int b)$/;
FIFO        fifo.v      /^module FIFO/;
reset_n     regs.v      /^input reset_n/;
MY_MACRO    defs.h      /^#define MY_MACRO/;
```

(format may vary slightly)

tag / symbol — file name — search pattern (used by vim)

7. USING TAGS WITH VI / VIM

Command	Purpose
:set tags=tags	tell vim to use tags file
Ctrl-]	jump to definition of symbol
Ctrl-t	return to previous location
:tag <symbol>	jump to specified symbol
:tags	show tag stack
:tnext	next tag in stack
:tprev	previous tag in stack

8. PRACTICAL USE IN EDA / VLSI ENVIRONMENT

- Helps to quickly navigate large RTL / verification code.
- Useful for multi-file designs with thousands of modules.
- Supports Verilog/SystemVerilog with exuberant ctags.
- Improves productivity when working in Unix servers.
- Often used along with vim in data-center / remote machines.

Viva / Interview Points

Q1: What is ctags ?
→ A tool to generate an index of symbols in source code files.

Q2: What is stored in tags file ?
→ symbol name, file name, line number and search pattern.

Q3: How to jump to a definition in vim ?
→ Use Ctrl-] after generating tags

Key Takeaways

- ctags → generate tags file
- -R → recursive search
- Use with vi/vim (Ctrl-])
- Very useful for large EDA / VLSI code bases
- Save time in navigation and debug

49. SOME MORE COMMANDS
(su , who am I ?, passwd , groups , users , who)

1. SU (Switch User)

- su is used to switch to another user account.
- Usually used to become root or another user.
- Prompts for the target user's password.
- Options :
  - su username → switch to user
  - su - → login shell (load env)
  - su -c "cmd" → run single command as user

Example :

```
$ su                        # switch to root (enter root password)
# whoami
  root
# exit                      # return to normal user
$ su - vlsiuser             # login as vlsiuser
$ su -c "ls -l /home" user1 # run command
```

2. whoami (Who am I ?)

- Displays the current logged-in user name.
- Useful to confirm which account you are using.

Example :

```
$ whoami
  tharun
```

3. passwd (Change Password)

- Used to change the password of a user.
- If no user is given, it changes the password of current user.
- Requires old password (for normal users).
- Syntax : passwd [username]

Example :

```
$ passwd                    # change current user password
$ passwd vlsiuser           # (as root) change other user pwd
```

4. groups (Display Group Membership)

- Shows the groups of the current user or a specified user..
- A user can be part of multiple groups.
- Syntax : groups [username]

Example :

```
$ groups                    # groups of current user
$ groups tharun             # groups of user tharun
```

5. users (List Logged-in Users)

- Displays the list of user names currently logged in to the system.
- Useful to quickly see active users.
- Syntax : users

Example :

```
$ users                                  # logged-in users
tharun  vlsiuser  pduser
```

6. who (Show Who is Logged In)

- Displays detailed information about users currently logged in.
- Shows user name, terminal, login time, and host.
- Common options :
  - who → default output
  - who -a → all information
  - who -r → run level
  - who -u → idle time
  - who -m → current terminal user
  - who -q → login summary (count)

Example :

```
$ who
  tharun    pts/0        Nov 25  10:12
  vlsiuser  pts/1        Nov 25  11:03   (192.168.1.10)
$ who -a                 # more detailed info
$ who -q                 # login summary
  users=2
```

7. QUICK REFERENCE

Command	Purpose	Example
su	switch user	su - root
whoami	current user name	whoami
passwd	change password	passwd
groups	show group membership	groups vlsiuser
users	list logged in users	users
who	show logged-in details	who -a

KEY POINTS

- Use su with caution (especially for root access).
- whoami helps to confirm current user.
- passwd changes user password.
- groups shows all groups of a user.
- users gives a quick list of logged-in users.
- who provides detailed login information.
- These commands are useful in multi-user EDA / server environments.

$ tar
$ gzip
$ gunzip
$ ls
$ file

LINUX FOR ECE / VLSI & EDA

Module 17

Archives, Compression & Links

~/project
.tar .gz .tgz
ls -ltr
|
.rpt

50. EXTRACTING FILES - .tar , GZIP
(explain these commands along examples too)

1. TAR (Tape ARchive)

- tar is used to archive multiple files/directories into a single file.
- Does not compress by default (just packs).
- Commonly used in Linux for backup, sharing, EDA project data, source code etc.
- Syntax : tar [options] [archive-file] [files/dir]

Common Options for tar

Option	Purpose
-c	create archive
-x	extract archive
-t	list contents
-v	verbose (show files)
-f	specify archive file name
-z	use gzip compression (tar + gzip)
-C dir	extract to specific directory

Examples :

a) Create an archive
```
$ tar -cvf project.tar project/     # archive project/
```
b) List contents
```
$ tar -tvf project.tar      # view files in archive
```
c) Extract files
```
$ tar -xvf project.tar      # extract in current dir
```
d) Extract to specific directory
```
$ tar -xvf project.tar -C /home/user/work
```
e) Create and compress using gzip
```
$ tar -czvf project.tar.gz project/     # tar + gzip
```
f) Extract .tar.gz file
```
$ tar -xzvf project.tar.gz
```



2. GZIP (GNU ZIP)

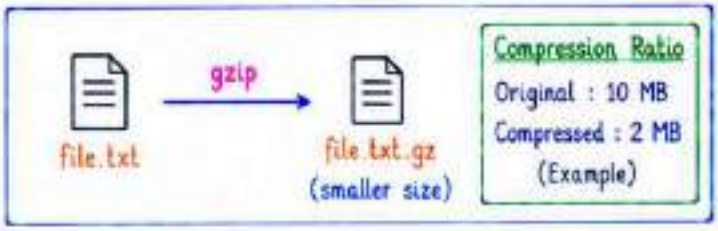
- gzip is used to compress files (reduces file size). Creates a .gz file and removes the original file (by default).
- Commonly used to compress logs, reports, large data files etc.
- Syntax : gzip [options] file

Common Options for gzip

Option	Purpose
-c	compress and write to stdout (keep original)
-d	decompress (same as gunzip)
-k	keep original file
-l	show compressed file information
-r	compress directories recursively
-v	verbose output
-9	maximum compression (1 to 9)

Examples :

a) Compress a file
```
$ gzip file.txt     # creates file.txt.gz (original removed)
```
b) Compress and keep original
```
$ gzip -k file.txt          # keeps file.txt
```
c) Decompress a file
```
$ gzip -d file.txt.gz       # restores file.txt
(or) $ gunzip file.txt.gz   # same as above
```
d) View compression details
```
$ gzip -l file.txt.gz
```
e) Compress a directory
```
$ gzip -r logs/       # compress all files in logs/
```



PRACTICAL OBSERVATION (EDA)

- Use tar to package entire project directories.
- Use tar -czvf to create a compressed backup.
- Use gzip to compress large log / rpt files generated by EDA tools (e.g., synthesis.log.gz).
- Useful for transferring files between servers.
- Saves disk space in shared NFS environments.

KEY POINTS

- tar → archive multiple files/directories.
- gzip → compress individual files.
- Use tar + gzip (.tar.gz) for backup and sharing.
- Use -k option in gzip to keep original file.
- tar -tvf helps to check contents before extraction.
- Common extensions: .tar , .tar.gz , .tgz , .gz

VIVA / INTERVIEW

Q1: What is the difference between tar and gzip ?
→ tar archives files, gzip compresses files.

Q2: Which command do you use to extract .tar.gz file ?
→ tar -xzvf file.tar.gz

$ ln
$ make
$ ls -l
$ cat
$ vi Makefile

LINUX FOR ECE / VLSI & EDA

Module 17

Archives, Compression & Links

~/project
.tar .gz .tgz
ls -ltr
|
.sh
Makefile

51. SOFTLINKS , MAKE FILES
(explain these commands along examples too)

1. SOFT LINKS (Symbolic Links)

- A soft link (symbolic link) is a special file that points to another file or directory using a path.
- It is like a shortcut.
- If the original file is deleted, the link becomes broken (dangling link).
- Useful in EDA projects to maintain common paths to tools, libs, source directories etc.

Syntax

```
ln -s [target]    [link_name]
```

Common Options

Option	Purpose
-s	create symbolic link
-f	force (remove existing link)
-v	verbose output
-n	treat link_name as normal file (useful with -f)

Examples :

a) Create a soft link to a file

```
$ ln -s ../lib/design.v design_link.v
$ ls -l design_link.v
lrwxrwxrwx 1 user group 15 Jan 10 10:00 design_link.v
    → ../lib/design.v
```

b) Create a soft link to a directory

```
$ ln -s /home/user/project common_proj
$ ls -ld common_proj
lrwxrwxrwx 1 user group 23 Jan 10 10:05 common_proj
    → /home/user/project
```

c) Remove a soft link

```
$ rm common_proj
```

d) Check broken link

```
$ ls -l broken_link
broken_link → /path/file (No such file or directory)
```



2. MAKE FILES

- A Makefile contains a set of rules to build a project automatically.
- make checks file dependencies and executes only the required commands (saves time).
- Widely used in Linux / EDA for compiling large projects (C/C++), simulations, reports etc.
- Targets, dependencies and commands are written in the Makefile.
- Command must start with a TAB (not spaces).

Structure / Syntax

```
target : dependencies
        command(s)      ← must be a TAB
```

Example Makefile

```
# Simple Makefile example
all : main.o util.o
        gcc -o myprog main.o util.o

main.o : main.c
        gcc -c main.c

util.o : util.c
        gcc -c util.c

clean :
        rm -f *.o myprog
```

target
↓
dependencies
↓
commands
(TAB indented)

Common make commands

Command	Purpose
make	build default target (usually first)
make <target>	build specified target
make clean	execute clean rule
make -n	show commands (dry run)
make -f <file>	use a different makefile
make -j <N>	parallel build (N jobs)

Practical Observation (EDA)

- Soft links are useful to point to common tool directories (e.g., /tools/syn, /tools/sta).
- Makefiles help automate compile/simulation/report flows.
- Used in large SoC projects to manage dependencies.
- Saves time and avoids manual errors.

Key Points

- Soft link : shortcut to a file/directory (use ln -s).
- If original is deleted → dangling link.
- Makefile : defines rules (target : dependencies).
- Use TAB before commands in Makefile.

Viva / Interview

Q1: What is a soft link?
→ A symbolic link that points to another file/directory.

Q2: What does make do?
→ It checks dependencies and executes only required commands.

52. SHELL PROGRAMMING , BASIC EXAMPLES (commands like /bin , /lib , /boot)

1. SHELL PROGRAMMING

- A shell script is a text file containing a series of Linux commands.
- Executed by a shell (e.g., bash) to automate tasks.
- Very useful in EDA flows – run simulations, setup environments, process logs, etc.
- Scripts help in repetitive tasks and reduce manual errors.

Basic Structure

```
#!/bin/bash    # shebang (interpreter)
# comments start with #
echo "Hello EDA"
pwd
ls -ltr
# more commands ...
```

Execution Steps

1. Create script file : vi run.sh
2. Add execute permission : chmod +x run.sh
3. Run the script : ./run.sh

Example - 1 : Simple script

```
$ vi run.sh
#!/bin/bash
echo "Starting EDA run"
echo "Current directory : `pwd`"
ls -ltr
echo "Run completed"
```

Save & Exit

```
$ chmod +x run.sh
$ ./run.sh
```

Example - 2 : Using variables

```
#!/bin/bash
proj=design
echo "Project name : $proj"
mkdir -p $proj/reports
cd $proj
echo "Created directory $proj/reports"
```

Variables are useful in EDA project automation

Key Points (Shell Scripting)

- Use #! to specify the shell interpreter.
- Give execute permission using chmod +x.
- Variables, loops and conditions can be used for advanced scripts (not covered here).
- Shell scripts are heavily used in EDA environments.

2. IMPORTANT LINUX DIRECTORIES

Directory	Purpose / Contents
/bin	Essential user commands (ls, cp, mv, cat ...) Available to all users
/lib	Shared libraries required by binaries (e.g., libc, ld, ...)
/boot	Boot loader files, Linux kernel (vmlinuz), initrd, configuration files

3. DETAILED EXPLANATION WITH EXAMPLES

3.1 /bin directory

- Contains essential executable commands.
- Needed for system boot and single-user mode.
- Commands: ls, cp, mv, cat, bash, sh, etc.

Example

```
$ ls /bin | head          # list few commands
$ ls -l /bin/ls           # details of a command
-rwxr-xr-x 1 root root 151344 Jan 10 2024 /bin/ls
```

3.2 /lib directory

- Contains shared libraries (.so files).
- These libraries are used by executables in /bin and /sbin.
- Example: libc.so, ld-linux.so, libm.so etc.

Example

```
$ ls /lib | head          # list libraries
$ ls -l /lib/libc.so*     # libc library
lrwxrwxrwx 1 root root 12 Jan 10 2024 /lib/libc.so -> libc.so.6
-rwxr-xr-x 1 root root 1861528 Jan 10 2024 /lib/libc.so.6
```

3.3 /boot directory

- Contains files required for system booting.
- Linux kernel image (vmlinuz), initrd, bootloader (GRUB) configuration files.

Example

```
$ ls /boot                # list boot files
$ ls -l /boot/vmlinuz*    # kernel image
-rwxr-xr-x 1 root root 8456784 Jan 10 2024 /boot/vmlinuz-5.15.0
```

EDA Perspective / Practical Observation

- Shell scripts are used to run tool flows (synthesis, P&R, etc.).
- /bin and /lib must be available for system to run.
- /boot contains the kernel used to start the Linux server.
- Understanding Linux directory structure helps in debugging tool and environment issues.

Viva / Interview Questions

Q1: What is a shell script?
→ A file with Linux commands executed by a shell (e.g., bash).

Q2: What is the purpose of /bin, /lib and /boot?
→ /bin: essential commands, /lib: shared libraries, /boot: kernel files.

$ whoami
$ useradd
$ top
$ df -h

LINUX FOR ECE / VLSI & EDA

Module 19

User Administration & System Performance

~/project
$PATH
.sh .log .rpt
ps | grep
top

53. USER ADMINSTRATION AND SYSTEM PERFORMANCE
(useradd, usermod, groupadd, nice, netstat, etc)

1. INTRODUCTION

- User administration → manage users, groups and permissions.
- System performance → monitor and control system resources (CPU, memory, processes, network etc).
- Useful in multi-user EDA environments (shared servers, tool access, job execution).

2. USER & GROUP MANAGEMENT COMMANDS

Command	Purpose	Example
useradd	Create a new user	useradd vlsiuser
usermod	Modify user properties	usermod -aG eda vlsiuser
userdel	Delete a user	userdel vlsiuser
passwd	Change user password	passwd vlsiuser
groupadd	Create a new group	groupadd eda
groupmod	Modify group properties	groupmod -n pd pd_old
groupdel	Delete a group	groupdel pd
groups	Show groups of a user	groups vlsiuser
users	List current logged in users	users
id	Show user & group ids	id vlsiuser
who	Show who is logged in	who
whoami	Show current logged in user	whoami

3. COMMAND DETAILS & EXAMPLES

3.1 useradd - create a new user

```
# syntax : useradd [options] username
# example : $ sudo useradd vlsiuser
  $ id vlsiuser
  uid=1001(vlsiuser) gid=1001(vlsiuser) groups=1001(vlsiuser)
```

3.2 usermod - modify user

```
# Add user to a group : usermod -aG <group> <user>
  $ sudo usermod -aG eda vlsiuser
  $ groups vlsiuser   → vlsiuser : vlsiuser eda
```

3.3 groupadd - create a group

```
$ sudo groupadd eda
$ getent group eda   → eda:x:1002:
```

3.4 passwd - change password

```
• $ passwd vlsiuser
  New password:
  Retype new password:
  passwd: password updated successfully
```

use sudo for admin commands (login as root or use sudo)

3.5 whoami - current user

```
$ whoami   → tharun
```

3.6 groups / users / who

```
$ groups vlsiuser   → vlsiuser : vlsiuser eda pd
$ users             → tharun ramesh edauser
$ who               →
  tharun   pts/0   2024-01-10 10:15
  ramesh   pts/1   2024-01-10 11:20
```

4. SYSTEM PERFORMANCE COMMANDS

Command	Purpose	Example
top	Real time process monitor	top
ps	Show running processes	ps -ef | grep cadence
nice	Run command with priority	nice -n 10 ./run.sh
renice	Change priority of process	renice 5 -p <pid>
df	Disk space usage	df -h
du	Directory size	du -sh project/
free	Memory usage	free -h
uptime	System up time & load avg	uptime
netstat	Network connections	netstat -an
kill	Terminate a process	kill -9 <pid>

5. EXAMPLES

```
$ top                    # monitor CPU usage
$ ps -ef | grep run.sh   # find process
$ nice -n 10 ./run.sh    # run with lower priority
$ df -h                  # disk usage
$ free -h                # memory usage
$ netstat -an | grep :22 # check if ssh port is open
```

KEY POINTS

- useradd → create new user
- usermod → modify user properties
- groupadd → create new group
- passwd → change password
- whoami / groups / users → get user information
- top / ps / nice / df → monitor system performance
- Essential in EDA server environments for resource sharing.

SAMPLE OUTPUT

```
$ who
  tharun   pts/0   Jan 10 10:15
  ramesh   pts/1   Jan 10 11:20
  edauser  pts/2   Jan 10 12:05

$ df -h
Filesystem  Size  Used  Avail  Use%  Mounted on
/dev/sda1   100G  45G   55G    45%   /
```

PRACTICAL OBSERVATION

- In EDA flows, multiple users share the same server.
- Monitoring CPU, memory and disk usage helps in better job scheduling and avoiding system overload.

VIVA QUESTION

Q. What is the use of nice command?
→ It runs a command with a lower (or higher) scheduling priority.

$ tar
$ gzip
$ gunzip
$ zip
$ unzip

LINUX FOR ECE / VLSI & EDA

Module 20

File Compression

~/project
$PATH
.tar .gz .zip
.bz2 .xz .Z
.rar .7z

54. FILE COMPRESSION
(9 commands with examples)

1. INTRODUCTION

- File compression reduces the file size and helps in storage and faster transfer of data.
- Useful for EDA projects (logs, reports, netlists, etc).
- Two types : (i) Archiving (combine multiple files) → tar, zip
  (ii) Compression (reduce size) → gzip, bzip2, xz, compress

2. WHEN TO USE ?

- Backup project directories
- Transfer files to remote server (scp)
- Save disk space
- Share EDA results / logs
- Maintain old releases

3. IMPORTANT COMPRESSION COMMANDS (9)

No.	Command	Purpose	Syntax	Example	Result / Output
1.	tar	Create an archive of files/directories	tar -cvf <name.tar> <files>	tar -cvf proj.tar /project	proj.tar (archive file)
2.	gzip	Compress a file	gzip <file>	gzip run.log	run.log.gz
3.	gunzip	Decompress .gz file	gunzip <file.gz>	gunzip run.log.gz	run.log
4.	bzip2	Compress using bzip2 (better compression)	bzip2 <file>	bzip2 data.txt	data.txt.bz2
5.	bunzip2	Decompress .bz2 file	bunzip2 <file.bz2>	bunzip2 data.txt.bz2	data.txt
6.	xz	Compress using xz (high compression)	xz <file>	xz netlist.v	netlist.v.xz
7.	unxz	Decompress .xz file	unxz <file.xz>	unxz netlist.v.xz	netlist.v
8.	zip	Create a zip archive	zip <name.zip> <files>	zip reports.zip *.rpt	reports.zip
9.	unzip	Extract zip archive	unzip <file.zip>	unzip reports.zip	Extracted files/directories

4. EXAMPLES (step by step)

4.1 Using tar + gzip (common in EDA)

```
$ tar -cvf project.tar /project/     # create archive
$ gzip project.tar                   # compress to .tar.gz
$ ls -l project.tar.gz               # check file
$ gunzip project.tar.gz              # decompress
$ tar -xvf project.tar               # extract files
```

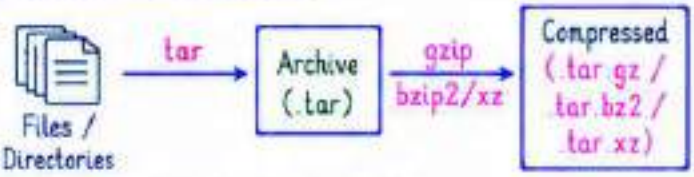
4.2 Using zip / unzip

```
$ zip run_files.zip *.log *.rpt      # create zip
$ unzip run_files.zip                # extract
```

4.3 Using bzip2 / xz

```
$ bzip2 bigfile.log
$ bunzip2 bigfile.log.bz2
$ xz database.txt
$ unxz database.txt.xz
```

5. COMPRESSION FLOW



6. COMMON FILE EXTENSIONS

Extension	Used by	Description
.tar	tar	Archive file (no compression)
.gz	gzip	Compressed file
.bz2	bzip2	Compressed file (better ratio)
.xz	xz	Compressed file (high ratio)
.zip	zip	Archive + compression

KEY POINTS

- Use tar for archiving multiple files.
- gzip is fast and widely used in EDA flows.
- bzip2 / xz provide better compression ratio.
- Use zip/unzip for easy file sharing.
- Always verify files after extraction.

PRACTICAL OBSERVATION

- EDA logs, reports and netlists are often archived and compressed to save disk space.
- Use .tar.gz for project backup before sharing with team / server.

VIVA QUESTIONS

Q. What is the difference between tar and gzip ?
→ tar creates an archive, gzip compresses a file.

Q. Which gives better compression ratio, gzip or bzip2 ?
→ bzip2 (usually high ratio).

$ uname
$ df -h
$ free -m
$ ps
$ top

LINUX FOR ECE / VLSI & EDA

Module 21

Linux Information Commands

~/project
$PATH
.log .rpt
ls -l
host

55. INFORMATION
(5 commands with examples)

1. INTRODUCTION

- Linux provides many information commands to understand system, hardware, memory, disk, processes and environment.
- These commands are very useful in EDA environments to check system status before running heavy jobs.
- Helps in debugging, performance monitoring and capacity planning.

2. USE CASES IN EDA

- Check server configuration before running tools
- Monitor memory & CPU while running synthesis / P&R
- Verify disk space in project / scratch directories
- Identify heavy processes consuming resources
- Useful for job debugging in shared servers (LSF)

3. IMPORTANT INFORMATION COMMANDS

No.	Command	Purpose	Syntax (common options)	Example	Sample Output (short)
1	uname	Display OS and system information	uname -a uname -r	uname -a	Linux server 5.15.0-92--generic x86_64 ...
2	hostname	Show system host name	hostname	hostname	eda-server-01
3	df	Display disk space usage (filesystem)	df -h df -T	df -h	Filesystem Size Used Avail Use% /dev/sda1 100G 40G 60G 40%
4	free	Show memory usage	free -m free -h	free -m	Mem: 32000 28000 4000 ... Swap: 8000 1000 7000 ...
5	ps	Display running processes	ps -ef ps aux | grep <name>	ps -ef | grep innovus	user 12345 1 0 10:23 ? innovus ...

4. EXAMPLES

4.1 uname
```
$ uname -a          # kernel + OS info
$ uname -r          # kernel version
```

4.2 hostname
```
$ hostname          # show host name
```

4.3 df
```
$ df -h             # human readable
$ df -T             # filesystem type
```

4.4 free
```
$ free -m           # memory in MB
$ free -h           # human readable
```

4.5 ps
```
$ ps -ef                    # all processes
$ ps aux | grep innovus     # search process
```

5. SAMPLE OUTPUTS

```
$ uname -a
Linux eda01 5.15.0-92-generic x86_64 GNU/Linux
```

```
$ hostname
eda-server-01
```

```
$ df -h
Filesystem      Size  Used  Avail  Use%
/dev/sda1       100G  40G   60G    40%
/dev/sdb1       500G  200G  300G   40%
```

```
$ free -m
              total     used     free     shared
Mem:          32000     28000    4000     500
Swap:         8000      1000     7000
```

```
$ ps aux | grep innovus
user     PID   %CPU %MEM  VSZ    RSS   TTY  STAT CMD
tharun   12345 95.0 10.2  8200m  1.2g  ?    R    innovus
```

KEY POINTS

- Use df -h to check disk space before running EDA tools.
- Use free -m to monitor memory usage.
- Use ps to identify heavy processes.
- uname and hostname help in system identification.
- These commands are read-only and safe to use.

EDA OBSERVATION

- Before running synthesis / P&R, check disk space using df -h
- Monitor memory using free -m
- Use ps to identify and kill stuck processes if required.
- Helps in smooth and efficient EDA tool execution.

VIVA QUESTIONS

Q. Which command shows memory usage ?
→ free -m

Q. Which command shows disk space usage ?
→ df -h

$ ssh
$ ping
$ mail
$ who
$ write

LINUX FOR ECE / VLSI & EDA

Module 22

Network & User Communication

~/project
$PATH
.log .rpt
ls -l
host

56. NETWORK AND MESSAGE B/W USERS
(9 + 7 commands with examples)

1. OVERVIEW

- Linux provides commands to communicate over network and also exchange messages between users.
- Useful in multi-user EDA environments (shared servers).
- Helps in remote login, file transfer, monitoring and sending alerts/notifications.

2. TYPES OF COMMANDS COVERED

- Network related commands (9)
  → check connectivity, remote login, file transfer, network info etc.
- User communication commands (7)
  → send messages, check logged in users, broadcast messages etc.

3. NETWORK COMMANDS (9)

No.	Command	Purpose	Example (syntax)
1	ping	Check host connectivity	ping <host> e.g. ping google.com
2	ssh	Remote login to server	ssh user@host e.g. ssh user@192.168.1.10
3	scp	Secure copy files b/w systems	scp file user@host:dir/ e.g. scp run.log user@server:/home
4	ftp	File transfer (FTP)	ftp <host> e.g. ftp 192.168.1.10
5	sftp	Secure file transfer	sftp user@host e.g. sftp user@server
6	telnet	Login to remote host (older)	telnet <host> e.g. telnet server
7	ifconfig	Display network interfaces	ifconfig e.g. ifconfig eth0
8	netstat	Show network connections/ports	netstat -an e.g. netstat -an | grep 22
9	nslookup	DNS lookup	nslookup <domain> e.g. nslookup google.com

4. USER COMMUNICATION COMMANDS (7)

No.	Command	Purpose	Example (syntax)
1	mail	Send email to user	mail user e.g. mail tharun (type message, Ctrl+D)
2	write	Send message to logged in user	write user e.g. write tharun "Job completed"
3	wall	Broadcast message to all users	wall "Maintenance at 10 PM"
4	who	Show logged in users	who e.g. who
5	w	Show users and what they are doing	w e.g. w
6	last	Show recent logins	last e.g. last -5
7	mesg	Control message permission	mesg y (allow) mesg n (deny) e.g. mesg n

5. EXAMPLE SESSIONS

5.1 ping example

```
$ ping google.com
PING google.com (142.250.72.14) 56(84) bytes.
64 bytes from 142.250.72.14: icmp_seq=1 ttl=115...
```

5.2 ssh example

```
$ ssh user@server
user@server's password:
Last login: Tue Jun 10 10:20:11 2024
[user@server ~]$
```

5.3 mail example

```
$ mail tharun
Subject: Run Status
EDA job completed successfully.
Ctrl+D (to send)
```

6. NETWORK COMMUNICATION



Common Ports

SSH	→ 22
FTP	→ 21
SFTP	→ 22
Telnet	→ 23

KEY POINTS

- Use ping to check connectivity.
- Use ssh/scp/sftp for secure access.
- Use mail, write, wall for user messages.
- Use who/w/last to check users.
- Useful in EDA shared server environments.

7. PRACTICAL OBSERVATIONS (EDA)

- Use ssh to login to remote compute servers.
- Use scp/sftp to transfer design files, logs, reports.
- Use wall to notify all users before server maintenance.
- Network commands are essential for distributed EDA flows and job monitoring.

8. VIVA / INTERVIEW QUESTIONS

Q1. Which command is used to send a message to a specific user ?
→ write user

Q2. What is the use of ping ?
→ To check network connectivity between local and remote host.

$ ssh
$ ping
$ mail
$ wrte
$ write

LINUX FOR ECE / VLSI & EDA

Module 23

Programming Utilities

~/project
$PATH
.sh .log
.rpt
ls -l

57. PROGRAMMING UTILITIES

(16 commands : 9 Network + 7 User Communication)

1. OVERVIEW

- Programming utilities in Linux help in network interaction and communication between users on local/remote systems.
- Useful for remote login, file transfer, monitoring network and sending messages in multi-user EDA environments.
- These commands are essential for collaboration, debugging on servers and managing distributed flows.

2. APPLICATIONS (EDA)

- Login to remote EDA servers using ssh/telnet.
- Transfer design files / logs using scp, sftp, ftp.
- Check connectivity to license / compute servers using ping.
- Communicate with team members using mail, write, wall.
- Useful in remote debugging, job monitoring and maintenance.

3. NETWORK RELATED COMMANDS (9)

No.	Command	Purpose	Example (syntax)
1	ping	Check host reachability	ping google.com
2	ssh	Secure remote login	ssh user@server
3	scp	Secure copy files	scp file user@server:/path
4	sftp	Interactive secure file transfer	sftp user@server
5	ftp	File transfer (FTP)	ftp server
6	telnet	Remote login (older)	telnet server
7	ifconfig	Display network configuration	ifconfig e.g. ifconfig eth0
8	netstat	Show network connections/ports	netstat -an
9	nslookup	DNS lookup	nslookup google.com

4. USER COMMUNICATION COMMANDS (7)

No.	Command	Purpose	Example (syntax)
1	mail	Send email to user	mail user
2	write	Send message to logged in user	write user "Hello"
3	wall	Broadcast message to all users	wall "System maintenance at 10 PM"
4	who	Show logged in users	who
5	w	Show users and what they are doing	w
6	mesg	Control message permission	mesg y (allow) mesg n (deny)
7	talk	Real time chat (between users)	talk user

5. EXAMPLES

5.1 ping example

```
$ ping google.com
PING google.com (142.250.72.14) 56(84) bytes of data.
64 bytes from 142.250.72.14: icmp_seq=1 ttl=115 ...
```

5.2 ssh example

```
$ ssh user@192.168.1.10
user@192.168.1.10's password:
Last login: Tue Jun 10 10:20:11 2024
```

5.3 scp example

```
$ scp design.v user@server:/home/user/
design.v   100%   10KB   0.1s
```

5.4 mail example

```
$ mail tharun
Subject: Meeting
Let's discuss the run results tomorrow.
Ctrl+D (to send)
```

6. SIMPLE NETWORK FLOW



Common Use Cases

- ssh → remote login
- scp/sftp → file transfer
- ping → check connectivity
- mail/write → send message
- wall → broadcast to all
- who/w → check users

KEY POINTS

- Use ssh for secure remote access.
- Use scp/sftp for transferring files.
- Use mail/write for user communication.
- Use ping/nslookup to check network issues.
- Useful in multi-user EDA server environments.

7. PRACTICAL OBSERVATIONS

- ssh and scp are widely used in EDA workflows to access servers and move large design files.
- wall is useful during maintenance windows.
- netstat helps in checking if license servers are reachable (port status).
- Always prefer ssh instead of telnet (secure).
- Use mesg n if you don't want to receive messages.

8. VIVA / INTERVIEW QUESTIONS

Q1. Which command is used to copy a file to a remote server ?
→ scp

Q2. How to send a message to all logged in users ?
→ wall

Q3. Which command is used to check network connectivity to a host ?
→ ping

$ pwd
$ date
$ uname
$ which
$ man

LINUX FOR ECE / VLSI & EDA

Module 24

Miscellaneous Linux Commands

~/project
$PATH
.sh .log
.rpt
ls -l

58. MISC COMMANDS

(40 useful commands with examples)

No.	Command	Purpose	Example
1	date	Show current date & time	date
2	cal	Display calendar	cal 2024
3	uname	System information	uname -a
4	hostname	Show system hostname	hostname
5	which	Locate command path	which gcc
6	whereis	Find binary/man/source	whereis python
7	man	Show manual page	man ls
8	info	GNU info documentation	info gcc
9	echo	Print message / variable	echo $PATH
10	env	Display environment vars	env
11	printenv	Print specific variable	printenv HOME
12	alias	Create command alias	alias ll='ls -l'
13	unalias	Remove alias	unalias ll
14	history	Show command history	history
15	clear	Clear terminal screen	clear
16	df	Disk space usage	df -h
17	du	Directory space usage	du -sh project
18	free	Show memory usage	free -h
19	uptime	System up time	uptime
20	whoami	Current logged in user	whoami

No.	Command	Purpose	Example
21	id	User and group details	id
22	groups	Show group memberships	groups
23	chmod	Change file permissions	chmod 755 run.sh
24	chown	Change file owner	chown user file
25	chgrp	Change file group	chgrp team file
26	umask	Default permission mask	umask
27	find	Search for files	find . -name "*.log"
28	locate	Fast file search (db)	locate design.v
29	grep	Search text pattern	grep "ERROR" log
30	sort	Sort lines of text	sort file.txt
31	wc	Count lines/words/bytes	wc -l file.txt
32	head	Show first lines	head -n 10 file.txt
33	tail	Show last lines	tail -n 10 log.txt
34	diff	Compare two files	diff file1 file2
35	cmp	Compare files (binary)	cmp file1 file2
36	cut	Extract columns	cut -d" " -f1 file
37	paste	Merge files horizontally	paste f1 f2
38	tr	Translate/delete chars	tr 'a-z' 'A-Z'
39	tee	Output to screen & file	ls -l | tee out.txt
40	xargs	Build command from input	find . -name "*.c" | xargs grep "main"

1. NOTES

- Misc commands are very useful for system information, file search, text processing and environment handling
- These commands help in daily Linux usage, debugging and EDA server management
- Use man <command> to learn more about any command.

2. KEY POINTS

- Use man / info for help
- which / whereis to locate binaries
- df / du / free to monitor system
- find / locate / grep to search files
- head / tail / sort / wc for text ops
- alias to save time
- Useful for EDA workflows, log analysis and server maintenance

3. PRACTICAL OBSERVATION

- Commands like grep, find, df, free are heavily used in EDA environments for log/debug, storage and performance checks
- Aliases and history improve productivity on remote servers

4. VIVA / INTERVIEW QUESTIONS

Q1. Which command is used to view system information (kernel) ?
→ uname

Q2. How to find a file named test.log in current directory ?
→ find . -name "test.log"

Q3. Which command is used to see memory usage ?
→ free -h

Q4. How to search for a string in a file ?
→ grep "string" filename

Q5. Which command displays command history ?
→ history

Learn | Practice | Explore | Build

```
/user@linux:~$ echo "Keep Learning!"
Keep Learning!
user@linux:~$
# Small Commands Big Opportunities !
```



LINUX

LEARN
PRACTICE
MASTER

Open Source
Open Future

Linux

Build | Automate | Innovate

"Great Engineers use Linux !"

SHELL
SCRIPTING
AUTOMATION
PRODUCTIVITY

learn
practice
explore
script
automate

THANK YOU

configure
troubleshoot
develop
grow
succeed

"From Commands to Possibilities"



APPLY KNOWLEDGE → AUTOMATE TASKS → IMPROVE SKILLS → ACHIEVE BIGGER GOALS

Keep Exploring Linux !

Same Terminal Bigger Dreams !

Thank you !

Keep Learning • Keep Scripting
Keep Growing • Keep Innovating

Linux Today
Better Engineer Tomorrow
Limitless Opportunities

Linux • Skills • Automation • Career • Future